

Babel

Code

Version 26.13.137049
2026/10/01

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	$\LaTeX$: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	16
4.2	Errors	24
4.3	More on selection	25
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	28
4.8	Shorthands	30
4.9	Language attributes	39
4.10	Support for saving and redefining macros	40
4.11	French spacing	42
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	49
4.15	Making glyphs available	50
4.15.1	Quotation marks	50
4.15.2	Letters	52
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	55
4.17	Load engine specific macros	55
4.18	Creating and modifying languages	55
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	68
4.21	French spacing (again)	74
4.22	Handle language system	75
4.23	Numerals	76
4.24	Casing	77
4.25	Getting info	78
4.26	BCP 47 related commands	79
5	Adjusting the Babel behavior	80
5.1	Cross referencing macros	83
5.2	Layout	85
5.3	Marks	87
5.4	Other packages	88
5.4.1	ifthen	88
5.4.2	varioref	88
5.4.3	hhline	89
5.5	Encoding and fonts	89
5.6	Basic bidi support	91
5.7	Local Language Configuration	94
5.8	Language options	95

6	Tagging	99
7	The kernel of Babel	99
8	Error messages	99
9	Loading hyphenation patterns	103
10	luatex + xetex: common stuff	107
11	Hooks for XeTeX and LuaTeX	111
11.1	XeTeX	111
11.2	Support for interchar	112
11.3	Layout	114
11.4	8-bit TeX	116
11.5	LuaTeX	117
11.6	Southeast Asian scripts	124
11.7	CJK line breaking	125
11.8	Arabic justification	127
11.9	Common stuff	131
11.10	Automatic fonts and ids switching	131
11.11	Bidi	138
11.12	Layout	141
11.13	Lua: transforms	150
11.14	Lua: Auto bidi with basic and basic-r	160
12	Data for CJK	173
13	The ‘nil’ language	173
14	Calendars	174
14.1	Islamic	174
14.2	Hebrew	176
14.3	Persian	180
14.4	Coptic and Ethiopic	180
14.5	Julian	181
14.6	Buddhist	182
15	Support for Plain TeX (plain.def)	183
15.1	Not renaming hyphen.tex	183
15.2	Emulating some L ^A T _E X features	184
15.3	General tools	184
15.4	Encoding related macros	188
16	Acknowledgements	191

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `{(name=value)}`, or with a series of lines between `{(*name)}` and `{(/name)}`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<version=26.13.137049>>
2 <<date=2026/10/01>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<(*Basic macros)>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8   {\def#1{#2}}%
9   {\expandafter\def\expandafter#1\expandafter{#1#2}}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@carg#1{\csname bbl@#1\endcsname}
17 \def\bbl@carg#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
```

```

19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \<.> for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35     \let\<\noexpand
36     \let\<\bbl@exp@en
37     \let\[\bbl@exp@ue
38     \edef\bbl@exp@aux{\endgroup#1}%
39     \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\@sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrode}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{ }%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@\expandafter{\the\toks@##1}%
120     \else
121       \toks@\expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize@undefined\else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax{%
129     \def\bbl@tempa{#1}%
130     \def\bbl@tempb{#2}%
131     \def\bbl@tempe{#3}}
132   \def\bbl@sreplace#1#2#3{%
133     \begingroup
134       \expandafter\bbl@parsedef\meaning#1\relax
135       \def\bbl@tempc{#2}%
136       \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137       \def\bbl@tempd{#3}%
138       \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139       \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140       \ifin@
141         \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142         \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143           \\makeatletter % "internal" macros with @ are assumed
144           \\scantokens{%
145             \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146             \noexpand\noexpand}%
147           \catcode64=\the\catcode64\relax}% Restore @
148       \else
149         \let\bbl@tempc@empty % Not \relax
150       \fi
151       \bbl@exp{% For the 'uplevel' assignments
152     \endgroup
153     \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bb@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bb@tempb{#1}%
158   \edef\bb@tempb{\expandafter\strip@prefix\meaning\bb@tempb}%
159   \protected@edef\bb@tempc{#2}%
160   \edef\bb@tempc{\expandafter\strip@prefix\meaning\bb@tempc}%
161   \ifx\bb@tempb\bb@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bb@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bb@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bb@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bb@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal \let's made by \MakeUppercase and \MakeLowercase between things like \oe and \OE.

```

184 \def\bb@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bb@afterelse\expandafter\MakeUppercase
190   \else
191     \bb@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

Ordered key/value lists (aka associative arrays). The setter either replaces the current value if the key exists, or adds the new key/value at the end. The getter returns either the value if the key exists or \relax. The code is based on arguments delimited by the keys. In the getter, note the value is supposed not to start with \@empty.

```

196 \def\bb@array@set#1#2#3{%
197   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux##4{%
198     \ifx\bb@noelt##2%
199       \def#1{##1\bb@elt#2\@{##4}}%
200     \else
201       \def\bb@array@aux####1\bb@elt#2\@####2####3\bb@array@aux####4{%
202         \def#1{####1\bb@elt#2\@{####4}####3}}%
203       \expandafter\bb@array@aux#1\bb@array@aux{#3}%
204     \fi}%
205   \expandafter\bb@array@aux#1\bb@elt#2\@{\bb@noelt\bb@array@aux{#3}}
206 \def\bb@array@get#1#2#3{%
207   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux{%
208     \ifx\@empty##2\let#3\relax\else\def#3{##2}\fi}%

```

```
209 \expandafter\bbbl@array@aux#1\bbbl@elt#2\@empty\bbbl@array@aux}
```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with #'s. Used to deal with alph, Alph and frenchspacing when there are already changes (with `\babel@save`).

```
210 \def\bbbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
211 \toks@{\expandafter\expandafter\expandafter{%
212 \cscname extras\language\endcsname}%
213 \bbbl@exp{\in@{#1}{\the\toks@}}%
214 \ifin@else
215 \@temptokena{#2}%
216 \edef\bbbl@tempc{\the\@temptokena\the\toks@}%
217 \toks@\expandafter{\bbbl@tempc#3}%
218 \expandafter\edef\cscname extras\language\endcsname{\the\toks@}%
219 \fi}
220 {/Basic macros}}
```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.

```
221 {(*Make sure ProvidesFile is defined)} ≡
222 \ifx\ProvidesFile\@undefined
223 \def\ProvidesFile#1[#2 #3 #4]{%
224 \wlog{File: #1 #4 #3 <#2>}%
225 \let\ProvidesFile\@undefined}
226 \fi
227 {/Make sure ProvidesFile is defined}}
```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```
228 {(*Define core switching macros)} ≡
229 \ifx\language\@undefined
230 \cscname newcount\endcsname\language
231 \fi
232 {/Define core switching macros}}
```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```
233 {(*Define core switching macros)} ≡
234 \countdef\last@language=19
235 \def\addlanguage{\cscname newlanguage\endcsname}
236 {/Define core switching macros}}
```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```
237 {*package}
238 \NeedsTeXFormat{LaTeX2e}
239 \ProvidesPackage{babel}%
240 [%@date@> v<@version@>
241 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]
```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

242 \ifpackagewith{babel}{debug}
243 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
244 \let\bbl@debug\@firstofone
245 \ifx\directlua\@undefined\else
246 \directlua{
247     Babel = Babel or {}
248     Babel.debug = true }%
249 \input{babel-debug.tex}%
250 \fi}
251 {\providecommand\bbl@trace[1]{}%
252 \let\bbl@debug\@gobble
253 \ifx\directlua\@undefined\else
254 \directlua{
255     Babel = Babel or {}
256     Babel.debug = false }%
257 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

258 \def\bbl@error#1{% Implicit #2#3#4
259 \begingroup
260 \catcode`\=0 \catcode`\==12 \catcode`\`=12
261 \input errbabel.def
262 \endgroup
263 \bbl@error{#1}}
264 \def\bbl@warning#1{%
265 \begingroup
266 \def\{\{MessageBreak}%
267 \PackageWarning{babel}{#1}%
268 \endgroup}
269 \def\bbl@infowarn#1{%
270 \begingroup
271 \def\{\{MessageBreak}%
272 \PackageNote{babel}{#1}%
273 \endgroup}
274 \def\bbl@info#1{%
275 \begingroup
276 \def\{\{MessageBreak}%
277 \PackageInfo{babel}{#1}%
278 \endgroup}

```

Many of the following options don’t do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

279 <@Basic macros>
280 \ifpackagewith{babel}{silent}
281 {\let\bbl@info\@gobble
282 \let\bbl@infowarn\@gobble
283 \let\bbl@warning\@gobble}
284 {}
285 %
286 \def\AfterBabelLanguage#1{%
287 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

288 \ifx\bbl@languages\@undefined\else
289 \begingroup
290 \catcode`\^^I=12
291 \@ifpackagewith{babel}{showlanguages}{%
292 \begingroup
293 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%

```

```

294      \wlog{<*languages>}%
295      \bbl@languages
296      \wlog{</languages>}%
297      \endgroup{}}
298  \endgroup
299  \def\bbl@elt#1#2#3#4{%
300    \ifnum#2=\z@
301      \gdef\bbl@nulllanguage{#1}%
302      \def\bbl@elt##1##2##3##4{%
303        \fi}%
304    \bbl@languages
305  \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```

306 \bbl@trace{Defining option 'base'}
307 \ifpackagewith{babel}{base}{%
308   \let\bbl@onlyswitch\@empty
309   \let\bbl@provide@locale\relax
310   \input babel.def
311   \let\bbl@onlyswitch\@undefined
312   \ifx\directlua\@undefined
313     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
314   \else
315     \input luababel.def
316     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
317   \fi
318   \DeclareOption{base}{}%
319   \DeclareOption{showlanguages}{}%
320   \ProcessOptions
321   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
322   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
323   \global\let\@ifl@ter@\@ifl@ter
324   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
325   \endinput}{}%

```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```

326 \bbl@trace{key=value and another general options}
327 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
328 \def\bbl@tempb#1.#2{% Removes trailing dot
329   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
330 \def\bbl@tempe#1=#2\@{
331   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
332 \def\bbl@tempd#1.#2\@nnil{%
333   \ifx\@empty#2%
334     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
335   \else
336     \in@{,provide=}{, #1}%
337     \ifin@
338       \edef\bbl@tempc{%
339         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
340     \else
341       \in@{$modifiers$}{$#1$}%

```

```

342 \ifin@
343 \bbl@tempe#2\@@
344 \else
345 \in@{=}{#1}%
346 \ifin@
347 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
348 \else
349 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
350 \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
351 \fi
352 \fi
353 \fi
354 \fi}
355 \let\bbl@tempc\@empty
356 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
357 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

358 \DeclareOption{KeepShorthandsActive}{}
359 \DeclareOption{activeacute}{}
360 \DeclareOption{activegrave}{}
361 \DeclareOption{debug}{}
362 \DeclareOption{debug-german}{} % Temporary
363 \DeclareOption{noconfigs}{}
364 \DeclareOption{showlanguages}{}
365 \DeclareOption{silent}{}
366 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
367 \chardef\bbl@iniflag\z@
368 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
369 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
370 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
371 \chardef\bbl@ldfflag\z@
372 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
373 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
374 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
375 \DeclareOption{licr=extended}{}
376 \DeclareOption{licr=unextended}{}
377 \let\bbl@ifmetadata\@gobble
378 \DeclareOption{babeltagging=on}{\input{tagbabel.def}}
379 % Don't use. Experimental.
380 \newif\ifbbl@single
381 \DeclareOption{selectors=off}{\bbl@singletrue}
382 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key `main`, and the third one loads the latter. First, we “flag” valid keys with a `nil` value.

```

383 \let\bbl@opt@shorthands\@nnil
384 \let\bbl@opt@config\@nnil
385 \let\bbl@opt@main\@nnil
386 \let\bbl@opt@headfoot\@nnil
387 \let\bbl@opt@layout\@nnil
388 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

389 \def\bbl@tempa#1=#2\bbl@tempa{%
390 \bbl@csarg\ifx{opt@#1}\@nnil
391 \bbl@csarg\edef{opt@#1}{#2}%
392 \else
393 \bbl@error{bad-package-option}{#1}{#2}{}%
394 \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```

395 \let\bbl@language@opts\@empty
396 \DeclareOption*{%
397   \bbl@xin@{\string=}{\CurrentOption}%
398   \ifin@
399     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
400   \else
401     \bbl@add@list\bbl@language@opts{\CurrentOption}%
402   \fi}

```

Now we finish the first pass (and start over).

```

403 \ProcessOptions*

```

3.5. Post-process some options

```

404 \ifx\bbl@opt@provide\@nnil
405   \let\bbl@opt@provide\@empty % %%% MOVE above
406 \else
407   \chardef\bbl@iniflag\@ne
408   \bbl@exp{\\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
409     \in@{,provide,}{, #1,}%
410     \ifin@
411       \def\bbl@opt@provide{#2}%
412     \fi}
413 \fi

```

If there is no `shorthands=` (*chars*), the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```

414 \bbl@trace{Conditional loading of shorthands}
415 \def\bbl@sh@string#1{%
416   \ifx#1\@empty\else
417     \ifx#1t\string~%
418     \else\ifx#1c\string,%
419     \else\string#1%
420   \fi\fi
421   \expandafter\bbl@sh@string
422 \fi}
423 \ifx\bbl@opt@shorthands\@nnil
424   \def\bbl@ifshorthand#1#2#3{#2}%
425 \else\ifx\bbl@opt@shorthands\@empty
426   \def\bbl@ifshorthand#1#2#3{#3}%
427 \else

```

The following macro tests if a shorthand is one of the allowed ones.

```

428 \def\bbl@ifshorthand#1{%
429   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
430   \ifin@
431     \expandafter\@firstoftwo
432   \else
433     \expandafter\@secondoftwo
434   \fi}

```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```

435 \edef\bbl@opt@shorthands{%
436   \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```

437 \bbl@ifshorthand{'}%
438 {\PassOptionsToPackage{activeacute}{babel}}{}

```

```

439 \bbl@ifshorthand{`}%
440 {\PassOptionsToPackage{activegrave}{babel}}{}
441 \fi\fi

```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\resetactivechars`, but seems to work.

```

442 \ifx\bbl@opt@headfoot\@nnil\else
443 \g@addto@macro\@resetactivechars{%
444 \set@typeset@protect
445 \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
446 \let\protect\noexpand}
447 \fi

```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to none.

```

448 \ifx\bbl@opt@safe\@undefined
449 \def\bbl@opt@safe{BR}
450 % \let\bbl@opt@safe\@empty % Pending of \cite
451 \fi

```

For layout an auxiliary macro is provided, available for packages and language styles. Optimization: if there is no layout, just do nothing.

```

452 \bbl@trace{Defining IfBabelLayout}
453 \ifx\bbl@opt@layout\@nnil
454 \newcommand\IfBabelLayout[3]{#3}%
455 \else
456 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
457 \in@{,layout,}{, #1,}%
458 \ifin@
459 \def\bbl@opt@layout{#2}%
460 \bbl@replace\bbl@opt@layout{ }{.}%
461 \fi}
462 \newcommand\IfBabelLayout[1]{%
463 \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
464 \ifin@
465 \expandafter\@firstoftwo
466 \else
467 \expandafter\@secondoftwo
468 \fi}
469 \fi
470 \<package>

```

3.6. Plain: `babel.def` (start)

Because of the way `docstrip` works, we need to insert some code for Plain here. However, the tools provided by the `babel` installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

471 \<core>
472 \ifx\ldf@quit\@undefined\else
473 \endinput\fi % Same line!
474 \<@Make sure ProvidesFile is defined@>
475 \ProvidesFile{babel.def}[\<@date@> v\<@version@> Babel common definitions]
476 \ifx\AtBeginDocument\@undefined
477 \<@Emulate LaTeX@>
478 \fi
479 \<@Basic macros@>
480 \</core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\text{\LaTeX}$. After it, we will resume the $\text{\LaTeX}$ -only stuff.

4. `babel.sty` and `babel.def` (common)

```

481 (*package|core)
482 \def\bbl@version{<@version>}
483 \def\bbl@date{<@date>}
484 <@Define core switching macros>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

485 \def\adddialect#1#2{%
486   \global\chardef#1#2\relax
487   \bbl@usehooks{adddialect}{#1}{#2}}%
488 \begingroup
489   \count@#1\relax
490   \def\bbl@elt##1##2##3##4{%
491     \ifnum\count@=##2\relax
492       \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
493       \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
494         set to \expandafter\string\csname l@##1\endcsname\%
495         (\string\language\the\count@). Reported}%
496       \def\bbl@elt####1####2####3####4{%
497         \fi}%
498       \bbl@cs{languages}%
499       \endgroup}

```

`\bbl@iflanguage` executes code only if the language `l@` exists. Otherwise raises an error.

The argument of `\bbl@fixname` has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when `\foreignlanguage` and the like appear in a `\MakeXXXcase`. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note `l@` is encapsulated, so that its case does not change.

```

500 \def\bbl@fixname#1{%
501   \begingroup
502   \def\bbl@tempe{l@}%
503   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
504   \bbl@tempd
505     {\lowercase\expandafter{\bbl@tempd}%
506      {\uppercase\expandafter{\bbl@tempd}%
507       \@empty
508        {\edef\bbl@tempd{\def\noexpand#1{#1}}%
509         \uppercase\expandafter{\bbl@tempd}}}%
510       {\edef\bbl@tempd{\def\noexpand#1{#1}}%
511        \lowercase\expandafter{\bbl@tempd}}}%
512   \@empty
513   \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
514   \bbl@tempd
515   \bbl@expf{\bbl@usehooks{language}{\language}{#1}}}%
516 \def\bbl@iflanguage#1{%
517   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that `sr-latn-ba` becomes `fr-Latn-BA`.

`\bbl@bcpllookup` returns `\bbl@bcp` with either the found ini tag or `\relax`. Temporary version, to be refactored with the new `\text_bcp_parse:n`, which is not currently fully expandable with invalid BCP 47 tags. The argument may be in some cases either a language name or a tag (e.g., `\foreignlanguage`). This was a huge mistake, because of the potential ambiguities (for example, the name `vai-latin` is also a syntactically valid tag).

```

518 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
519 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
520 \def\bbl@bcpllookup#1{%
521   \let\bbl@templ\@empty % language
522   \let\bbl@temps\@empty % script
523   \let\bbl@tempr\@empty % regions
524   \let\bbl@tempv\@empty % variant

```

```

525 \edef\bbbl@tempa{#1}%
526 \bbbl@replace\bbbl@tempa{-}{,}%
527 \bbbl@replace\bbbl@tempa_{}{,}%
528 \bbbl@foreach\bbbl@tempa{%
529   \ifx\bbbl@templ\@empty
530     \lowercase{\def\bbbl@templ{##1}}%
531   \else\ifnum\bbbl@cntchars{##1}=4
532     \bbbl@xin@{\@car##1\@nil}{0123456789}%
533     \ifin@
534       \def\bbbl@tempv{##1}% eg, 1901
535     \else
536       \uppercase{\edef\bbbl@tempb{\@car##1\@nil}}%
537       \lowercase{\edef\bbbl@tempb{\bbbl@tempb\@gobble##1}}%
538     \fi
539   \else\ifnum\bbbl@cntchars{##1}<4
540     \uppercase{\def\bbbl@tempv{##1}}%
541   \else\ifnum\bbbl@cntchars{##1}>4
542     \lowercase{\def\bbbl@tempv{##1}}%
543   \fi\fi\fi\fi}%
544 \bbbl@expf{\bbbl@bcpllookup@i{\bbbl@templ}{\bbbl@tempb}{\bbbl@tempv}}%
545 \def\bbbl@bcpllookup@try#1{%
546   \ifx\bbbl@bcp\relax
547     \IfFileExists{babel-#1\bbbl@tempe.ini}{\edef\bbbl@bcp{#1\bbbl@tempe}}{}%
548 \fi}
549 \def\bbbl@bcpllookup@i#1#2#3#4{%
550   \let\bbbl@bcp\relax
551   \def\bbbl@tempe{#4}%
552   \ifx\bbbl@tempe\@empty\else
553     \edef\bbbl@tempe{-#4}%
554   \fi
555   \edef\bbbl@tempa{#2#3\bbbl@tempe}% en
556   \ifx\bbbl@tempa\@empty
557     \bbbl@bcpllookup@try{#1}%
558   \else
559     \edef\bbbl@tempa{#3\bbbl@tempe}% en-Latn
560     \ifx\bbbl@tempa\@empty
561       \bbbl@bcpllookup@try{#1-#2}%
562       \bbbl@bcpllookup@try{#1}%
563     \else
564       \edef\bbbl@tempa{#2\bbbl@tempe}% en-US, en-001
565       \ifx\bbbl@tempa\@empty
566         \bbbl@bcpllookup@try{#1-#3}%
567         \bbbl@bcpllookup@try{#1}%
568       \else % en-Latn-US
569         \bbbl@bcpllookup@try{#1-#2-#3}%
570         \bbbl@bcpllookup@try{#1-#2}%
571         \bbbl@bcpllookup@try{#1-#3}%
572         \bbbl@bcpllookup@try{#1}%
573       \fi
574     \fi
575   \fi
576   \ifx\bbbl@bcp\relax
577     \ifx\bbbl@tempe\@empty\else\bbbl@bcpllookup@i{#1}{#2}{#3}\fi%
578 \fi}
579 %
580 \let\bbbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

581 \def\iflanguage#1{%

```

```

582 \bbl@iflanguage{#1}{%
583   \ifnum\cscname l@#1\endcsname=\language
584     \expandafter\@firstoftwo
585   \else
586     \expandafter\@secondoftwo
587   \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

588 \let\bbl@select@type\z@
589 \edef\selectlanguage{%
590   \noexpand\protect
591   \expandafter\noexpand\cscname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage_`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.

```

592 \ifx\@undefined\protect\let\protect\relax\fi

```

The following definition is preserved for backwards compatibility (e.g., `arabi`, `koma`). It is related to a trick for 2.09, now discarded.

```

593 \let\xstring\string

```

Since version 3.5 `babel` writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's `aftergroup` mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bbl@pop@language` to be executed at the end of the group. It calls `\bbl@set@language` with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bbl@language@stack` and initially empty.

```

594 \def\bbl@language@stack{}

```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```

595 \def\bbl@push@language{%
596   \ifx\language\@undefined\else
597     \ifx\currentgrouplevel\@undefined
598       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
599     \else
600       \ifnum\currentgrouplevel=\z@
601         \xdef\bbl@language@stack{\language+}%
602       \else
603         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
604       \fi
605     \fi
606   \fi}

```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro `\language`. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the ‘+’-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
607 \def\bbl@pop@lang#1+#2\@@{%
608   \edef\language{#1}%
609   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed T_EX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a ‘+’-sign (zero language names won’t occur as this macro will only be called after something has been pushed on the stack).

```
610 \let\bbl@ifrestoring\@secondoftwo
611 \def\bbl@pop@language{%
612   \expandafter\bbl@pop@lang\bbl@language@stack\@@
613   \let\bbl@ifrestoring\@firstoftwo
614   \expandafter\bbl@set@language\expandafter{\language}%
615   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to \bbl@set@language to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of \localeid. This means \l@. . . will be reserved for hyphenation patterns (so that two locales can share the same rules).

```
616 \chardef\localeid\z@
617 \gdef\bbl@id@last{0}    % No real need for a new counter
618 \def\bbl@id@assign{%
619   \bbl@ifunset\bbl@id@\language}%
620   {\count@\bbl@id@last\relax
621    \advance\count@\@ne
622    \global\bbl@csarg\chardef{id@\language}\count@
623    \xdef\bbl@id@last{\the\count@}%
624    \ifcase\bbl@engine\or
625      \directlua{
626        Babel.locale_props[\bbl@id@last] = {}
627        Babel.locale_props[\bbl@id@last].name = '\language'
628        Babel.locale_props[\bbl@id@last].vars = {}
629      }%
630    \fi}%
631  }%
632  \chardef\localeid\bbl@c{id@}}
```

The unprotected part of \selectlanguage. In case it is used as environment, declare \endselectlanguage, just for safety.

```
633 \let\bbl@select@opts\@empty
634 \expandafter\def\csname selectlanguage \endcsname{%
635   \@ifnextchar[\bbl@select@s{\bbl@select@s[]}}
636 \def\bbl@select@s[#1]#2{%
637   \def\bbl@select@opts{#1}%
638   \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\tw\fi
639   \bbl@push@language
640   \aftergroup\bbl@pop@language
641   \bbl@set@language{#2}}
642 \let\endselectlanguage\relax
```

\bbl@set@language The macro \bbl@set@language takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of \language. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in \language are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining \BabelContentsFiles, but make sure they are loaded inside a group (as aux, toc, lof, and lot do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from `hyperref`, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

643 \def\BabelContentsFiles{toc,lof,lot}
644 \def\bbl@set@language#1{% from selectlanguage, pop@
645 % The old buggy way. Preserved for compatibility, but simplified
646 \edef\language{\expandafter\string#1\@empty}%
647 \select@language{\language}%
648 \bbl@xin@{,main,},{,\bbl@select@opts,}%
649 \ifin@
650 \let\bbl@main@language\localename
651 \let\mainlocalename\localename
652 \fi
653 \let\bbl@select@opts\@empty
654 % write to aux files
655 \expandafter\ifx\csname date\language\endcsname\relax\else
656 \if@filesw
657 \bbl@xin@{,nofiles,},{,\bbl@select@opts,}%
658 \ifin@else
659 \ifx\babel@aux@gobbletwo\else % Set if single in the first, redundant
660 \bbl@savelastskip
661 \protected@write\@auxout{\string\babel@aux{\bbl@auxname}{}}%
662 \bbl@restorelastskip
663 \fi
664 \bbl@usehooks{write}{}%
665 \fi
666 \fi
667 \fi}
668 %
669 \let\bbl@restorelastskip\relax
670 \let\bbl@savelastskip\relax
671 %
672 \def\select@language#1{% from set@, babel@aux, babel@toc
673 \ifx\bbl@selectorname\@empty
674 \def\bbl@selectorname{select}%
675 \fi
676 % set hymap
677 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
678 % set name (when coming from babel@aux)
679 \edef\language{#1}%
680 \bbl@fixname\language
681 % define \localename when coming from set@, with a trick
682 \ifx\scantokens\undefined
683 \def\localename{??}%
684 \else
685 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
686 \fi
687 \bbl@provide@locale
688 \bbl@iflanguage\language{%
689 \let\bbl@select@type\z@
690 \expandafter\bbl@switch\expandafter{\language}}
691 \def\babel@aux#1#2{%
692 \select@language{#1}%
693 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
694 \@writefile{##1}{\babel@toc{#1}{#2}\relax}}%
695 \def\babel@toc#1#2{%
696 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring `TEX` in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To

save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

697 \newif\ifbbl@usedategroup
698 \let\bbl@savextras\@empty
699 \def\bbl@switch#1{% from select@, foreign@
700 % restore
701 \originalTeX
702 \expandafter\def\expandafter\originalTeX\expandafter{%
703   \csname noextras#1\endcsname
704   \let\originalTeX\@empty
705   \babel@beginsave}%
706 \bbl@usehooks{afterreset}{}%
707 \languageshorthands{none}%
708 % set the locale id
709 \bbl@id@assign
710 % switch captions, date
711 \bbl@bsphack
712   \ifcase\bbl@select@type
713     \csname captions#1\endcsname\relax
714     \csname date#1\endcsname\relax
715   \else
716     \bbl@xin@{,captions,}{,\bbl@select@opts,}%
717     \ifin@
718       \csname captions#1\endcsname\relax
719     \fi
720     \bbl@xin@{,date,}{,\bbl@select@opts,}%
721     \ifin@ % if \foreign... within \<language>date
722       \csname date#1\endcsname\relax
723     \fi
724   \fi
725 \bbl@esphack
726 % switch extras
727 \csname bbl@preextras@#1\endcsname
728 \bbl@usehooks{beforeextras}{}%
729 \csname extras#1\endcsname\relax
730 \bbl@usehooks{afterextras}{}%
731 % > babel-ensure
732 % > babel-sh-<short>
733 % > babel-bidi
734 % > babel-fontspec
735 \let\bbl@savextras\@empty
736 % hyphenation - case mapping
737 \ifcase\bbl@opt@hyphenmap\or
738   \def\BabelLower##1##2{\lccode##1=##2\relax}%
739   \ifnum\bbl@hymapsel>4\else
740     \csname\language @bbl@hyphenmap\endcsname
741   \fi
742   \chardef\bbl@opt@hyphenmap\z@
743 \else
744   \ifnum\bbl@hymapsel>\bbl@opt@hyphenmap\else
745     \csname\language @bbl@hyphenmap\endcsname
746   \fi
747 \fi
748 \let\bbl@hymapsel\@cclv
749 % hyphenation - select rules

```

```

750 \ifnum\csname l@\language\endcsname=\l@unhyphenated
751 \edef\bbl@tempa{u}%
752 \else
753 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
754 \fi
755 % linebreaking - handle u, e, k (v in the future)
756 \bbl@xin@{/u}{/\bbl@tempa}%
757 \ifin@else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
758 \ifin@else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
759 \ifin@else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
760 \ifin@else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
761 % hyphenation - save mins
762 \babel@savevariable\lefthyphenmin
763 \babel@savevariable\righthyphenmin
764 \ifnum\bbl@engine=\@ne
765 \babel@savevariable\hyphenationmin
766 \fi
767 \ifin@
768 % unhyphenated/kashida/elongated/padding = allow stretching
769 \language\l@unhyphenated
770 \babel@savevariable\emergencystretch
771 \emergencystretch\maxdimen
772 \babel@savevariable\hbadness
773 \hbadness\@M
774 \else
775 % other = select patterns
776 \bbl@patterns{#1}%
777 \fi
778 % hyphenation - set mins
779 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
780 \set@hyphenmins\tw@\thr@@\relax
781 \@nameuse{\bbl@hyphenmins@}%
782 \else
783 \expandafter\expandafter\expandafter\set@hyphenmins
784 \csname #1hyphenmins\endcsname\relax
785 \fi
786 \@nameuse{\bbl@hyphenmins@}%
787 \@nameuse{\bbl@hyphenmins@\language}%
788 \@nameuse{\bbl@hyphenatmin@}%
789 \@nameuse{\bbl@hyphenatmin@\language}%
790 \let\bbl@selectorname\@empty

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

791 \edef\otherlanguage{%
792 \noexpand\protect
793 \expandafter\noexpand\csname otherlanguage \endcsname}
794 \expandafter\def\csname otherlanguage \endcsname{%
795 \@ifstar{\@nameuse{otherlanguage*}}{\bbl@otherlanguage}
796 \def\bbl@otherlanguage#1{%
797 \def\bbl@selectorname{other}%
798 \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\thr@@\fi
799 \csname selectlanguage \endcsname{#1}%
800 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

801 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of

`\foreign@language.`

```

802 \expandafter\def\csname otherlanguage*\endcsname{%
803   \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
804 \def\bbl@otherlanguage@s[#1]#2{%
805   \def\bbl@selectorname{other*}%
806   \ifnum\bbl@hymapset=\@ccclv\chardef\bbl@hymapset4\relax\fi
807   \def\bbl@select@opts{#1}%
808   \foreign@language{#2}}

```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```

809 \expandafter\let\csname endotherlanguage*\endcsname\relax

```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the `\extras<language>` command doesn’t make any \global changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a ‘text’ command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```

810 \providecommand\bbl@beforeforeign{}
811 \edef\foreignlanguage{%
812   \noexpand\protect
813   \expandafter\noexpand\csname foreignlanguage \endcsname}
814 \expandafter\def\csname foreignlanguage \endcsname{%
815   \@ifstar\bbl@foreign@s\bbl@foreign@x}
816 \providecommand\bbl@foreign@x[3][]{%
817   \begingroup
818     \def\bbl@selectorname{foreign}%
819     \def\bbl@select@opts{#1}%
820     \let\BabelText\@firstofone
821     \bbl@beforeforeign
822     \foreign@language{#2}%
823     \bbl@ifmetadata{\UseTaggingSocket{babel/foreign/begin}}}%
824     \bbl@usehooks{foreign}{}%
825     \BabelText{#3}% Now in horizontal mode!
826     \bbl@ifmetadata{\UseTaggingSocket{babel/foreign/end}}}%
827   \endgroup}
828 \def\bbl@foreign@s#1#2{%
829   \begingroup
830     {\par}%
831     \def\bbl@selectorname{foreign*}%
832     \let\bbl@select@opts\@empty
833     \let\BabelText\@firstofone
834     \foreign@language{#1}%
835     \bbl@usehooks{foreign*}{}%
836     \bbl@dirparastext
837     \BabelText{#2}% Still in vertical mode!
838     {\par}%

```

```

839 \endgroup}
840 \providecommand\BabelWrapText[1]{%
841 \def\bbl@tempa{\def\BabelText####1}%
842 \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}

```

\foreign@language This macro does the work for \foreignlanguage and the otherlanguage* environment. First we need to store the name of the language and check that it is a known language. Then it just calls bbl@switch.

```

843 \def\foreign@language#1{%
844 % set name
845 \edef\language#1}%
846 \ifbbl@usedategroup
847 \bbl@add\bbl@select@opts{,date,}%
848 \bbl@usedategroupfalse
849 \fi
850 \bbl@fixname\language
851 \let\localename\language
852 \bbl@provide@locale
853 \bbl@iflanguage\language{%
854 \let\bbl@select@type\@ne
855 \expandafter\bbl@switch\expandafter{\language}}

```

The following macro executes conditionally some code based on the selector being used.

```

856 \def\IfBabelSelectorTF#1{%
857 \bbl@xin@{\bbl@selectorname,}{,\zap@space#1 \@empty,}%
858 \ifin@
859 \expandafter\@firstoftwo
860 \else
861 \expandafter\@secondoftwo
862 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

863 \let\bbl@hyphlist\@empty
864 \let\bbl@hyphenation@\relax
865 \let\bbl@pttnlist\@empty
866 \let\bbl@patterns@\relax
867 \let\bbl@hymapsel=\@cclv
868 \def\bbl@patterns#1{%
869 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
870 \csname l@#1\endcsname
871 \edef\bbl@tempa{#1}%
872 \else
873 \csname l@#1:\f@encoding\endcsname
874 \edef\bbl@tempa{#1:\f@encoding}%
875 \fi
876 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
877 % > luatex
878 \@ifundefined{bbl@hyphenation@}{% Can be \relax!
879 \begingroup
880 \bbl@xin@{\, \number\language,}{,\bbl@hyphlist}%
881 \ifin@\else
882 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
883 \hyphenation{%
884 \bbl@hyphenation@
885 \@ifundefined{bbl@hyphenation@#1}%

```

```

886         \@empty
887         {\space\csname bbl@hyphenation@#1\endcsname}}%
888         \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
889         \fi
890     \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change `\language` and when the hyphenation rules specified were not loaded it has no effect. Note however, `\lccode`'s and font encodings are not set at all, so in most cases you should use `otherlanguage*`.

```

891 \def\hyphenrules#1{%
892   \edef\bbl@tempf{#1}%
893   \bbl@fixname\bbl@tempf
894   \bbl@iflanguage\bbl@tempf{%
895     \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
896     \ifx\languageshorthands\@undefined\else
897       \languageshorthands{none}%
898     \fi
899     \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
900       \set@hyphenmins\tw@thr@@\relax
901     \else
902       \expandafter\expandafter\expandafter\set@hyphenmins
903       \csname\bbl@tempf hyphenmins\endcsname\relax
904     \fi}}
905 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\(language)hyphenmins` is already defined this command has no effect.

```

906 \def\providehyphenmins#1#2{%
907   \expandafter\ifx\csname #1hyphenmins\endcsname\relax
908     \@namedef{#1hyphenmins}{#2}%
909   \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

910 \def\set@hyphenmins#1#2{%
911   \lefthyphenmin#1\relax
912   \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_{\epsilon}$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by `babel`.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

913 \ifx\ProvidesFile\@undefined
914   \def\ProvidesLanguage#1[#2 #3 #4]{%
915     \wlog{Language: #1 #4 #3 <#2>}%
916   }
917 \else
918   \def\ProvidesLanguage#1{%
919     \begingroup
920     \catcode`\ 10 %
921     \@makeother\/%
922     \@ifnextchar[%]
923       {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}
924   \def\@provideslanguage#1[#2]{%
925     \wlog{Language: #1 #2}%
926     \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
927   \endgroup}
928 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\TeX$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```
929 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi
```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```
930 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi
```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```
931 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}{}
932 \let\uselocale\setlocale
933 \let\locale\setlocale
934 \let\selectlocale\setlocale
935 \let\textlocale\setlocale
936 \let\textlanguage\setlocale
937 \let\languagegettext\setlocale
```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn’t been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\LaTeX 2_{\epsilon}$, so we can safely use its error handling interface. Otherwise we’ll have to ‘keep it simple’.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```
938 \edef\bbl@nulllanguage{\string\language=0}
939 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
940 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
941   \global\@namedef{#2}{\textbf{?#1?}}}%
942   \@nameuse{#2}%
943   \edef\bbl@tempa{#1}%
944   \bbl@sreplace\bbl@tempa{name}}}%
945   \bbl@warning{%
946     \@backslashchar#1 not set for '\language'. Please,\\%
947     define it after the language has been loaded\\%
948     (typically in the preamble) with:\\%
949     \string\setlocalecaption{\language}\bbl@tempa{.}\\%
950     Feel free to contribute on github.com/latex3/babel.\\%
951     Reported}}
952 \def\bbl@tentative{\protect\bbl@tentative@i}
953 \def\bbl@tentative@i#1{%
954   \bbl@warning{%
955     Some functions for '#1' are tentative.\\%
956     They might not work as expected and their behavior\\%
957     could change in the future.\\%
958     Reported}}
959 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}}{}{}
960 \def\@nopatterns#1{%
961   \bbl@warning
962     {No hyphenation patterns were preloaded for\\%
963     the language '#1' into the format.\\%
964     Please, configure your TeX system to add them and\\%
965     rebuild the format. Now I will use the patterns\\%
966     preloaded for \bbl@nulllanguage\space instead}}
967 \let\bbl@usehooks@gobbletwo
```

Here ended the now discarded switch.def.
 Here also (currently) ends the base option.
 968 \ifx\bbl@onlyswitch\@empty\endinput\fi

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\bbl@e@<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\bbl@e@<language>` contains `\bbl@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\bbl@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

969 \bbl@trace{Defining babelensure}
970 \newcommand\babelensure[2][]{%
971   \AddBabelHook{babel-ensure}{afterextras}{%
972     \ifcase\bbl@select@type
973       \bbl@cl{e}%
974     \fi}%
975   \begingroup
976     \let\bbl@ens@include\@empty
977     \let\bbl@ens@exclude\@empty
978     \def\bbl@ens@fontenc{\relax}%
979     \def\bbl@tempb##1{%
980       \ifx\@empty##1\else\noexpand##1\expandafter\bbl@tempb\fi}%
981     \edef\bbl@tempa{\bbl@tempb#1\@empty}%
982     \def\bbl@tempb##1=##2\@{\@namedef{\bbl@ens@##1}{##2}}%
983     \bbl@foreach\bbl@tempa{\bbl@tempb##1\@}%
984     \def\bbl@tempc{\bbl@ensure}%
985     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
986       \expandafter{\bbl@ens@include}}%
987     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
988       \expandafter{\bbl@ens@exclude}}%
989     \toks@\expandafter{\bbl@tempc}%
990     \bbl@exp{%
991   \endgroup
992   \def<\bbl@e@#2>{\the\toks@{\bbl@ens@fontenc}}}%
993 \def\bbl@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
994   \def\bbl@tempb##1{% elt for (excluding) \bbl@captionslist list
995     \ifx##1\undefined % 3.32 - Don't assume the macro exists
996       \edef##1{\noexpand\bbl@nocaption
997         {\bbl@stripslash##1}{\language\name\bbl@stripslash##1}}%
998     \fi
999     \ifx##1\@empty\else
1000       \in@{##1}{#2}%
1001       \ifin@\else
1002         \let\bbl@tempc\language\name
1003         \bbl@replace\bbl@tempc{-}{@}%
1004         \bbl@ifunset{\bbl@ensure@\bbl@tempc}%
1005         {\bbl@exp{%
1006           \\DeclareRobustCommand\<\bbl@ensure@\bbl@tempc>[1]{%
1007             \\foreignlanguage{\language\name}%
1008             {\ifx\relax#3\else
1009               \\fontencoding{#3}\\selectfont
1010             \fi
1011             #####1}}}%
1012         {}%
1013         \toks@\expandafter{##1}%
1014         \edef##1{%
1015           \bbl@csarg\noexpand{ensure@\bbl@tempc}%

```

```

1016         {\the\toks@}}}%
1017     \fi
1018     \expandafter\babel@tempb
1019 \fi}%
1020 \expandafter\babel@tempb\babel@captionslist\today\@empty
1021 \def\babel@tempa##1{% elt for include list
1022     \ifx##1\@empty\else
1023         \let\babel@tempc\language\language
1024         \babel@replace\babel@tempc{-}{@}%
1025         \babel@csarg\in@{\ensure@babel@tempc\expandafter}\expandafter{##1}%
1026         \ifin@ \else
1027             \babel@tempb##1\@empty
1028         \fi
1029         \expandafter\babel@tempa
1030     \fi}%
1031 \babel@tempa#1\@empty}
1032 \def\babel@captionslist{%
1033     \prefacename\refname\abstractname\bibname\chaptername\appendixname
1034     \contentsname\listfigurename\listtablename\indexname\figurename
1035     \tablename\partname\enclname\ccname\headtoname\pagename\seename
1036     \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```

1037 \babel@trace{Short tags}
1038 \newcommand\babeltags[1]{%
1039     \edef\babel@tempa{\zap@space#1 \@empty}%
1040     \def\babel@tempb##1=##2\@{
1041         \edef\babel@tempc{
1042             \noexpand\newcommand
1043             \expandafter\noexpand\csname ##1\endcsname{
1044                 \noexpand\protect
1045                 \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1046             \noexpand\newcommand
1047             \expandafter\noexpand\csname text##1\endcsname{
1048                 \noexpand\foreignlanguage{##2}}}
1049     \babel@tempc}%
1050 \babel@for\babel@tempa\babel@tempa{
1051     \expandafter\babel@tempb\babel@tempa\@{

```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```

1052 \babel@trace{Compatibility with language.def}
1053 \ifx\directlua\@undefined\else
1054     \ifx\babel@luapatterns\@undefined
1055         \input luababel.def
1056     \fi
1057 \fi
1058 \ifx\babel@languages\@undefined
1059     \ifx\directlua\@undefined
1060         \openin0 = language.def
1061         \ifeof0
1062             \closein0
1063             \message{I couldn't find the file language.def}
1064         \else
1065             \closein0
1066             \begingroup
1067                 \def\addlanguage#1#2#3#4#5{

```

```

1068      \expandafter\ifx\csname lang@#1\endcsname\relax\else
1069      \global\expandafter\let\csname l@#1\expandafter\endcsname
1070      \csname lang@#1\endcsname
1071      \fi}%
1072      \def\uselanguage#1{%
1073      \input language.def
1074      \endgroup
1075      \fi
1076      \fi
1077      \chardef\l@english\z@
1078      \fi

```

\addto It takes two arguments, a *(control sequence)* and TeX-code to be added to the *(control sequence)*.

If the *(control sequence)* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```

1079 \def\addto#1#2{%
1080   \ifx#1\@undefined
1081     \def#1{#2}%
1082   \else
1083     \ifx#1\relax
1084       \def#1{#2}%
1085     \else
1086       {\toks@\expandafter{#1#2}%
1087        \xdef#1{\the\toks@}}%
1088     \fi
1089   \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1090 \bbl@trace{Hooks}
1091 \newcommand\AddBabelHook[3][ ]{%
1092   \bbl@iifunset\bbl@hk@#2{\EnableBabelHook{#2}}}%
1093   \def\bbl@tempa##1,##3=##2,##3\@empty{\def\bbl@tempb{##2}}%
1094   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1095   \bbl@iifunset\bbl@ev@#2@#3@#1{%
1096     {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1097     {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1098   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1099 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1100 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1101 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1102 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1103   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1104   \def\bbl@elth##1{%
1105     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1106     \bbl@cs{ev@#2@}%
1107   \ifx\language\@undefined\else % Test required for Plain (?)
1108     \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1109   \def\bbl@elth##1{%
1110     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1@#3}}%
1111     \bbl@cs{ev@#2@#1}%
1112   \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1113 \def\bbl@evargs{,% <- don't delete this comma

```

```

1114 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1115 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1116 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1117 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1118 beforestart=0,language=2,begindocument=1}
1119 \ifx\NewHook\undefined\else % Test for Plain (?)
1120 \def\bbl@tempa#1=#2\@{\NewHook{babel/#1}\NewHook{babel/*/#1}}
1121 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1122 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1123 \providecommand\PassOptionsToLocale[2]{%
1124 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit \LdfInit macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, '=', because it is sometimes used in constructions with the \let primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to \LdfInit is a control sequence. We do that by looking at the first token after passing #2 through string. When it is equal to \@backslashchar we are dealing with a control sequence which we can compare with \@undefined.

If so, we call \ldf@quit to set the main language, restore the category code of the @-sign and call \endinput

When #2 was *not* a control sequence we construct one and compare it with \relax.

Finally we check \originalTeX.

```

1125 \bbl@trace{Macros for setting language files up}
1126 \def\bbl@ldfinit{%
1127 \let\bbl@screset\@empty
1128 \let\BabelStrings\bbl@opt@string
1129 \let\BabelOptions\@empty
1130 \let\BabelLanguages\relax
1131 \ifx\originalTeX\undefined
1132 \let\originalTeX\@empty
1133 \else
1134 \originalTeX
1135 \fi}
1136 \def\LdfInit#1#2{%
1137 \chardef\atcatcode=\catcode`\@
1138 \catcode`\@=11\relax
1139 \chardef\eqcatcode=\catcode`\=
1140 \catcode`\==12\relax
1141 \ifpackagewith{babel}{ensureinfo=off}{}%
1142 {\ifx\InputIfFileExists\undefined\else
1143 \bbl@ifunset{\bbl@lname@#1}%
1144 {\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1145 \def\language{#1}%
1146 \bbl@id@assign
1147 \bbl@load@info{#1}}}%
1148 {}%
1149 \fi}%
1150 \expandafter\if\expandafter\@backslashchar
1151 \expandafter\@car\string#2\@nil
1152 \ifx#2\undefined\else
1153 \ldf@quit{#1}%

```

```

1154 \fi
1155 \else
1156 \expandafter\ifx\csname#2\endcsname\relax\else
1157 \ldf@quit{#1}%
1158 \fi
1159 \fi
1160 \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1161 \def\ldf@quit#1{%
1162 \expandafter\main@language\expandafter{#1}%
1163 \catcode`\@=\atcatcode \let\atcatcode\relax
1164 \catcode`\==\eqcatcode \let\eqcatcode\relax
1165 \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1166 \def\bbl@afterldf{%
1167 \bbl@afterlang
1168 \let\bbl@afterlang\relax
1169 \let\BabelModifiers\relax
1170 \let\bbl@screset\relax}%
1171 \def\ldf@finish#1{%
1172 \loadlocalcfg{#1}%
1173 \bbl@afterldf
1174 \expandafter\main@language\expandafter{#1}%
1175 \catcode`\@=\atcatcode \let\atcatcode\relax
1176 \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in $\LaTeX$.

```

1177 \@onlypreamble\LdfInit
1178 \@onlypreamble\ldf@quit
1179 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1180 \def\main@language#1{%
1181 \def\bbl@main@language{#1}%
1182 \let\language\bbl@main@language
1183 \let\localename\bbl@main@language
1184 \let\mainlocalename\bbl@main@language
1185 \bbl@id@assign
1186 \ifcase\bbl@engine\or
1187 \ifx\setattribute\@undefined\else
1188 \setattribute\bbl@attr@locale\localeid
1189 \fi
1190 \fi
1191 \bbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1192 \def\bbl@beforestart{%

```

```

1193 \def\@nolanerr##1{%
1194   \bbl@carg\chardef{l@##1}\z@
1195   \bbl@warning{Undefined language '##1' in aux.\\Reported}}%
1196 \bbl@usehooks{beforestart}{}%
1197 \global\let\bbl@beforestart\relax
1198 \AtBeginDocument{%
1199   {\@nameuse\bbl@beforestart}}% Group!
1200 \if@filesw
1201   \providecommand\babel@aux[2]{}%
1202   \immediate\write\@mainaux{\unexpanded{%
1203     \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1204   \immediate\write\@mainaux{\string\@nameuse\bbl@beforestart}}%
1205 \fi
1206 \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1207 \ifbbl@single % must go after the line above.
1208   \renewcommand\selectlanguage[1]{}%
1209   \renewcommand\foreignlanguage[2]{#2}%
1210   \global\let\babel@aux\@gobbletwo % Also as flag
1211 \fi}

A bit of optimization. Select in heads/feet the language only if necessary.

1212 \def\select@language@x#1{%
1213   \ifcase\bbl@select@type
1214     \bbl@ifsamestring\language{#1}{\select@language{#1}}%
1215   \else
1216     \select@language{#1}%
1217   \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1218 \bbl@trace{Shorhands}
1219 \def\bbl@withactive#1#2{%
1220   \begingroup
1221   \lccode`~=#2\relax
1222   \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1223 \def\bbl@add@special#1{% 1:a macro like "\", \?, etc.
1224   \bbl@add\dospecials{\do#1}% test @sanitize = \relax, for back. compat.
1225   \bbl@ifunset{@sanitize}{\bbl@add\@sanitize{\makeother#1}}%
1226   \ifx\nfss@catcodes\undefined\else
1227     \begingroup
1228     \catcode`#1\active
1229     \nfss@catcodes
1230     \ifnum\catcode`#1=\active
1231       \endgroup
1232       \bbl@add\nfss@catcodes{\@makeother#1}%
1233     \else
1234       \endgroup
1235     \fi
1236   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char⟨char⟩` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char⟨char⟩` by default (`⟨char⟩` being the character to be made active). Later its definition can be changed to expand to `\active@char⟨char⟩` by calling `\bbl@activate{⟨char⟩}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\⟨level⟩@group`, `⟨level⟩@active` and `⟨next-level⟩@active` (except in system).

```
1237 \def\bbl@active@def#1#2#3#4{%
1238   \namedef{#3#1}{%
1239     \expandafter\ifx\csname#2@sh@#1\endcsname\relax
1240       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1241     \else
1242       \bbl@afterfi\csname#2@sh@#1\endcsname
1243     \fi}%

```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```
1244   \long\@namedef{#3@arg#1}##1{%
1245     \expandafter\ifx\csname#2@sh@#1@string##1\endcsname\relax
1246       \bbl@afterelse\csname#4#1\endcsname##1%
1247     \else
1248       \bbl@afterfi\csname#2@sh@#1@string##1\endcsname
1249     \fi}}%

```

`\initiate@active@char` calls `\@initiate@active@char` with 3 arguments. All of them are the same character with different catcodes: active, other (`\string’ed`) and the original one. This trick simplifies the code a lot.

```
1250 \def\@initiate@active@char#1{%
1251   \bbl@ifunset{active@char\string#1}%
1252   {\bbl@withactive
1253     {\expandafter\@initiate@active@char\expandafter}#1\string#1#1}%
1254   {}}

```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them `\relax` and preserving some degree of protection).

```
1255 \def\@initiate@active@char#1#2#3{%
1256   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1257   \ifx#1\@undefined
1258     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1259   \else
1260     \bbl@csarg\let{oridef@#2}#1%
1261     \bbl@csarg\edef{oridef@#2}{%
1262       \let\noexpand#1%
1263       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1264   \fi

```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define `\normal@char⟨char⟩` to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example `'`) the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to `"8000 a posteriori`).

```
1265   \ifx#1#3\relax

```

```

1266 \expandafter\let\csname normal@char#2\endcsname#3%
1267 \else
1268 \bbl@info{Making #2 an active character}%
1269 \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1270 \@namedef{normal@char#2}{%
1271 \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1272 \else
1273 \@namedef{normal@char#2}{#3}%
1274 \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at `\begin{document}`. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of `\bibitem` for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1275 \bbl@restoreactive{#2}%
1276 \AtBeginDocument{%
1277 \catcode`#2\active
1278 \if@filesw
1279 \immediate\write\@mainaux{\catcode`\string#2\active}%
1280 \fi}%
1281 \expandafter\bbl@add@special\csname#2\endcsname
1282 \catcode`#2\active
1283 \fi

```

Now we have set `\normal@char{char}`, we must define `\active@char{char}`, to be executed when the character is activated. We define the first level expansion of `\active@char{char}` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active{char}` to start the search of a definition in the user, language and system levels (or eventually `normal@char{char}`).

```

1284 \let\bbl@tempa\@firstoftwo
1285 \if\string^#2%
1286 \def\bbl@tempa{\noexpand\textormath}%
1287 \else
1288 \ifx\bbl@mathnormal\@undefined\else
1289 \let\bbl@tempa\bbl@mathnormal
1290 \fi
1291 \fi
1292 \expandafter\edef\csname active@char#2\endcsname{%
1293 \bbl@tempa
1294 {\noexpand\if@safe@actives
1295 \noexpand\expandafter
1296 \expandafter\noexpand\csname normal@char#2\endcsname
1297 \noexpand\else
1298 \noexpand\expandafter
1299 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1300 \noexpand\fi}%
1301 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1302 \bbl@csarg\edef{doactive#2}{%
1303 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

$$\text{active@prefix}\langle char\rangle\text{normal@char}\langle char\rangle$$

(where `\active@char{char}` is *one* control sequence!).

```

1304 \bbl@csarg\edef{active@#2}{%
1305 \noexpand\active@prefix\noexpand#1%
1306 \expandafter\noexpand\csname active@char#2\endcsname}%
1307 \bbl@csarg\edef{normal@#2}{%
1308 \noexpand\active@prefix\noexpand#1%
1309 \expandafter\noexpand\csname normal@char#2\endcsname}%
1310 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn't exist we check for a shorthand with an argument.

```
1311 \bbl@active@def#2\user@group{user@active}{language@active}%
1312 \bbl@active@def#2\language@group{language@active}{system@active}%
1313 \bbl@active@def#2\system@group{system@active}{normal@char}%
```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ' ' ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```
1314 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1315 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1316 \expandafter\edef\csname\user@group @sh#2@\string\protect\endcsname
1317 {\expandafter\noexpand\csname user@active#2\endcsname}%
```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (') active we need to change `\pr@m@s` as well. Also, make sure that a single ' in math mode 'does the right thing'. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```
1318 \if\string'#2%
1319 \let\prim@s\bbl@prim@s
1320 \let\active@math@prime#1%
1321 \fi
1322 \bbl@usehooks{initiateactive}{{#1}{#2}{#3}}}
```

The following package options control the behavior of shorthands in math mode.

```
1323 <(*More package options)> ≡
1324 \DeclareOption{math=active}{}
1325 \DeclareOption{math=normal}{{\def\bbl@mathnormal{\noexpand\textormath}}}
1326 <(/More package options)>
```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```
1327 \ifpackagewith{babel}{KeepShorthandsActive}%
1328 {\let\bbl@restoreactive@gobble}%
1329 {\def\bbl@restoreactive#1{%
1330 \bbl@exp{%
1331 \\\AfterBabelLanguage\\CurrentOption
1332 {\catcode`#1=\the\catcode`#1\relax}%
1333 \\\AtEndOfPackage
1334 {\catcode`#1=\the\catcode`#1\relax}}}%
1335 \AtEndOfPackage{\let\bbl@restoreactive@gobble}}
```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```
1336 \def\bbl@sh@select#1#2{%
1337 \expandafter\ifx\csname#1@sh#2@sel\endcsname\relax
1338 \bbl@afterelse\bbl@scndcs
1339 \else
1340 \bbl@afterfi\csname#1@sh#2@sel\endcsname
1341 \fi}
```

\active@prefix Used in the expansion of active characters has a function similar to \OT1-cmd in that it \protects the active character whenever \protect is *not* \@typeset@protect. The \gobble is needed to remove a token such as \activechar: (when the double colon was the active character to be dealt with). There are two definitions, depending on \ifincsname is available. If there is, the expansion will be more robust.

```

1342 \begingroup
1343 \bbl@ifunset{ifincsname}
1344 {\gdef\active@prefix#1{%
1345   \ifx\protect\@typeset@protect
1346   \else
1347     \ifx\protect\@unexpandable@protect
1348     \noexpand#1%
1349     \else
1350     \protect#1%
1351     \fi
1352     \expandafter\@gobble
1353   \fi}}
1354 {\gdef\active@prefix#1{%
1355   \ifincsname
1356   \string#1%
1357   \expandafter\@gobble
1358   \else
1359     \ifx\protect\@typeset@protect
1360     \else
1361       \ifx\protect\@unexpandable@protect
1362       \noexpand#1%
1363       \else
1364       \protect#1%
1365       \fi
1366       \expandafter\expandafter\expandafter\@gobble
1367     \fi
1368   \fi}}
1369 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char⟨char⟩. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string’ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1370 \newif\if@safe@actives
1371 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1372 \def\bbl@restore@actives{\if@safe@actives\@safe@activesfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char⟨char⟩ in the case of \bbl@activate, or \normal@char⟨char⟩ in the case of \bbl@deactivate.

```

1373 \chardef\bbl@activated\z@
1374 \def\bbl@activate#1{%
1375   \chardef\bbl@activated\@ne
1376   \bbl@withactive{\expandafter\let\expandafter}#1%
1377   \csname bbl@active@\string#1\endcsname}
1378 \def\bbl@deactivate#1{%
1379   \chardef\bbl@activated\tw@
1380   \bbl@withactive{\expandafter\let\expandafter}#1%

```

```
1381 \csname bbl@normal@\string#1\endcsname}
```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```
1382 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1383 \def\bbl@scndcs#1#2{\csname#2\endcsname}
```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The $\TeX$ code in text mode, (2) the string for `hyperref`, (3) the $\TeX$ code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn’t discriminate the mode). This macro may be used in `ldf` files.

```
1384 \def\babel@texpdf#1#2#3#4{%
1385   \ifx\texorpdfstring\undefined
1386     \textormath{#1}{#3}%
1387   \else
1388     \texorpdfstring{\textormath{#1}{#3}}{#2}%
1389     % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1390   \fi}
1391 %
1392 \def\declare@shorthand#1#2{\@decl@short{#1}#2@nil}
1393 \def\@decl@short#1#2#3@nil#4{%
1394   \def\bbl@tempa{#3}%
1395   \ifx\bbl@tempa\empty
1396     \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1397     \bbl@ifunset{#1@sh@\string#2@}{}%
1398     {\def\bbl@tempa{#4}%
1399      \expandafter\ifx\csname#1@sh@\string#2@\endcsname\bbl@tempa
1400      \else
1401        \bbl@info
1402        {Redefining #1 shorthand \string#2\}%
1403        in language \CurrentOption}%
1404      \fi}%
1405   \@namedef{#1@sh@\string#2@}{#4}%
1406 \else
1407   \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1408   \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1409   {\def\bbl@tempa{#4}%
1410    \expandafter\ifx\csname#1@sh@\string#2@\string#3@\endcsname\bbl@tempa
1411    \else
1412      \bbl@info
1413      {Redefining #1 shorthand \string#2\string#3\}%
1414      in language \CurrentOption}%
1415    \fi}%
1416   \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1417 \fi}
```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```
1418 \def\textormath{%
1419   \ifmmode
1420     \expandafter\@secondoftwo
1421   \else
1422     \expandafter\@firstoftwo
1423   \fi}
```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```
1424 \def\user@group{user}
1425 \def\language@group{english}
1426 \def\system@group{system}
```

\usesshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```
1427 \def\usesshorthands{%
1428   \ifstar\bb@usesesh@s{\bb@usesesh@x{}}
1429   \def\bb@usesesh@s#1{%
1430     \bb@usesesh@x
1431     {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bb@activate{#1}}}%
1432     {#1}}
1433   \def\bb@usesesh@x#1#2{%
1434     \bb@ifshorthand{#2}%
1435     {\def\user@group{user}%
1436       \initiate@active@char{#2}%
1437       #1%
1438       \bb@activate{#2}}%
1439     {\bb@error{shorthand-is-off}{#2}}}
```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(\language) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bb@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```
1440 \def\user@language@group{user@\language@group}
1441 \def\bb@set@user@generic#1#2{%
1442   \bb@ifunset{user@generic@active#1}%
1443   {\bb@active@def#1\user@language@group{user@active}{user@generic@active}%
1444     \bb@active@def#1\user@group{user@generic@active}{language@active}%
1445     \expandafter\edef\csname#2@sh@#1@\endcsname{%
1446       \expandafter\noexpand\csname normal@char#1\endcsname}%
1447     \expandafter\edef\csname#2@sh@#1@\string\protect\endcsname{%
1448       \expandafter\noexpand\csname user@active#1\endcsname}}%
1449   \@empty}
1450 \newcommand\defineshorthand[3][user]{%
1451   \edef\bb@tempa{\zap@space#1 \@empty}%
1452   \bb@for\bb@tempb\bb@tempa{%
1453     \if*\expandafter\@car\bb@tempb\@nil
1454       \edef\bb@tempb{user@\expandafter\@gobble\bb@tempb}%
1455       \@expandtwoargs
1456       \bb@set@user@generic{\expandafter\string\@car#2\@nil}\bb@tempb
1457     \fi
1458     \declare@shorthand{\bb@tempb}{#2}{#3}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```
1459 \def\languageshorthands#1{%
1460   \bb@ifsamestring{none}{#1}{}%
1461   \bb@once{short-\localename-#1}{%
1462     \bb@info{'\localename' activates '#1' shorthands.\@Reported}}%
1463   \def\language@group{#1}}
```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshandands{"}{/}` is `\active@prefix / \active@char/`, so we still need to let the latter to `\active@char`".

```

1464 \def\aliasshorthand#1#2{%
1465   \bbl@ifshorthand{#2}%
1466   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1467     \ifx\document\@notprerr
1468       \@notshorthand{#2}%
1469     \else
1470       \initiate@active@char{#2}%
1471       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1472       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1473       \bbl@activate{#2}%
1474     \fi
1475   \fi}%
1476   {\bbl@error{shorthand-is-off}{#2}{}}}
```

\@notshorthand

```

1477 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}
```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```

1478 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1479 \DeclareRobustCommand*\shorthandoff{%
1480   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1481 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}
```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char`" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```

1482 \def\bbl@switch@sh#1#2{%
1483   \ifx#2\@nnil\else
1484     \bbl@ifunset{bbl@active@\string#2}%
1485     {\bbl@error{not-a-shorthand-b}{#2}{}}%
1486     {\ifcase#1 off, on, off*
1487       \catcode`#2\relax
1488     \or
1489       \catcode`#2\active
1490       \bbl@ifunset{bbl@shdef@\string#2}%
1491       {}%
1492       {\bbl@withactive{\expandafter\let\expandafter}#2%
1493         \csname bbl@shdef@\string#2\endcsname
1494         \bbl@csarg\let{shdef@\string#2}\relax}%
1495       \ifcase\bbl@activated\or
1496         \bbl@activate{#2}%
1497       \else
1498         \bbl@deactivate{#2}%
1499       \fi
1500     \or
1501       \bbl@ifunset{bbl@shdef@\string#2}%
1502       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1503       {}%
1504       \csname bbl@oricat@\string#2\endcsname
1505       \csname bbl@oridef@\string#2\endcsname
1506     \fi}%

```

```

1507 \bbl@afterfi\bbl@switch@sh#1%
1508 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1509 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1510 \def\bbl@putsh#1{%
1511 \bbl@ifunset{\bbl@active@\string#1}%
1512 {\bbl@putsh@i#1\@empty\@nnil}%
1513 {\csname bbl@active@\string#1\endcsname}}
1514 \def\bbl@putsh@i#1#2\@nnil{%
1515 \csname\language@group @sh@\string#1@%
1516 \ifx\@empty#2\else\string#2@\fi\endcsname}
1517 %
1518 \ifx\bbl@opt@shorthands\@nnil\else
1519 \let\bbl@s@initiate@active@char@initiate@active@char
1520 \def\initiate@active@char#1{%
1521 \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1522 \let\bbl@s@switch@sh\bbl@switch@sh
1523 \def\bbl@switch@sh#1#2{%
1524 \ifx#2\@nnil\else
1525 \bbl@afterfi
1526 \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1527 \fi}
1528 \let\bbl@s@activate\bbl@activate
1529 \def\bbl@activate#1{%
1530 \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1531 \let\bbl@s@deactivate\bbl@deactivate
1532 \def\bbl@deactivate#1{%
1533 \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1534 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1535 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting `\prime` for each right quote in mathmode is `\prim@s`. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1536 \def\bbl@prim@s{%
1537 \prime\futurelet\@let@token\bbl@pr@m@s}
1538 \def\bbl@if@primes#1#2{%
1539 \ifx#1\@let@token
1540 \expandafter\@firstoftwo
1541 \else\ifx#2\@let@token
1542 \bbl@afterelse\expandafter\@firstoftwo
1543 \else
1544 \bbl@afterfi\expandafter\@secondoftwo
1545 \fi\fi}
1546 \begingroup
1547 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^
1548 \catcode`\'=12 \catcode`\\"=\active \lccode`\\"='\^
1549 \lowercase{%
1550 \gdef\bbl@pr@m@s{%
1551 \bbl@if@primes" '%
1552 \pr@@@s
1553 {\bbl@if@primes*\^{\pr@@@t\egroup}}}
1554 \endgroup

```

Usually the `~` is active and expands to `\penalty\@M\_\_`. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character `~` as a start character for a shorthand, it

is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```
1555 \initiate@active@char{~}
1556 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1557 \bbl@activate{~}
```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```
1558 \expandafter\def\csname OT1dqpos\endcsname{127}
1559 \expandafter\def\csname T1dqpos\endcsname{4}
```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```
1560 \ifx\f@encoding\undefined
1561   \def\f@encoding{OT1}
1562 \fi
```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro \languageattribute checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in \bbl@known@attrs. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated T_EX code.

```
1563 \bbl@trace{Language attributes}
1564 \newcommand\languageattribute[2]{%
1565   \def\bbl@tempc{#1}%
1566   \bbl@fixname\bbl@tempc
1567   \bbl@iflanguage\bbl@tempc{%
1568     \bbl@vforeach{#2}{%
1569       \ifx\bbl@known@attrs\undefined
1570         \in@false
1571       \else
1572         \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1573       \fi
1574       \ifin@
1575         \bbl@warning{%
1576           You have more than once selected the attribute '##1'\%
1577           for language #1. Reported}%
1578       \else
1579         \bbl@info{Activated '##1' attribute for\\%
1580           '\bbl@tempc'. Reported}%
1581         \bbl@exp{%
1582           \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1583         \edef\bbl@tempa{\bbl@tempc-##1}%
1584         \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1585         {\csname\bbl@tempc @attr##1\endcsname}%
1586         {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1587       \fi}}
1588 \onlypreamble\languageattribute
```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```
1589 \def\bbl@declare@ttribute#1#2#3{%
1590   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1591   \ifin@
1592     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1593   \fi
1594   \bbl@add@list\bbl@attributes{#1-#2}%
1595   \expandafter\def\csname#1@attr@#2\endcsname{#3}}
```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```
1596 \def\bbl@ifattributeset#1#2#3#4{%
1597   \ifx\bbl@known@attribs\@undefined
1598     \in@false
1599   \else
1600     \bbl@xin@{,#1-#2,}{,\bbl@known@attribs,}%
1601   \fi
1602   \ifin@
1603     \bbl@afterelse#3%
1604   \else
1605     \bbl@afterfi#4%
1606   \fi}
```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```
1607 \def\bbl@ifknown@ttrib#1#2{%
1608   \let\bbl@tempa\@secondoftwo
1609   \bbl@loopx\bbl@tempb{#2}{%
1610     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{,#1,}%
1611     \ifin@
1612       \let\bbl@tempa\@firstoftwo
1613     \else
1614       \fi}%
1615   \bbl@tempa}
```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```
1616 \def\bbl@clear@ttribs{%
1617   \ifx\bbl@attributes\@undefined\else
1618     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1619       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1620     \let\bbl@attributes\@undefined
1621   \fi}
1622 \def\bbl@clear@ttrib#1-#2.{%
1623   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1624 \AtBeginDocument{\bbl@clear@ttribs}
```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence

to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are `\relax`’ed.

`\babel@savecnt`

`\babel@beginsave` The initialization of a new save cycle: reset the counter to zero.

```
1625 \bbl@trace{Macros for saving definitions}
1626 \def\babel@beginsave{\babel@savecnt\z@}
```

Before it’s forgotten, allocate the counter and initialize all.

```
1627 \newcount\babel@savecnt
1628 \babel@beginsave
```

`\babel@save`

`\babel@savevariable` The macro `\babel@save{csname}` saves the current meaning of the control sequence `(csname)` to `\originalTeX` (which has to be expandable, i.e., you shouldn’t let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable{variable}` saves the value of the variable. `(variable)` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```
1629 \def\babel@save#1{%
1630   \def\bbl@tempa{{, #1,}}% Clumsy, for Plain
1631   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1632     \expandafter{\expandafter, \bbl@savextras,}}%
1633   \expandafter\in@\bbl@tempa
1634   \ifin@ \else
1635     \bbl@add\bbl@savextras{, #1,}%
1636     \bbl@carg\let\babel@number\babel@savecnt\#1\relax
1637     \toks@ \expandafter{\originalTeX\let#1=}%
1638     \bbl@exp{%
1639       \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1640     \advance\babel@savecnt@ne
1641   \fi}
1642 \def\babel@savevariable#1{%
1643   \toks@ \expandafter{\originalTeX #1=}%
1644   \bbl@exp{\def\\originalTeX{\the\toks@ \the#1\relax}}}
```

`\bbl@redefine` To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```
1645 \def\bbl@redefine#1{%
1646   \edef\bbl@tempa{\bbl@stripslash#1}%
1647   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1648   \expandafter\def\csname\bbl@tempa\endcsname}
1649 \onlypreamble\bbl@redefine
```

`\bbl@redefine@long` This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```
1650 \def\bbl@redefine@long#1{%
1651   \edef\bbl@tempa{\bbl@stripslash#1}%
1652   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1653   \long\expandafter\def\csname\bbl@tempa\endcsname}
1654 \onlypreamble\bbl@redefine@long
```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo`. So it is necessary to check whether `\foo` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo`.

```

1655 \def\bbl@redefineroobust#1{%
1656   \edef\bbl@tempa{\bbl@stripslash#1}%
1657   \bbl@iifunset{\bbl@tempa\space}%
1658   {\expandafter\let\csname org@\bbl@tempa\endcsname#1%
1659    \bbl@exp{\def\#1{\protect\<\bbl@tempa\space>}}}%
1660   {\bbl@exp{\let\<org@\bbl@tempa>\<\bbl@tempa\space>}}}%
1661   \@namedef{\bbl@tempa\space}}
1662 \@onlypreamble\bbl@redefineroobust

```

4.11. French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don't want that. The command `\bbl@frenchspacing` switches it on when it isn't already in effect and `\bbl@nonfrenchspacing` switches it off if necessary.

```

1663 \def\bbl@frenchspacing{%
1664   \ifnum\the\sfcode`\.\=\@m
1665     \let\bbl@nonfrenchspacing\relax
1666   \else
1667     \frenchspacing
1668     \let\bbl@nonfrenchspacing\nonfrenchspacing
1669   \fi}
1670 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with `ini` files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1671 \let\bbl@elt\relax
1672 \edef\bbl@fs@chars{%
1673   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1674   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1675   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1676 \def\bbl@pre@fs{%
1677   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1678   \edef\bbl@save@sfcodes{\bbl@fs@chars}%
1679 \def\bbl@post@fs{%
1680   \bbl@save@sfcodes
1681   \edef\bbl@tempa{\bbl@cl{frspc}}%
1682   \edef\bbl@tempa{\expandafter\car\bbl@tempa\@nil}%
1683   \if u\bbl@tempa      % do nothing
1684   \else\if n\bbl@tempa % non french
1685     \def\bbl@elt##1##2##3{%
1686       \ifnum\sfcode`##1=##2\relax
1687       \babel@savevariable{\sfcode`##1}%
1688       \sfcode`##1=##3\relax
1689     \fi}%
1690     \bbl@fs@chars
1691   \else\if y\bbl@tempa % french
1692     \def\bbl@elt##1##2##3{%
1693       \ifnum\sfcode`##1=##3\relax
1694       \babel@savevariable{\sfcode`##1}%
1695       \sfcode`##1=##2\relax
1696     \fi}%
1697     \bbl@fs@chars
1698   \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@(<language>)` for language ones. See

\bbl@patterns above for further details. We make sure there is a space between words when multiple commands are used.

```

1699 \bbl@trace{Hyphens}
1700 \@onlypreamble\babelhyphenation
1701 \AtEndOfPackage{%
1702   \newcommand\babelhyphenation[2][\@empty]{%
1703     \ifx\bbl@hyphenation@\relax
1704       \let\bbl@hyphenation@\@empty
1705     \fi
1706     \ifx\bbl@hyphlist@\@empty\else
1707       \bbl@warning{%
1708         You must not intermingle \string\selectlanguage\space and\\%
1709         \string\babelhyphenation\space or some exceptions will not\\%
1710         be taken into account. Reported}%
1711       \fi
1712       \ifx\@empty#1%
1713         \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1714       \else
1715         \bbl@vforeach{#1}{%
1716           \def\bbl@tempa{##1}%
1717           \bbl@fixname\bbl@tempa
1718           \bbl@iflanguage\bbl@tempa{%
1719             \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1720               \bbl@ifunset\bbl@hyphenation@\bbl@tempa}%
1721             }%
1722             {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1723             #2}}}%
1724       \fi}}

```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1725 \ifx\NewDocumentCommand\@undefined\else
1726   \NewDocumentCommand\babelhyphenmins{sommo}{%
1727     \IfNoValueTF{#2}%
1728       {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1729       \IfValueT{#5}{%
1730         \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1731       \IfBooleanT{#1}{%
1732         \lefthyphenmin=#3\relax
1733         \righthyphenmin=#4\relax
1734         \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1735     {\edef\bbl@tempb{\zap@space#2 \@empty}%
1736       \bbl@for\bbl@tempa\bbl@tempb{%
1737         \@namedef\bbl@hyphenmins@\bbl@tempa{\set@hyphenmins{#3}{#4}}}%
1738       \IfValueT{#5}{%
1739         \@namedef\bbl@hyphenatmin@\bbl@tempa{\hyphenationmin=#5\relax}}}%
1740       \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1741   \fi

```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than \nobreak \hskip 0pt plus 0pt. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1742 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1743 \def\bbl@t@one{Tl}
1744 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in \babelhyphen. Instead of protecting it with \DeclareRobustCommand, which could insert a \relax, we use the same procedure as shorthands, with \active@prefix.

```

1745 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1746 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1747 \def\bbl@hyphen{%

```

```

1748 \ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i\@empty}}
1749 \def\bbl@hyphen@i#1#2{%
1750 \lowercase{\bbl@ifunset{\bbl@hy#1#2\@empty}}}%
1751 {\csname bbl@lusehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1752 {\lowercase{\csname bbl@hy#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. \nobreak is always preceded by \leavevmode, in case the shorthand starts a paragraph.

```

1753 \def\bbl@usehyphen#1{%
1754 \leavevmode
1755 \ifdim\lastskip>z@\mbox{#1}\else\nobreak#1\fi
1756 \nobreak\hskip\z@skip}
1757 \def\bbl@usehyphen#1{%
1758 \leavevmode\ifdim\lastskip>z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1759 \def\bbl@hyphenchar{%
1760 \ifnum\hyphenchar\font=\m@ne
1761 \babe\nullhyphen
1762 \else
1763 \char\hyphenchar\font
1764 \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in ldf’s. After a space, the \mbox in \bbl@hy@nobreak is redundant.

```

1765 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1766 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1767 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1768 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1769 \def\bbl@hy@nobreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1770 \def\bbl@hy@nobreak{\mbox{\bbl@hyphenchar}}
1771 \def\bbl@hy@repeat{%
1772 \bbl@usehyphen{
1773 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1774 \def\bbl@hy@repeat{%
1775 \bbl@usehyphen{
1776 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1777 \def\bbl@hy@empty{\hskip\z@skip}
1778 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro \bbl@disc is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1779 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by luatex and xetex. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1780 \bbl@trace{Multiencoding strings}
1781 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated \SetCase.

```

1782 <(<*More package options>>) ≡
1783 \DeclareOption{nocase}{}
1784 <(</More package options>>)

```

The following package options control the behavior of `\SetString`.

```

1785 ({*More package options}) ≡
1786 \let\bbl@opt@strings\@nnil % accept strings=value
1787 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1788 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1789 \def\BabelStringsDefault{generic}
1790 (/More package options)

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1791 \@onlypreamble\StartBabelCommands
1792 \def\StartBabelCommands{%
1793   \begingroup
1794   \@tempcnta="7F
1795   \def\bbl@tempa{%
1796     \ifnum\@tempcnta>"FF\else
1797       \catcode\@tempcnta=11
1798       \advance\@tempcnta\@ne
1799       \expandafter\bbl@tempa
1800     \fi}%
1801   \bbl@tempa
1802   <@Macros local to BabelCommands@>
1803   \def\bbl@provstring##1##2{%
1804     \providecommand##1{##2}%
1805     \bbl@tglobal##1}%
1806   \global\let\bbl@scafter\@empty
1807   \let\StartBabelCommands\bbl@startcmds
1808   \ifx\BabelLanguages\relax
1809     \let\BabelLanguages\CurrentOption
1810   \fi
1811   \begingroup
1812   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1813   \StartBabelCommands}
1814 \def\bbl@startcmds{%
1815   \ifx\bbl@screset\@nnil\else
1816     \bbl@usehooks{stopcommands}{}%
1817   \fi
1818   \endgroup
1819   \begingroup
1820   \@ifstar
1821   {\ifx\bbl@opt@strings\@nnil
1822     \let\bbl@opt@strings\BabelStringsDefault
1823     \fi
1824     \bbl@startcmds@i}%
1825   \bbl@startcmds@i}
1826 \def\bbl@startcmds@i#1#2{%
1827   \edef\bbl@L{\zap@space#1 \@empty}%
1828   \edef\bbl@G{\zap@space#2 \@empty}%
1829   \bbl@startcmds@ii}
1830 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of `\SetString`. There are two main cases, depending of if there is an optional argument: without it and `strings=encoded`, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and `strings=encoded`, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in `strings=`) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1831 \newcommand\bbl@startcmds@ii[1][\@empty]{%

```

```

1832 \let\SetString\@gobbletwo
1833 \let\bbl@stringdef\@gobbletwo
1834 \let\AfterBabelCommands\@gobble
1835 \ifx\@empty#1%
1836   \def\bbl@sc@label{generic}%
1837   \def\bbl@encstring##1##2{%
1838     \ProvideTextCommandDefault##1{##2}%
1839     \bbl@tglobal##1%
1840     \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1841   \let\bbl@sctest\in@true
1842 \else
1843   \let\bbl@sc@charset\space % <- zapped below
1844   \let\bbl@sc@fontenc\space % <- " "
1845   \def\bbl@tempa##1=##2\@nil{%
1846     \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1847   \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1848   \def\bbl@tempa##1 ##2{% space -> comma
1849     ##1%
1850     \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1851   \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1852   \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1853   \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1854   \def\bbl@encstring##1##2{%
1855     \bbl@foreach\bbl@sc@fontenc{%
1856       \bbl@ifunset{T@###1}%
1857       {}%
1858       {\ProvideTextCommand##1{###1}{##2}%
1859         \bbl@tglobal##1%
1860         \expandafter
1861         \bbl@tglobal\csname###1\string##1\endcsname}}}%
1862   \def\bbl@sctest{%
1863     \bbl@xin@{\bbl@opt@strings,}{\bbl@sc@label,\bbl@sc@fontenc,}%
1864   \fi
1865   \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1866   \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1867     \let\AfterBabelCommands\bbl@aftercmds
1868     \let\SetString\bbl@setstring
1869     \let\bbl@stringdef\bbl@encstring
1870   \else % i.e., strings=value
1871     \bbl@sctest
1872   \fin@
1873     \let\AfterBabelCommands\bbl@aftercmds
1874     \let\SetString\bbl@setstring
1875     \let\bbl@stringdef\bbl@provstring
1876   \fi\fi\fi
1877   \bbl@scswitch
1878   \ifx\bbl@G\@empty
1879     \def\SetString##1##2{%
1880       \bbl@error{missing-group}{##1}{}}%
1881   \fi
1882   \ifx\@empty#1%
1883     \bbl@usehooks{defaultcommands}{}%
1884   \else
1885     \@expandtwoargs
1886     \bbl@usehooks{encodedcommands}{\bbl@sc@charset}\bbl@sc@fontenc}%
1887   \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\langle group \rangle \langle language \rangle` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date \langle language \rangle` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has

been loaded).

```

1888 \def\bbl@forlang#1#2{%
1889   \bbl@for#1\bbl@L{%
1890     \bbl@xin@{,#1,},{,\BabelLanguages,}%
1891     \ifin@#2\relax\fi}}
1892 \def\bbl@scswitch{%
1893   \bbl@forlang\bbl@tempa{%
1894     \ifx\bbl@G\@empty\else
1895       \ifx\SetString@gobbletwo\else
1896         \edef\bbl@GL{\bbl@G\bbl@tempa}%
1897         \bbl@xin@{,\bbl@GL,}{,\bbl@screset,}%
1898         \ifin@\else
1899           \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1900           \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1901         \fi
1902       \fi
1903     \fi}}
1904 \AtEndOfPackage{%
1905   \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1906   \let\bbl@scswitch\relax}
1907 \@onlypreamble\EndBabelCommands
1908 \def\EndBabelCommands{%
1909   \bbl@usehooks{stopcommands}{}%
1910   \endgroup
1911   \endgroup
1912   \bbl@scafter}
1913 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1914 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1915   \bbl@forlang\bbl@tempa{%
1916     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1917     \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1918     {\bbl@exp{%
1919       \global\\bbl@add\<\bbl@G\bbl@tempa>{\\bbl@scset\\#1\<\bbl@LC>}}}%
1920     }%
1921     \def\BabelString{#2}%
1922     \bbl@usehooks{stringprocess}{}%
1923     \expandafter\bbl@stringdef
1924     \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1925 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1926 <(*Macros local to BabelCommands)> ≡
1927 \def\SetStringLoop##1#2{%
1928   \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1929   \count@\z@
1930   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1931     \advance\count@\@ne
1932     \toks@\expandafter{\bbl@tempa}%
1933     \bbl@exp{%
1934       \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%

```

```

1935 \count@=\the\count@\relax}}}%
1936 <</Macros local to BabelCommands>>

```

Delaying code Now the definition of \AfterBabelCommands when it is activated.

```

1937 \def\bbl@aftercmds#1{%
1938 \toks@\expandafter{\bbl@scafter#1}%
1939 \xdef\bbl@scafter{\the\toks@}}

```

Case mapping The command \SetCase is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1940 <<*Macros local to BabelCommands>> ≡
1941 \newcommand\SetCase[3][]{%
1942 \def\bbl@tempa####1####2{%
1943 \ifx####1\@empty\else
1944 \bbl@carg\bbl@add{extras\CurrentOption}{%
1945 \bbl@carg\babel@save{c__text_uppercase_\string####1_tl}%
1946 \bbl@carg\def{c__text_uppercase_\string####1_tl}{####2}%
1947 \bbl@carg\babel@save{c__text_lowercase_\string####2_tl}%
1948 \bbl@carg\def{c__text_lowercase_\string####2_tl}{####1}}%
1949 \expandafter\bbl@tempa
1950 \fi}%
1951 \bbl@tempa##1\@empty\@empty
1952 \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1953 <</Macros local to BabelCommands>>

```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1954 <<*Macros local to BabelCommands>> ≡
1955 \newcommand\SetHyphenMap[1]{%
1956 \bbl@forlang\bbl@tempa{%
1957 \expandafter\bbl@stringdef
1958 \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1959 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1960 \newcommand\BabelLower[2]{% one to one.
1961 \ifnum\lccode#1=#2\else
1962 \babel@savevariable{\lccode#1}%
1963 \lccode#1=#2\relax
1964 \fi}
1965 \newcommand\BabelLowerMM[4]{% many-to-many
1966 \@tempcnta=#1\relax
1967 \@tempcntb=#4\relax
1968 \def\bbl@tempa{%
1969 \ifnum\@tempcnta>#2\else
1970 \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1971 \advance\@tempcnta#3\relax
1972 \advance\@tempcntb#3\relax
1973 \expandafter\bbl@tempa
1974 \fi}%
1975 \bbl@tempa}
1976 \newcommand\BabelLowerM0[4]{% many-to-one
1977 \@tempcnta=#1\relax
1978 \def\bbl@tempa{%
1979 \ifnum\@tempcnta>#2\else
1980 \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1981 \advance\@tempcnta#3
1982 \expandafter\bbl@tempa
1983 \fi}%
1984 \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1985 (<{*More package options}>) ≡
1986 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1987 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap\@ne}
1988 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1989 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@@}
1990 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1991 (<{/More package options}>)

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1992 \AtEndOfPackage{%
1993   \ifx\bbl@opt@hyphenmap\undefined
1994     \bbl@xin@{,}{\bbl@language@opts}%
1995     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1996   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1997 \newcommand\setlocalecaption{%
1998   \ifstar\bbl@setcaption@s\bbl@setcaption@x}
1999 \def\bbl@setcaption@x#1#2#3{% language caption-name string
2000   \bbl@trim@def\bbl@tempa{#2}%
2001   \bbl@xin@{.template}{\bbl@tempa}%
2002   \ifin@
2003     \bbl@ini@captions@template{#3}{#1}%
2004   \else
2005     \edef\bbl@tempd{%
2006       \expandafter\expandafter\expandafter
2007       \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
2008     \bbl@xin@
2009       {\expandafter\string\csname #2name\endcsname}%
2010       {\bbl@tempd}%
2011     \ifin@ % Renew caption
2012       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
2013     \ifin@
2014       \bbl@exp{%
2015         \\bbl@ifsamestring{\bbl@tempa}{\language}%
2016         {\bbl@scset\<#2name>\<#1#2name>}%
2017         {}}%
2018       \else % Old way converts to new way
2019         \bbl@ifunset{#1#2name}%
2020         {\bbl@exp{%
2021           \\bbl@add\<captions#1>{\def\<#2name>{\<#1#2name>}}%
2022           \\bbl@ifsamestring{\bbl@tempa}{\language}%
2023           {\def\<#2name>{\<#1#2name>}}%
2024           {}}}%
2025         {}}%
2026       \fi
2027     \else
2028       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2029       \ifin@ % New way
2030       \bbl@exp{%
2031         \\bbl@add\<captions#1>{\bbl@scset\<#2name>\<#1#2name>}%
2032         \\bbl@ifsamestring{\bbl@tempa}{\language}%
2033         {\bbl@scset\<#2name>\<#1#2name>}%
2034         {}}%
2035       \else % Old way, but defined in the new way
2036       \bbl@exp{%
2037         \\bbl@add\<captions#1>{\def\<#2name>{\<#1#2name>}}%
2038         \\bbl@ifsamestring{\bbl@tempa}{\language}%

```

```

2039      {\def\<#2name>{\<#1#2name>}}%
2040      {}}%
2041      \fi%
2042      \fi
2043      \@namedef{#1#2name}{#3}%
2044      \toks@{\expandafter{\bbl@captionslist}%
2045      \bbl@exp{\in{\<#2name>}{\the\toks@}}}%
2046      \ifin@else
2047      \bbl@exp{\bbl@add{\bbl@captionslist{\<#2name>}}}%
2048      \bbl@tglobal\bbl@captionslist
2049      \fi
2050      \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2051 \bbl@trace{Macros related to glyphs}
2052 \def\set@low@box#1{\setbox\tw@{\hbox{,}}\setbox\z@{\hbox{#1}}%
2053   \dimen\z@{\ht\z@ \advance\dimen\z@ -\ht\tw@}%
2054   \setbox\z@{\hbox{\lower\dimen\z@ \box\z@}\ht\z@{\ht\tw@ \dp\z@\dp\tw@}}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2055 \def\save@sf@q#1{\leavevmode
2056   \begingroup
2057   \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2058   \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2059 \ProvideTextCommand{\quotedblbase}{OT1}{%
2060   \save@sf@q{\set@low@box{\textquotedblright/}}%
2061   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2062 \ProvideTextCommandDefault{\quotedblbase}{%
2063   \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2064 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2065   \save@sf@q{\set@low@box{\textquoteright/}}%
2066   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2067 \ProvideTextCommandDefault{\quotesinglbase}{%
2068   \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```

2069 \ProvideTextCommand{\guillemetleft}{OT1}{%
2070   \ifmmode
2071     \ll
2072   \else
2073     \save@sf@q{\nobreak
2074       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2075   \fi}
2076 \ProvideTextCommand{\guillemetright}{OT1}{%
2077   \ifmmode
2078     \gg
2079   \else
2080     \save@sf@q{\nobreak
2081       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2082   \fi}
2083 \ProvideTextCommand{\guillemotleft}{OT1}{%
2084   \ifmmode
2085     \ll
2086   \else
2087     \save@sf@q{\nobreak
2088       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2089   \fi}
2090 \ProvideTextCommand{\guillemotright}{OT1}{%
2091   \ifmmode
2092     \gg
2093   \else
2094     \save@sf@q{\nobreak
2095       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2096   \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2097 \ProvideTextCommandDefault{\guillemetleft}{%
2098   \UseTextSymbol{OT1}{\guillemetleft}}
2099 \ProvideTextCommandDefault{\guillemetright}{%
2100   \UseTextSymbol{OT1}{\guillemetright}}
2101 \ProvideTextCommandDefault{\guillemotleft}{%
2102   \UseTextSymbol{OT1}{\guillemotleft}}
2103 \ProvideTextCommandDefault{\guillemotright}{%
2104   \UseTextSymbol{OT1}{\guillemotright}}

```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```

2105 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2106   \ifmmode
2107     <%
2108   \else
2109     \save@sf@q{\nobreak
2110       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2111   \fi}
2112 \ProvideTextCommand{\guilsinglright}{OT1}{%
2113   \ifmmode
2114     >%
2115   \else
2116     \save@sf@q{\nobreak
2117       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2118   \fi}

```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```

2119 \ProvideTextCommandDefault{\guilsinglleft}{%
2120   \UseTextSymbol{OT1}{\guilsinglleft}}
2121 \ProvideTextCommandDefault{\guilsinglright}{%
2122   \UseTextSymbol{OT1}{\guilsinglright}}

```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter 'ij'. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2123 \DeclareTextCommand{\ij}{OT1}{%
2124   i\kern-0.02em\bbl@allowhyphens j}
2125 \DeclareTextCommand{\IJ}{OT1}{%
2126   I\kern-0.02em\bbl@allowhyphens J}
2127 \DeclareTextCommand{\ij}{T1}{\char188}
2128 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2129 \ProvideTextCommandDefault{\ij}{%
2130   \UseTextSymbol{OT1}{\ij}}
2131 \ProvideTextCommandDefault{\IJ}{%
2132   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2133 \def\crrtic@{\hrule height0.1ex width0.3em}
2134 \def\crttic@{\hrule height0.1ex width0.33em}
2135 \def\ddj@{%
2136   \setbox0\hbox{d}\dimen@=\ht0
2137   \advance\dimen@lex
2138   \dimen@.45\dimen@
2139   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2140   \advance\dimen@ii.5ex
2141   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2142 \def\DDJ@{%
2143   \setbox0\hbox{D}\dimen@=.55\ht0
2144   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2145   \advance\dimen@ii.15ex % correction for the dash position
2146   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2147   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2148   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crttic@}}}}
2149 %
2150 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2151 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2152 \ProvideTextCommandDefault{\dj}{%
2153   \UseTextSymbol{OT1}{\dj}}
2154 \ProvideTextCommandDefault{\DJ}{%
2155   \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2156 \DeclareTextCommand{\SS}{OT1}{SS}
2157 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```
2158 \ProvideTextCommandDefault{\glq}{%  
2159 \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2160 \ProvideTextCommand{\grq}{T1}{%  
2161 \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}  
2162 \ProvideTextCommand{\grq}{TU}{%  
2163 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}  
2164 \ProvideTextCommand{\grq}{OT1}{%  
2165 \save@sf@q{\kern-.0125em  
2166 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%  
2167 \kern.07em\relax}}  
2168 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}
```

\glqq

\grqq The ‘german’ double quotes.

```
2169 \ProvideTextCommandDefault{\glqq}{%  
2170 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}
```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2171 \ProvideTextCommand{\grqq}{T1}{%  
2172 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2173 \ProvideTextCommand{\grqq}{TU}{%  
2174 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2175 \ProvideTextCommand{\grqq}{OT1}{%  
2176 \save@sf@q{\kern-.07em  
2177 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%  
2178 \kern.07em\relax}}  
2179 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}
```

\flq

\frq The ‘french’ single guillemets.

```
2180 \ProvideTextCommandDefault{\flq}{%  
2181 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}  
2182 \ProvideTextCommandDefault{\frq}{%  
2183 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

\flqq

\frqq The ‘french’ double guillemets.

```
2184 \ProvideTextCommandDefault{\flqq}{%  
2185 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}  
2186 \ProvideTextCommandDefault{\frqq}{%  
2187 \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command “\” needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```

2188 \def\umlauthigh{%
2189   \def\bbbl@umlauta##1{\leavevmode\bgroup%
2190     \accent\csname\fontencoding dqpos\endcsname
2191     ##1\bbbl@allowhyphens\egroup}%
2192   \let\bbbl@umlaute\bbbl@umlauta}
2193 \def\umlautlow{%
2194   \def\bbbl@umlauta{\protect\lower@umlaut}}
2195 \def\umlautelowlow{%
2196   \def\bbbl@umlaute{\protect\lower@umlaut}}
2197 \umlauthigh

```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra (*dimen*) register.

```

2198 \expandafter\ifx\csname U@D\endcsname\relax
2199   \csname newdimen\endcsname\U@D
2200 \fi

```

The following code fools $\TeX$'s `make\_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```

2201 \def\lower@umlaut#1{%
2202   \leavevmode\bgroup
2203   \U@D lex%
2204   {\setbox\z@\hbox{%
2205     \char\csname\fontencoding dqpos\endcsname}%
2206     \dimen@ -.45ex\advance\dimen@ht\z@
2207     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2208   \accent\csname\fontencoding dqpos\endcsname
2209   \fontdimen5\font\U@D #1%
2210   \egroup}

```

For all vowels we declare `\` to be a composite command which uses `\bbbl@umlauta` or `\bbbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbbl@umlauta` and/or `\bbbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```

2211 \AtBeginDocument{%
2212   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbbl@umlauta{a}}%
2213   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbbl@umlaute{e}}%
2214   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbbl@umlaute{i}}%
2215   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbbl@umlaute{i}}%
2216   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbbl@umlauta{o}}%
2217   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbbl@umlauta{u}}%
2218   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbbl@umlauta{A}}%
2219   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbbl@umlaute{E}}%
2220   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbbl@umlaute{I}}%
2221   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbbl@umlauta{O}}%
2222   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```

2223 \ifx\l@english\undefined
2224   \chardef\l@english\z@
2225 \fi

```

```

2226 % The following is used to cancel rules in ini files (see Amharic).
2227 \ifx\l@unhyphenated\@undefined
2228   \newlanguage\l@unhyphenated
2229 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2230 \bbl@trace{Bidi layout}
2231 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2232 \bbl@trace{Input engine specific macros}
2233 \ifcase\bbl@engine
2234   \input txtbabel.def
2235 \or
2236   \input luababel.def
2237 \or
2238   \input xebabel.def
2239 \fi
2240 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}
2241 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}
2242 \ifx\babelposthyphenation\@undefined
2243   \let\babelposthyphenation\babelprehyphenation
2244   \let\babelpatterns\babelprehyphenation
2245   \let\babelcharproperty\babelprehyphenation
2246 \fi
2247 (/package|core)

```

4.18. Creating and modifying languages

Continue with $\LaTeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded `ldf` files.

```

2248 (*package)
2249 \bbl@trace{Creating languages and reading ini files}
2250 \let\bbl@extend@ini\@gobble
2251 \newcommand\babelprovide[2][]{%
2252   \let\bbl@savelangname\languagename
2253   \edef\bbl@savelocaleid{\the\localeid}%
2254   \global\let\bbl@afterload\@empty
2255   % Set name and locale id
2256   \edef\languagename{#2}%
2257   \bbl@id@assign
2258   % Initialize keys
2259   \bbl@vforeach{captions,date,import,main,script,language,%
2260     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2261     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2262     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2263     @import}%
2264     {\bbl@csarg\let{KVP@##1}\@nnil}%
2265   \global\let\bbl@release@transforms\@empty
2266   \global\let\bbl@release@casing\@empty
2267   \let\bbl@calendars\@empty
2268   \global\let\bbl@inidata\@empty
2269   \global\let\bbl@extend@ini\@gobble
2270   \global\let\bbl@included@inis\@empty
2271   \gdef\bbl@key@list{;}%

```

```

2272 \bbl@ifunset{bbl@passto@#2}%
2273   {\def\bbl@tempa{#1}}%
2274   {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2275 \expandafter\bbl@forkv\expandafter\bbl@tempa}%
2276 \in@{/}{#1}% With /, (re)sets a value in the ini
2277 \ifin@
2278   \bbl@renewinikey##1\@{##2}%
2279 \else
2280   \bbl@csarg\ifx{KVP@##1}\@nnil\else
2281     \bbl@error{unknown-provide-key}{#1}{}%
2282   \fi
2283   \bbl@csarg\def{KVP@##1}{##2}%
2284 \fi}%
2285 \chardef\bbl@howloaded= 0:none; 1:ldf without ini; 2:ini
2286 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2287 % == init ==
2288 \ifx\bbl@screset\undefined
2289   \bbl@ldfinit
2290 \fi
2291 % ==
2292 % If there is no import (last wins), use @import (internal, there
2293 % must be just one). To consider any order (because
2294 % \PassOptionsToLocale).
2295 \ifx\bbl@KVP@import\@nnil
2296   \let\bbl@KVP@import\bbl@KVP@@import
2297 \fi
2298 % == date (as option) ==
2299 % \ifx\bbl@KVP@date\@nnil\else
2300 % \fi
2301 % ==
2302 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2303 \ifcase\bbl@howloaded
2304   \let\bbl@lbkflag\@empty % new
2305 \else
2306   \ifx\bbl@KVP@hyphenrules\@nnil\else
2307     \let\bbl@lbkflag\@empty
2308   \fi
2309   \ifx\bbl@KVP@import\@nnil\else
2310     \let\bbl@lbkflag\@empty
2311   \fi
2312 \fi
2313 % == import, captions ==
2314 \ifx\bbl@KVP@import\@nnil\else
2315   \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2316   {\ifx\bbl@initoload\relax
2317     \begingroup
2318       \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2319       \bbl@input@texini{##2}%
2320     \endgroup
2321   \else
2322     \xdef\bbl@KVP@import{\bbl@initoload}%
2323   \fi}%
2324   {}%
2325   \let\bbl@KVP@date\@empty
2326 \fi
2327 \let\bbl@KVP@captions@@\bbl@KVP@captions
2328 \ifx\bbl@KVP@captions\@nnil
2329   \let\bbl@KVP@captions\bbl@KVP@import
2330 \fi
2331 % ==
2332 \ifx\bbl@KVP@transforms\@nnil\else
2333   \bbl@replace\bbl@KVP@transforms{ }{,}%
2334 \fi

```

```

2335 % Load ini
2336 % -----
2337 \ifcase\bbl@howloaded
2338   \bbl@provide@new{#2}%
2339 \else
2340   \bbl@ifblank{#1}%
2341   {}% With \bbl@load@basic below
2342   {\bbl@provide@renew{#2}}%
2343 \fi
2344 % Post tasks
2345 % -----
2346 % == subsequent calls after the first provide for a locale ==
2347 \ifx\bbl@inidata\@empty\else
2348   \bbl@extend@ini{#2}%
2349 \fi
2350 % ==
2351 \ifx\bbl@KVP@mapdot\@nnil\else
2352   \def\bbl@tempa{\@empty}%
2353   \ifx\bbl@KVP@mapdot\bbl@tempa\else
2354     \bbl@exp{\gdef\<bbl@map@. @\language>{\[bbl@KVP@mapdot]}}%
2355   \fi
2356 \fi
2357 % == ensure captions ==
2358 \ifx\bbl@KVP@captions\@nnil\else
2359   \bbl@ifunset{bbl@extracaps@#2}%
2360   {\bbl@exp{\[bbl@babelensure[exclude=\\today]{#2}}}%
2361   {\bbl@exp{\[bbl@babelensure[exclude=\\today,
2362     include=\[bbl@extracaps@#2]]{#2}}}%
2363   \let\bbl@tempc\language
2364   \bbl@replace\bbl@tempc{-}{@}%
2365   \bbl@ifunset{bbl@ensure@\bbl@tempc}%
2366   {\bbl@exp{%
2367     \\DeclareRobustCommand\<bbl@ensure@\bbl@tempc>[1]{%
2368       \\foreignlanguage{\language}%
2369       {###1}}}%
2370   }%
2371   \bbl@exp{%
2372     \\bbl@tglobal\<bbl@ensure@\bbl@tempc>%
2373     \\bbl@tglobal\<bbl@ensure@\bbl@tempc\space>%
2374   \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2375 \bbl@load@basic{#2}%
2376 % == script, language ==
2377 % Override the values from ini or defines them
2378 \ifx\bbl@KVP@script\@nnil\else
2379   \bbl@csarg\edef{sname@#2}{\bbl@KVP@script}%
2380 \fi
2381 \ifx\bbl@KVP@language\@nnil\else
2382   \bbl@csarg\edef{lname@#2}{\bbl@KVP@language}%
2383 \fi
2384 \ifcase\bbl@engine\or
2385   \bbl@ifunset{bbl@chrng@\language}%
2386   {\directlua{
2387     Babel.set_chranges_b('\bbl@cl{sbc}', '\bbl@cl{chrng}') }}%
2388 \fi
2389 % == Line breaking: intraspace, intrapenalty ==
2390 % For CJK, East Asian, Southeast Asian, if interspace in ini
2391 \ifx\bbl@KVP@intraspace\@nnil\else % We can override the ini or set
2392   \bbl@csarg\edef{intsp@#2}{\bbl@KVP@intraspace}%
2393 \fi

```

```

2394 \bbl@provide@intraspace
2395 % == Line breaking: justification ==
2396 \ifx\bbl@KVP@justification\@nnil\else
2397   \let\bbl@KVP@linebreaking\bbl@KVP@justification
2398 \fi
2399 \ifx\bbl@KVP@linebreaking\@nnil\else
2400   \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2401   {,elongated,kashida,cjk,padding,unhyphenated,}%
2402 \ifin@
2403   \bbl@csarg\xdef
2404     {\lbrk@{\language\name}}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2405 \fi
2406 \fi
2407 \bbl@xin@{/e}{/\bbl@cl{\lbrk}}%
2408 \ifin@ \else \bbl@xin@{/k}{/\bbl@cl{\lbrk}} \fi
2409 \ifin@ \bbl@arabicjust \fi
2410 \bbl@xin@{/p}{/\bbl@cl{\lbrk}}%
2411 \ifin@ \AtBeginDocument{\@nameuse{\bbl@tibetanjust}} \fi
2412 % == Line breaking: hyphenate.other.(locale|script) ==
2413 \ifx\bbl@lbfkflag\@empty
2414   \bbl@ifunset{\bbl@hyotl@{\language\name}}{ }%
2415   {\bbl@csarg\bbl@replace{\hyotl@{\language\name}}{ }{,}%
2416   \bbl@startcommands*{\language\name}}{ }%
2417   \bbl@csarg\bbl@foreach{\hyotl@{\language\name}}{ }%
2418   \ifcase\bbl@engine
2419     \ifnum##1<257
2420       \SetHyphenMap{\BabelLower{##1}{##1}}%
2421     \fi
2422   \else
2423     \SetHyphenMap{\BabelLower{##1}{##1}}%
2424   \fi}%
2425 \bbl@endcommands}%
2426 \bbl@ifunset{\bbl@hyots@{\language\name}}{ }%
2427 {\bbl@csarg\bbl@replace{\hyots@{\language\name}}{ }{,}%
2428 \bbl@csarg\bbl@foreach{\hyots@{\language\name}}{ }%
2429 \ifcase\bbl@engine
2430   \ifnum##1<257
2431     \global\lccode##1=##1\relax
2432   \fi
2433   \else
2434     \global\lccode##1=##1\relax
2435   \fi}}%
2436 \fi
2437 % == Counters: maparabic ==
2438 % Native digits, if provided in ini (TeX level, xe and lua)
2439 \ifcase\bbl@engine\else
2440   \bbl@ifunset{\bbl@dgnat@{\language\name}}{ }%
2441   {\expandafter\ifx\csname\bbl@dgnat@{\language\name}\endcsname\@empty\else
2442     \expandafter\expandafter\expandafter
2443     \bbl@setdigits\csname\bbl@dgnat@{\language\name}\endcsname
2444     \ifx\bbl@KVP@maparabic\@nnil\else
2445       \ifx\bbl@latinarabic\@undefined
2446         \expandafter\let\expandafter\@arabic
2447           \csname\bbl@counter@{\language\name}\endcsname
2448       \else % i.e., if layout=counters, which redefines \@arabic
2449         \expandafter\let\expandafter\bbl@latinarabic
2450           \csname\bbl@counter@{\language\name}\endcsname
2451       \fi
2452     \fi
2453   \fi}%
2454 \fi
2455 % == Counters: mapdigits ==
2456 % > luababel.def

```

```

2457 % == Counters: alph, Alph ==
2458 \ifx\bbbl@KVP@alph\@nnil\else
2459   \bbbl@exp{%
2460     \\bbbl@add\<bbbl@preextras@\language\name>{%
2461       \\babel@save\\@alph
2462       \let\\@alph\<bbbl@cntr@\bbbl@KVP@alph @\language\name>}}%
2463 \fi
2464 \ifx\bbbl@KVP@Alph\@nnil\else
2465   \bbbl@exp{%
2466     \\bbbl@add\<bbbl@preextras@\language\name>{%
2467       \\babel@save\\@Alph
2468       \let\\@Alph\<bbbl@cntr@\bbbl@KVP@Alph @\language\name>}}%
2469 \fi
2470 % == Counters: mapdot ==
2471 \ifx\bbbl@KVP@mapdot\@nnil\else
2472   \bbbl@foreach\bbbl@list@the{%
2473     \bbbl@ifunset{the##1}{}%
2474     {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2475       \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2476       \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2477         \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2478         \fi}}%
2479   \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2480   \bbbl@foreach\bbbl@tempb{%
2481     \bbbl@ifunset{label##1}{}%
2482     {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2483       \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2484       \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2485         \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2486         \fi}}%
2487 \fi
2488 % == Casing ==
2489 \bbbl@release@casing
2490 \ifx\bbbl@KVP@casing\@nnil\else
2491   \bbbl@csarg\xdef{casing@\language\name}%
2492   {\@nameuse{bbbl@casing@\language\name}\bbbl@maybextx\bbbl@KVP@casing}%
2493 \fi
2494 % == Calendars ==
2495 \ifx\bbbl@KVP@calendar\@nnil
2496   \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2497 \fi
2498 \def\bbbl@tempe##1 ##2\@{ % Get first calendar
2499   \def\bbbl@tempa{##1}%
2500   \bbbl@exp{\bbbl@tempe\bbbl@KVP@calendar\space\@}%
2501 \def\bbbl@tempe##1.##2.##3.##4\@{ %
2502   \def\bbbl@tempf{##3}% First
2503   \let\bbbl@tempc\@empty \let\bbbl@tempb\@empty
2504   \bbbl@cal@getconvert\bbbl@tempc{##1}%
2505   \bbbl@cal@getconvert\bbbl@tempb{##2}%
2506   \expandafter\bbbl@tempe\bbbl@tempa...\@
2507   \bbbl@csarg\edef{calpr@\language\name}{%
2508     \ifx\bbbl@tempc\@empty\else
2509       calendar=\bbbl@tempc
2510     \fi
2511     \ifx\bbbl@tempb\@empty\else
2512       ,variant=\bbbl@tempb
2513     \fi
2514     \ifx\bbbl@tempf\@empty\else
2515       ,convert=\bbbl@tempf
2516     \fi}%
2517 % == engine specific extensions ==
2518 % Defined in XXXbabel.def
2519 \bbbl@provide@extra{##2}%

```

```

2520 % == require.babel in ini ==
2521 % To load or reload the babel-*.tex, if require.babel in ini
2522 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2523   \bbl@ifunset{bbl@rqtex@\language\name}{}%
2524     {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else
2525       \let\BabelBeforeIni@gobbletwo
2526       \chardef\atcatcode=\catcode\@
2527       \catcode\@=11\relax
2528       \def\CurrentOption{#2}%
2529       \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2530       \catcode\@=\atcatcode
2531       \let\atcatcode\relax
2532       \global\bbl@csarg\let{rqtex@\language\name}\relax
2533     \fi}%
2534 \bbl@foreach\bbl@calendars{%
2535   \bbl@ifunset{bbl@ca##1}{%
2536     \chardef\atcatcode=\catcode\@
2537     \catcode\@=11\relax
2538     \InputIfFileExists{babel-ca-##1.tex}{}{}%
2539     \catcode\@=\atcatcode
2540     \let\atcatcode\relax}%
2541   {}}%
2542 \fi
2543 % == frenchspacing ==
2544 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2545 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2546 \ifin@
2547   \bbl@extras@wrap{\bbl@pre@fs}%
2548   {\bbl@pre@fs}%
2549   {\bbl@post@fs}%
2550 \fi
2551 % == transforms ==
2552 % > luababel.def
2553 \def\CurrentOption{#2}%
2554 \@nameuse{bbl@icsave#2}%
2555 % == main ==
2556 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2557   \let\language\bbl@savelangname
2558   \chardef\localeid\bbl@savelocaleid\relax
2559 \fi
2560 % == ==
2561 \ifx\ldf@finish\@onlypreamble\else
2562   \bbl@afterload
2563 \fi
2564 % == hyphenrules (apply if current) ==
2565 \ifx\bbl@KVP@hyphenrules\@nnil\else
2566   \ifnum\bbl@savelocaleid=\localeid
2567     \language\@nameuse{l@\language\name}%
2568   \fi
2569 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2570 \def\bbl@provide@new#1{%
2571   \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2572   \@namedef{extras#1}{}%
2573   \@namedef{noextras#1}{}%
2574   \bbl@startcommands*{#1}{captions}%
2575   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2576     \def\bbl@tempb##1{% elt for \bbl@captionslist
2577       \ifx##1\@nnil\else
2578         \bbl@exp{%
2579           \\SetString\\##1{%

```

```

2580         \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2581     \expandafter\bbl@tempb
2582 \fi}%
2583 \expandafter\bbl@tempb\bbl@captionslist\@nnil
2584 \else
2585     \ifx\bbl@initoload\relax
2586         \bbl@read@ini{\bbl@KVP@captions}2% % Here letters cat = 11
2587     \else
2588         \bbl@read@ini{\bbl@initoload}2% % Same
2589     \fi
2590 \fi
2591 \StartBabelCommands*{#1}{date}%
2592 \ifx\bbl@KVP@date\@nnil
2593     \bbl@exp{%
2594         \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2595 \else
2596     \bbl@savetoday
2597     \bbl@savedate
2598 \fi
2599 \bbl@endcommands
2600 \bbl@load@basic{#1}%
2601 % == hyphenmins == (only if new)
2602 \bbl@exp{%
2603     \gdef\<#1hyphenmins>{%
2604         {\bbl@ifunset{\bbl@lfthm@#1}{2}{\bbl@cs{lfthm@#1}}}%
2605         {\bbl@ifunset{\bbl@rgthm@#1}{3}{\bbl@cs{rgthm@#1}}}}}%
2606 % == hyphenrules (also in renew) ==
2607 \bbl@provide@hyphens{#1}%
2608 % == main ==
2609 \ifx\bbl@KVP@main\@nnil\else
2610     \expandafter\main@language\expandafter{#1}%
2611 \fi}
2612 %
2613 \def\bbl@provide@renew#1{%
2614     \ifx\bbl@KVP@captions\@nnil\else
2615         \StartBabelCommands*{#1}{captions}%
2616         \bbl@read@ini{\bbl@KVP@captions}2% % Here all letters cat = 11
2617     \EndBabelCommands
2618 \fi
2619 \ifx\bbl@KVP@date\@nnil\else
2620     \StartBabelCommands*{#1}{date}%
2621     \bbl@savetoday
2622     \bbl@savedate
2623 \EndBabelCommands
2624 \fi
2625 % == hyphenrules (also in new) ==
2626 \ifx\bbl@lbfkflag\@empty
2627     \bbl@provide@hyphens{#1}%
2628 \fi
2629 % == main ==
2630 \ifx\bbl@KVP@main\@nnil\else
2631     \expandafter\main@language\expandafter{#1}%
2632 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2633 \def\bbl@load@basic#1{%
2634     \ifcase\bbl@howloaded\or\or
2635         \ifcase\csname bbl@llevel@\language\endcsname
2636             \bbl@csarg\let{lname@\language}\relax
2637         \fi
2638     \fi

```

```

2639 \bbl@ifunset{bbl@lname@#1}%
2640 {\def\BabelBeforeIni##1##2{%
2641   \begingroup
2642     \let\bbl@ini@captions@aux\@gobbletwo
2643     \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6}%
2644     \bbl@read@ini{##1}l%
2645     \ifx\bbl@initoload\relax\endinput\fi
2646   \endgroup}%
2647   \begingroup      % boxed, to avoid extra spaces:
2648   \ifx\bbl@initoload\relax
2649     \bbl@input@texini{#1}%
2650   \else
2651     \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2652   \fi
2653   \endgroup}%
2654   {}%

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2655 \def\bbl@load@info#1{%
2656   \def\BabelBeforeIni##1##2{%
2657     \begingroup
2658       \bbl@read@ini{##1}0%
2659       \endinput      % babel- .tex may contain onlypreamble's
2660     \endgroup}%
2661     {\bbl@input@texini{#1}}%

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2662 \def\bbl@provide@hyphens#1{%
2663   \@tempcnta@m@ne % a flag
2664   \ifx\bbl@KVP@hyphenrules\@nnil\else
2665     \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2666     \bbl@foreach\bbl@KVP@hyphenrules{%
2667       \ifnum\@tempcnta=\m@ne % if not yet found
2668         \bbl@ifsamestring{##1}{+}%
2669         {\bbl@carg\addlanguage{l@#1}}%
2670       }%
2671       \bbl@ifunset{l@#1}% After a possible +
2672       {}%
2673       {\@tempcnta\@nameuse{l@#1}}%
2674     \fi}%
2675   \ifnum\@tempcnta=\m@ne
2676     \bbl@warning{%
2677       Requested 'hyphenrules' for '\language' not found:\\%
2678       \bbl@KVP@hyphenrules.\\%
2679       Using the default value. Reported}%
2680   \fi
2681   \fi
2682   \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2683     \ifx\bbl@KVP@captions@\@nnil
2684       \bbl@ifunset{bbl@hyphr@#1}{}% use value in ini, if exists
2685       {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2686       }%
2687       {\bbl@ifunset{l@\bbl@cl{hyphr}}}%
2688       {}%
2689       {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}%
2690     \fi
2691     \fi
2692     \bbl@ifunset{l@#1}%
2693     {\ifnum\@tempcnta=\m@ne
2694       \bbl@carg\adddialect{l@#1}\language

```

```

2695 \else
2696 \bbl@carg\adddialect{l@#1}\@tempcnta
2697 \fi}%
2698 {\ifnum\@tempcnta=\m@ne\else
2699 \global\bbl@carg\chardef{l@#1}\@tempcnta
2700 \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2701 \def\bbl@input@texini#1{%
2702 \bbl@bsphack
2703 \bbl@exp{%
2704 \catcode`\\=\14 \catcode`\\=\0
2705 \catcode`\\=\1 \catcode`\\=\2
2706 \lowercase{\\InputIfFileExists{babel-#1.tex}{}}}%
2707 \catcode`\\=\the\catcode`\relax
2708 \catcode`\\=\the\catcode`\\relax
2709 \catcode`\\=\the\catcode`\relax
2710 \catcode`\\=\the\catcode`\relax}%
2711 \bbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```

2712 \def\bbl@iniline#1\bbl@iniline{%
2713 \@ifnextchar[\bbl@iniset{\@ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2714 \def\bbl@iniset[#1]#2\@@{\def\bbl@section{#1}}
2715 \def\bbl@iniskip#1\@@{% if starts with ;
2716 \def\bbl@inistore#1=#2\@@{% full (default)
2717 \bbl@trim@def\bbl@tempa{#1}%
2718 \bbl@trim\toks@{#2}%
2719 \bbl@ifsamestring{\bbl@tempa}{\include}%
2720 {\bbl@read@subini{\the\toks@}}%
2721 {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2722 \ifin@ \else
2723 \bbl@xin@{,identification/include.}%
2724 {,\bbl@section/\bbl@tempa}%
2725 \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2726 \bbl@exp{%
2727 \\g@addto@macro\\bbl@inidata{%
2728 \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2729 \fi}}
2730 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2731 \bbl@trim@def\bbl@tempa{#1}%
2732 \bbl@trim\toks@{#2}%
2733 \bbl@xin@{.identification.}{.\bbl@section.}%
2734 \ifin@
2735 \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2736 \\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2737 \fi}

```

4.19. Main loop in 'provide'

Now, the 'main loop', \bbl@read@ini, which **must be executed inside a group**. At this point, \bbl@inidata may contain data declared in \babelprovide, with 'slashed' keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, 'export' some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```

2738 \def\bbl@loop@ini#1{%
2739   \loop
2740     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2741     \endlinechar\m@ne
2742     \read#1 to \bbl@line
2743     \endlinechar\^^M
2744     \ifx\bbl@line\empty\else
2745       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2746     \fi
2747   \repeat}
2748 %
2749 \def\bbl@read@subini#1{%
2750   \ifx\bbl@readsubstream\undefined
2751     \csname newread\endcsname\bbl@readsubstream
2752   \fi
2753   \openin\bbl@readsubstream=babel-#1.ini
2754   \ifeof\bbl@readsubstream
2755     \bbl@error{no-ini-file}{#1}{}{}%
2756   \else
2757     {\bbl@loop@ini\bbl@readsubstream}%
2758   \fi
2759   \closein\bbl@readsubstream}
2760 %
2761 \ifx\bbl@readstream\undefined
2762   \csname newread\endcsname\bbl@readstream
2763 \fi
2764 \def\bbl@read@ini#1#2{%
2765   \global\let\bbl@extend@ini@gobble
2766   \openin\bbl@readstream=babel-#1.ini
2767   \ifeof\bbl@readstream
2768     \bbl@error{no-ini-file}{#1}{}{}%
2769   \else
2770     % == Store ini data in \bbl@inidata ==
2771     \catcode\ =10 \catcode\ =12 \catcode\#=12
2772     \catcode\[ =12 \catcode\] =12 \catcode\&=12 \catcode\&=12
2773     \catcode\;=12 \catcode\| =12 \catcode\% =14 \catcode\-=12
2774     \ifnum#2=\m@ne % Just for the info
2775       \edef\language{tag \bbl@metalang}%
2776     \fi
2777     \ifnum#2=\m@ne
2778       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.\\%
2779         Fetching the locale name from babel-#1.ini\@gobble}%
2780     \else
2781       \bbl@info{Importing
2782         \ifcase#2font and identification \or basic \fi
2783         data for '\language'\%
2784         from babel-#1.ini. Reported}%
2785     \fi
2786     \ifnum#2<\@ne
2787       \global\let\bbl@inidata\empty
2788       \let\bbl@inistore\bbl@inistore@min % Remember it's local
2789     \fi
2790     \def\bbl@section{identification}%
2791     \bbl@exp{%
2792       \\bbl@inistore tag.ini=#1\\@@
2793       \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\@@}%
2794     \bbl@loop@ini\bbl@readstream
2795     % == Process stored data ==
2796     \ifnum#2=\m@ne
2797       \def\bbl@tempa##1 ##2\@{##1}% Get first name
2798       \def\bbl@elt##1##2##3{%

```

```

2799     \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2800     {\edef\languagename{\bbl@tempa##3 \@@}%
2801     \let\localename\languagename
2802     \bbl@id@assign
2803     \def\bbl@elt###1###2###3{}}%
2804     {}}%
2805     \bbl@inidata
2806 \fi
2807 \bbl@csarg\xdef\luni@\languagename}{#1}%
2808 \bbl@read@ini@aux
2809 % == 'Export' data ==
2810 \bbl@ini@exports{#2}%
2811 \global\bbl@csarg\let\inidata@\languagename\bbl@inidata
2812 \global\let\bbl@inidata\empty
2813 \bbl@xin@{,\languagename,},{,\bbl@ini@loaded,}%
2814 \ifin@else
2815     \bbl@exp{\bbl@add@list\bbl@ini@loaded{\languagename}}%
2816     \bbl@tglobal\bbl@ini@loaded
2817 \fi
2818 \@ifpackagewith{babel}{licr=unextended}{}%
2819     {\ifcase\bbl@engine\else % Find a better place
2820     \bbl@xin@{,\bbl@cl{sbcpr},},{,Cyr1,}%
2821     \ifin@
2822     \bbl@once{licr-cryl}{\g@addto@macro\bbl@afterload{\input{cyr12uni.def}}}%
2823     \fi
2824     \fi}%
2825 \fi
2826 \closein\bbl@readstream}
2827 \def\bbl@read@ini@aux{%
2828     \let\bbl@savestrings\empty
2829     \let\bbl@savetoday\empty
2830     \let\bbl@savestate\empty
2831     \def\bbl@elt###1##2###3{%
2832     \def\bbl@section{##1}%
2833     \in@{=date.}{=##1}% Find a better place
2834     \ifin@
2835     \bbl@ifunset{bbl@inikv@##1}%
2836     {\bbl@ini@calendar{##1}}%
2837     {}}%
2838     \fi
2839     \bbl@ifunset{bbl@inikv@##1}{}%
2840     {\csname bbl@inikv@##1\endcsname{##2}{##3}}}%
2841     \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2842 \def\bbl@extend@ini@aux#1{%
2843     \bbl@startcommands*{#1}{captions}%
2844     % Activate captions/... and modify exports
2845     \bbl@csarg\def{inikv@captions.licr}##1##2{%
2846     \setlocalecaption{#1}{##1}{##2}}%
2847     \def\bbl@inikv@captions##1##2{%
2848     \bbl@ini@captions@aux{##1}{##2}}%
2849     \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2850     \def\bbl@exportkey##1##2##3{%
2851     \bbl@ifunset{bbl@kv@##2}{}%
2852     {\expandafter\ifx\csname bbl@kv@##2\endcsname\empty\else
2853     \bbl@exp{\global\let<bbl@##1@\languagename>\<bbl@kv@##2>}}%
2854     \fi}}%
2855     % As with \bbl@read@ini, but with some changes
2856     \bbl@read@ini@aux
2857     \bbl@ini@exports\tw@
2858     % Update inidata@lang by pretending the ini is read.

```

```

2859 \def\bbel@elt##1##2##3{%
2860 \def\bbel@section{##1}%
2861 \bbel@iniline##2=##3\bbel@iniline}%
2862 \csname bbl@inidata@#1\endcsname
2863 \global\bbel@csarg\let{inidata@#1}\bbel@inidata
2864 \StartBabelCommands*{#1}{date}% And from the import stuff
2865 \def\bbel@stringdef##1##2{\gdef##1{##2}}%
2866 \bbel@savetoday
2867 \bbel@savestate
2868 \bbel@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2869 \def\bbel@ini@calendar#1{%
2870 \lowercase{\def\bbel@tempa{=#1=}}%
2871 \bbel@replace\bbel@tempa{=date.gregorian}{}%
2872 \bbel@replace\bbel@tempa{=date.}{}%
2873 \in@{.licr=}{#1=}%
2874 \ifin@
2875 \ifcase\bbel@engine
2876 \bbel@replace\bbel@tempa{.licr=}{}%
2877 \else
2878 \let\bbel@tempa\relax
2879 \fi
2880 \fi
2881 \ifx\bbel@tempa\relax\else
2882 \bbel@replace\bbel@tempa{=}{}%
2883 \ifx\bbel@tempa@empty\else
2884 \xdef\bbel@calendars{\bbel@calendars,\bbel@tempa}%
2885 \fi
2886 \bbel@exp{%
2887 \def\<bbel@inikv@#1>####1####2{%
2888 \\\bbel@inidate####1...\relax{####2}{\bbel@tempa}}}%
2889 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbel@inistore above).

```

2890 \def\bbel@renewinikey#1/#2\@#3{%
2891 \global\let\bbel@extend@ini\bbel@extend@ini@aux
2892 \edef\bbel@tempa{\zap@space #1 \@empty}% section
2893 \edef\bbel@tempb{\zap@space #2 \@empty}% key
2894 \bbel@trim\toks@{#3}% value
2895 \bbel@exp{%
2896 \edef\\bbel@key@list{\bbel@key@list \bbel@tempa/\bbel@tempb;}%
2897 \\g@addto@macro\\bbel@inidata{%
2898 \\\bbel@elt{\bbel@tempa}{\bbel@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2899 \def\bbel@exportkey#1#2#3{%
2900 \bbel@ifunset{\bbel@kv@#2}%
2901 {\bbel@csarg\gdef{#1@language}{#3}}%
2902 {\expandafter\ifx\csname bbl@kv@#2\endcsname\@empty
2903 \bbel@csarg\gdef{#1@language}{#3}%
2904 \else
2905 \bbel@exp{\global\let\<bbel@#1@language>\<bbel@kv@#2>}}%
2906 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bbel@ini@exports is called always (via \bbel@inisec), while \bbel@after@ini must be called explicitly after \bbel@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdfTeX to select a font encoding valid (and preloaded) for a language loaded on the fly.

```
2907 \def\bbl@iniwarning#1{%
2908   \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2909   {\bbl@warning{%
2910     From babel-\bbl@cs{lini@language}.ini:\\%
2911     \bbl@cs{@kv@identification.warning#1}\\%
2912     Reported}}}%
2913 %
2914 \let\bbl@release@transforms\@empty
2915 \let\bbl@release@casing\@empty
```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```
2916 \def\bbl@ini@exports#1{%
2917   % Identification always exported
2918   \bbl@iniwarning{}%
2919   \ifcase\bbl@engine
2920     \bbl@iniwarning{.pdfLaTeX}%
2921   \or
2922     \bbl@iniwarning{.luaLaTeX}%
2923   \or
2924     \bbl@iniwarning{.XeLaTeX}%
2925   \fi%
2926   \bbl@exportkey{lllevel}{identification.load.level}{}%
2927   \bbl@exportkey{elname}{identification.name.english}{}%
2928   \bbl@exp{\bbl@exportkey{lname}{identification.name.opentype}%
2929     {\csname bbl@elname@language\endcsname}}%
2930   \bbl@exportkey{tbcpr}{identification.tag.bcp47}{}%
2931   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2932   \bbl@exportkey{lbcpr}{identification.language.tag.bcp47}{}%
2933   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2934   \bbl@exportkey{esname}{identification.script.name}{}%
2935   \bbl@exp{\bbl@exportkey{sname}{identification.script.name.opentype}%
2936     {\csname bbl@esname@language\endcsname}}%
2937   \bbl@exportkey{sbcpr}{identification.script.tag.bcp47}{}%
2938   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2939   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2940   \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2941   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2942   \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2943   \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2944   % Also maps bcp47 -> language
2945   \bbl@csarg\def{bcp@map@bbl@cl{tbcpr}}{\language}%
2946   \ifcase\bbl@engine\or
2947     \directlua{%
2948       Babel.locale_props[\the\bbl@cs{id@language}].script
2949       = '\bbl@cl{sbcpr}'}%
2950   \fi
2951   % Conditional
2952   \ifnum#1>\z@ % -1 or 0 = only info, 1 = basic, 2 = (re)new
2953     \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2954     \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2955     \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2956     \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%

```

```

2957 \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2958 \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2959 \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2960 \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2961 \bbl@exportkey{intsp}{typography.intraspace}{}%
2962 \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2963 \bbl@exportkey{chrng}{characters.ranges}{}%
2964 \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2965 \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2966 \ifnum#1=\tw@ % only (re)new
2967 \bbl@exportkey{rqtex}{identification.require.babel}{}%
2968 \bbl@tglobal\bbl@savetoday
2969 \bbl@tglobal\bbl@savedate
2970 \bbl@savestrings
2971 \fi
2972 \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2973 \def\bbl@inikv#1#2{%      key=value
2974 \toks@{#2}%              This hides #'s from ini values
2975 \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2976 \let\bbl@inikv@identification\bbl@inikv
2977 \let\bbl@inikv@date\bbl@inikv
2978 \let\bbl@inikv@typography\bbl@inikv
2979 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2980 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\language}\empty x-\fi}
2981 \def\bbl@inikv@characters#1#2{%
2982 \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2983 {\bbl@exp{%
2984 \\\g@addto@macro{\bbl@release@casing{%
2985 \\\bbl@casemapping}{\language}\unexpanded{#2}}}%
2986 {\in@{$casing.}{$#1}% e.g., casing.Uv = uV
2987 \ifin@
2988 \lowercase{\def\bbl@tempb{#1}%
2989 \bbl@replace\bbl@tempb{casing.}}}%
2990 \bbl@exp{\\\g@addto@macro{\bbl@release@casing{%
2991 \\\bbl@casemapping
2992 {\\\bbl@maybextx\bbl@tempb}{\language}\unexpanded{#2}}}%
2993 \else
2994 \bbl@inikv{#1}{#2}%
2995 \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2996 \def\bbl@inikv@counters#1#2{%
2997 \bbl@ifsamestring{#1}{digits}%
2998 {\bbl@error{digits-is-reserved}{}}}%
2999 {}%
3000 \def\bbl@tempc{#1}%
3001 \bbl@trim@def{\bbl@tempb*}{#2}%
3002 \in@{.1$}{#1$}%
3003 \ifin@
3004 \bbl@replace\bbl@tempc{.1}{}%
3005 \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\language}{%

```

```

3006 \noexpand\bbl@alphanumeric{\bbl@tempc}}%
3007 \fi
3008 \in@{.F.}{#1}%
3009 \ifin@else\in@{.S.}{#1}\fi
3010 \ifin@
3011 \bbl@csarg\protected@xdef{cnt@#1@language}{\bbl@tempb*}%
3012 \else
3013 \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
3014 \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
3015 \bbl@csarg\global\expandafter\let{cnt@#1@language}\bbl@tempa
3016 \fi
3017 \in@{.D$}{#1$}%
3018 \ifin@else\in@{.C$}{#1$}\fi
3019 \ifin@
3020 \bbl@replace\bbl@tempc{.D}{}%
3021 \bbl@replace\bbl@tempc{.C}{}%
3022 \bbl@exp\gdef\<bbl@cnt@bbl@tempc @language>####1{%
3023 \expandafter\\bbl@zhnumeral\\expandafter{\\number####1}{\bbl@tempc}}}%
3024 \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3025 \ifcase\bbl@engine
3026 \bbl@csarg\def{inikv@captions.licr}#1#2{%
3027 \bbl@ini@captions@aux{#1}{#2}}
3028 \else
3029 \def\bbl@inikv@captions#1#2{%
3030 \bbl@ini@captions@aux{#1}{#2}}
3031 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3032 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3033 \bbl@replace\bbl@tempa{.template}{}%
3034 \def\bbl@toreplace{#1}{}%
3035 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3036 \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3037 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3038 \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}}%
3039 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}}%
3040 \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3041 \ifin@
3042 \@nameuse{\bbl@patch\bbl@tempa}%
3043 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3044 \fi
3045 \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3046 \ifin@
3047 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3048 \bbl@exp\gdef\<fnum@\bbl@tempa>{%
3049 \\bbl@ifunset{\bbl@bbl@tempa fmt@\\language}%
3050 {[fnum@\bbl@tempa]}%
3051 {\\@nameuse{\bbl@bbl@tempa fmt@\\language}}}%
3052 \fi}
3053 %
3054 \def\bbl@ini@captions@aux#1#2{%
3055 \bbl@trim\def\bbl@tempa{#1}%
3056 \bbl@xin@{.template}{\bbl@tempa}%
3057 \ifin@
3058 \bbl@ini@captions@template{#2}\language
3059 \else
3060 \bbl@ifblank{#2}%
3061 {\bbl@exp{%
3062 \toks@{\\bbl@nocaption{\bbl@tempa name}{\language\bbl@tempa name}}}%
3063 {\bbl@trim\toks@{#2}}}%

```

```

3064 \bbl@exp{%
3065   \\\bbl@add\\bbl@savestrings{%
3066     \\\SetString<\bbl@tempa name>{\the\toks@}}}%
3067 \toks@ \expandafter{\bbl@captionslist}%
3068 \bbl@exp{\\\in@{\<\bbl@tempa name>}{\the\toks@}}%
3069 \ifin@ \else
3070   \bbl@exp{%
3071     \\\bbl@add<\bbl@extracaps@ \language name>{\<\bbl@tempa name>}}%
3072     \\\bbl@to global \<\bbl@extracaps@ \language name>}%
3073 \fi
3074 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3075 \def\bbl@list@the{%
3076   part,chapter,section,subsection,subsubsection,paragraph,%
3077   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3078   table,page,footnote,mpfootnote,mpfn}
3079 %
3080 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3081   \bbl@ifunset{\bbl@map@#1@ \language name}%
3082     {\@nameuse{#1}}%
3083     {\@nameuse{\bbl@map@#1@ \language name}}}
3084 %
3085 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3086   \ifin@ \else
3087     \bbl@ifunset{\bbl@map@#1@ \language name}%
3088       {#1}%
3089       {\@nameuse{\bbl@map@#1@ \language name}}}
3090 \fi}
3091 %
3092 \def\bbl@inikv@labels#1#2{%
3093   \in@{.map}{#1}%
3094   \ifin@
3095     \in@{,dot.map,}{, #1,}%
3096     \ifin@
3097       \global\@namedef{\bbl@map@. @ \language name}{#2}%
3098       \fi
3099   \ifx\bbl@KVP@labels\@nnil\else
3100     \bbl@xin@{ map }{ \bbl@KVP@labels \space}%
3101     \ifin@
3102       \def\bbl@tempc{#1}%
3103       \bbl@replace\bbl@tempc{.map}{}%
3104       \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3105       \bbl@exp{%
3106         \gdef\<\bbl@map@\bbl@tempc @ \language name>%
3107         {\ifin@ \<#2> \else \\\localecounter{#2} \fi}}%
3108       \bbl@foreach\bbl@list@the{%
3109         \bbl@ifunset{the##1}{}%
3110         {\bbl@ncarg\let\bbl@tempd{the##1}%
3111          \bbl@exp{%
3112            \\\bbl@sreplace\<the##1>%
3113            {\<\bbl@tempc>{##1}}%
3114            {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3115            \\\bbl@sreplace\<the##1>%
3116            {\<\@empty @\bbl@tempc>\<c@##1>%
3117             {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3118             \\\bbl@sreplace\<the##1>%
3119             {\\\csname @\bbl@tempc\\endcsname\<c@##1>%
3120              {\\\bbl@map@cnt{\bbl@tempc}{##1}}}%
3121             \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3122               \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3123               \fi}}%
3124       \fi

```

```

3125 \fi
3126 %
3127 \else
3128 % The following code is still under study. You can test it and make
3129 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3130 % language dependent.
3131 \in@{enumerate.}{#1}%
3132 \ifin@
3133 \def\bbl@tempa{#1}%
3134 \bbl@replace\bbl@tempa{enumerate.}{}%
3135 \def\bbl@toreplace{#2}%
3136 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}%
3137 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3138 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3139 \toks@{\expandafter\bbl@toreplace}%
3140 \bbl@exp{%
3141 \\\bbl@add<extras\language>{%
3142 \\\babel@save<labelenum\romannumeral\bbl@tempa>
3143 \def<labelenum\romannumeral\bbl@tempa>{\the\toks@}%
3144 \\\bbl@tglobal<extras\language>}%
3145 \fi
3146 \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3147 \def\bbl@chapttype{chapter}
3148 \ifx\@makechapterhead\@undefined
3149 \let\bbl@patchchapter\relax
3150 \else\ifx\thechapter\@undefined
3151 \let\bbl@patchchapter\relax
3152 \else\ifx\ps@headings\@undefined
3153 \let\bbl@patchchapter\relax
3154 \else
3155 \def\bbl@patchchapter{%
3156 \global\let\bbl@patchchapter\relax
3157 \gdef\bbl@chfmt{%
3158 \bbl@ifunset\bbl@\bbl@chapttype fmt@\language}%
3159 {\@chapapp\space\thechapter}%
3160 {\@nameuse\bbl@\bbl@chapttype fmt@\language}}}%
3161 \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3162 \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3163 \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3164 \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3165 \bbl@tglobal\appendix
3166 \bbl@tglobal\ps@headings
3167 \bbl@tglobal\chaptermark
3168 \bbl@tglobal\@makechapterhead}
3169 \let\bbl@patchappendix\bbl@patchchapter
3170 \fi\fi\fi
3171 \ifx\@part\@undefined
3172 \let\bbl@patchpart\relax
3173 \else
3174 \def\bbl@patchpart{%
3175 \global\let\bbl@patchpart\relax
3176 \gdef\bbl@partformat{%
3177 \bbl@ifunset\bbl@partfmt@\language}%
3178 {\@partname\nobreakspace\thepart}%
3179 {\@nameuse\bbl@partfmt@\language}}}%
3180 \bbl@sreplace\@part{\@partname\nobreakspace\thepart}{\bbl@partformat}%
3181 \bbl@tglobal\@part}
3182 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3183 \def\bbl@cal@getconvert#1#2{% Used in provide
3184   \in{:}{#2}%
3185   \ifin@def\bbl@tempf{#2}\else\def#1{#2}\fi
3186 \let\bbl@calendar@empty
3187 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3188 \def\bbl@localedate#1#2#3#4{%
3189   \begingroup
3190     \edef\bbl@they{#2}%
3191     \edef\bbl@them{#3}%
3192     \edef\bbl@thed{#4}%
3193     \edef\bbl@tempe{%
3194       \bbl@ifunset\bbl@calpr{\language\language}\bbl@cl{calpr}},%
3195       #1}%
3196     \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3197     \let\bbl@ld@calendar@empty
3198     \let\bbl@ld@variant@empty
3199     \let\bbl@ld@convert@relax
3200     \let\bbl@ld@@convert@relax
3201     \expandafter\bbl@forkv\expandafter{\bbl@tempe}{\bbl@csarg\edef{ld##1}{##2}}%
3202     \bbl@replace\bbl@ld@calendar{gregorian}{}%
3203     \ifx\bbl@ld@convert@relax
3204       \let\bbl@ld@convert\bbl@ld@@convert
3205     \fi
3206     \ifx\bbl@ld@convert@empty
3207       \ifx\bbl@ld@calendar@empty
3208         \let\bbl@ld@convert@relax
3209       \else
3210         \let\bbl@ld@convert\bbl@ld@calendar
3211       \fi
3212     \fi
3213     \ifx\bbl@ld@convert@relax\else
3214       \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3215       {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3216     \fi
3217     \@nameuse{\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3218     \edef\bbl@calendar{% Used in \month..., too
3219       \bbl@ld@calendar
3220       \ifx\bbl@ld@variant@empty\else
3221         .\bbl@ld@variant
3222       \fi}%
3223     \bbl@cased
3224     {\@nameuse{\bbl@date@\language @\bbl@calendar}%
3225      \bbl@they\bbl@them\bbl@thed}%
3226   \endgroup}
3227 %
3228 \def\bbl@printdate#1{%
3229   \ifnextchar[\bbl@printdate@i{#1}]{\bbl@printdate@i{#1}[]}}
3230 \def\bbl@printdate@i#1[#2]#3#4#5{%
3231   \bbl@usedategrouptrue
3232   \def\bbl@tempc{#1}%
3233   \bbl@replace\bbl@tempc{-}{@}%
3234   \@nameuse{\bbl@ensure@\bbl@tempc}{\localedate[#2][#3][#4][#5]}
3235 %
3236 % e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3237 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3238   \bbl@trim@def\bbl@tempa{#1.#2}%
3239   \bbl@ifsamestring{\bbl@tempa}{months.wide}%      to savedate
3240   {\bbl@trim@def\bbl@tempa{#3}%
3241    \bbl@trim\toks@{#5}%
3242    \@temptokena\expandafter{\bbl@savedate}%
3243    \bbl@exp{% Reverse order - in ini last wins

```

```

3244 \def\\bbl@savestate{%
3245   \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3246   \the\temptokena}}}%
3247 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3248 {\lowercase{\def\bbl@tempb{#6}}}%
3249 \bbl@trim@def\bbl@toreplace{#5}%
3250 \bbl@TG@date
3251 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3252 \ifx\bbl@savestate\empty
3253 \bbl@exp{%
3254   \\AfterBabelCommands{%
3255     \gdef\<\language name date>{\\protect\<\language name date >}%
3256     \gdef\<\language name date >{\\bbl@printdate{\language name}}}%
3257   \def\\bbl@savestate{%
3258     \\SetString\\today{%
3259       \<\language name date>[@convert]%
3260       {\the\year}{\the\month}{\the\day}}}%
3261   \fi}%
3262   }}%

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after `\bbl@replace \toks@` contains the resulting string, which is used by `\bbl@replace@finish@iii` (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3263 \let\bbl@calendar\empty
3264 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3265   \bbl@xin@{\$calendrica:}{\$#2}%
3266   \ifin@
3267     \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3268     \edef\bbl@tempa{\$#2}%
3269     \bbl@replace\bbl@tempa{\$calendrica:}{}%
3270     \bbl@replace\bbl@tempa{-}{_}%
3271     \bbl@afterelse
3272     \bbl@exp{\\bbl@calendrica#1\\@@{\bbl@tempa}}%
3273   \else
3274     \bbl@afterfi
3275     \@nameuse{\bbl@ca#2}#1\@@
3276   \fi}
3277 \def\bbl@calendrica#1-#2-#3\@@#4#5#6#7{%
3278   \directlua{
3279     local d = Babel.calendrica.new_date(
3280       {year=\number#1, month=\number#2, day=\number#3},
3281       Babel.calendrica.cal_gregorian)
3282     local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3283     token.set_macro('\bbl@stripslash#5', r.year)
3284     token.set_macro('\bbl@stripslash#6', r.month)
3285     token.set_macro('\bbl@stripslash#7', r.day) }}
3286 \newcommand\BabelDateSpace{\nobreakspace}
3287 \newcommand\BabelDateDot{.\@}
3288 \newcommand\BabelDated[1]{\number#1}
3289 \newcommand\BabelDatedd[1]{\ifnum#1<10 0\fi\number#1}
3290 \newcommand\BabelDateM[1]{\number#1}
3291 \newcommand\BabelDateMM[1]{\ifnum#1<10 0\fi\number#1}
3292 \newcommand\BabelDateMMM[1]{%
3293   \csname month\romannumeral#1\bbl@calendar name\endcsname}%
3294 \newcommand\BabelDatey[1]{\number#1}%
3295 \newcommand\BabelDateyy[1]{%
3296   \ifnum#1<10 0\number#1 %
3297   \else\ifnum#1<100 \number#1 %
3298   \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3299   \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3300   \else

```

```

3301 \bbl@error{limit-two-digits}{\}{\}%
3302 \fi\fi\fi\fi}}
3303 \newcommand\BabelDateyyy[1]{\number#1}}
3304 \newcommand\BabelDateU[1]{\number#1}}%
3305 \def\bbl@replace@finish@iii#1{%
3306 \bbl@exp{\def\#1###1####2####3{\the\toks@}}
3307 \def\bbl@TG@date{%
3308 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace{}}%
3309 \bbl@replace\bbl@toreplace{[. ]}{\BabelDateDot{}}%
3310 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{####1}}%
3311 \bbl@replace\bbl@toreplace{[y|]}{\bbl@datecctr{####1|}}%
3312 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{####1}}%
3313 \bbl@replace\bbl@toreplace{[yyyy]}{\BabelDateyyyy{####1}}%
3314 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{####2}}%
3315 \bbl@replace\bbl@toreplace{[M|]}{\bbl@datecctr{####2|}}%
3316 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{####2}}%
3317 \bbl@replace\bbl@toreplace{[MMM]}{\BabelDateMMM{####2}}%
3318 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{####3}}%
3319 \bbl@replace\bbl@toreplace{[d|]}{\bbl@datecctr{####3|}}%
3320 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{####3}}%
3321 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{####1}}%
3322 \bbl@replace\bbl@toreplace{[U|]}{\bbl@datecctr{####1|}}%
3323 \bbl@replace@finish@iii\bbl@toreplace}
3324 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3325 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3326 \AddToHook{begindocument/before}{%
3327 \let\bbl@normalsf\normalsfcodes
3328 \let\normalsfcodes\relax}
3329 \AtBeginDocument{%
3330 \ifx\bbl@normalsf\empty
3331 \ifnum\sfcodes\@m
3332 \let\normalsfcodes\frenchspacing
3333 \else
3334 \let\normalsfcodes\nonfrenchspacing
3335 \fi
3336 \else
3337 \let\normalsfcodes\bbl@normalsf
3338 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3339 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3340 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3341 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3342 #1[#2]{#3}{#4}{#5}}
3343 \begingroup
3344 \catcode\%=12
3345 \catcode\&=14
3346 \gdef\bbl@transforms#1#2#3{\&%
3347 \directlua{
3348 local str = [=[#2]=]
3349 str = str:gsub('%.%d+%.%d+$', '')
3350 token.set_macro('babeltempa', str)
3351 }&%

```

```

3352 \def\babeltempc{&%
3353 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,&%
3354 \ifin@else
3355 \bbl@xin@{: \babeltempa,}{,\bbl@KVP@transforms,&%
3356 \fi
3357 \ifin@
3358 \bbl@foreach\bbl@KVP@transforms{&%
3359 \bbl@xin@{: \babeltempa,}{,##1,&%
3360 \ifin@ &% font:font:transform syntax
3361 \directlua{
3362 local t = {}
3363 for m in string.gmatch('##1'..'':', '(.):') do
3364 table.insert(t, m)
3365 end
3366 table.remove(t)
3367 token.set_macro('babeltempc', ', fonts=' .. table.concat(t, ' '))
3368 }&%
3369 \fi}&%
3370 \in@{.0$}{#2$}&%
3371 \ifin@
3372 \directlua{&% (\attribute) syntax
3373 local str = string.match([[ \bbl@KVP@transforms]],
3374 '%([^(%[-])%)[^%)]-\babeltempa')
3375 if str == nil then
3376 token.set_macro('babeltempb', '')
3377 else
3378 token.set_macro('babeltempb', ', attribute=' .. str)
3379 end
3380 }&%
3381 \toks@{#3}&%
3382 \bbl@exp{&%
3383 \\g@addto@macro\\bbl@release@transforms{&%
3384 \relax &% Closes previous \bbl@transforms@aux
3385 \\bbl@transforms@aux
3386 \#1{label=\babeltempa\babeltempb\babeltempc}&%
3387 {\language\the\toks@}}&%
3388 \else
3389 \g@addto@macro\bbl@release@transforms{, {#3}}&%
3390 \fi
3391 \fi}
3392 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3393 \def\bbl@provide@lsys#1{%
3394 \bbl@ifunset\bbl@lname@#1{%
3395 {\bbl@load@info{#1}}%
3396 }%
3397 \bbl@csarg\let{lsys@#1}\@empty
3398 \bbl@ifunset\bbl@sname@#1{\bbl@csarg\gdef{sname@#1}{Default}}{&%
3399 \bbl@ifunset\bbl@sotf@#1{\bbl@csarg\gdef{sotf@#1}{DFLT}}{&%
3400 \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3401 \bbl@ifunset\bbl@lname@#1{&%
3402 {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3403 \ifcase\bbl@engine\or\or
3404 \bbl@ifunset\bbl@prehc@#1{&%
3405 {\bbl@exp{\\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3406 }%
3407 {\ifx\bbl@xenohyph\undefined

```

```

3408      \global\let\bbl@xenoxyph\bbl@xenoxyph@d
3409      \ifx\AtBeginDocument\@notprerr
3410        \expandafter\@secondoftwo % to execute right now
3411      \fi
3412      \AtBeginDocument{%
3413        \bbl@patchfont{\bbl@xenoxyph}%
3414        {\expandafter\select@language\expandafter{\language}}}%
3415    \fi}%
3416  \fi
3417  \bbl@csarg\bbl@toglobal{\sys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the `ini` file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\text{T}_{\text{E}}\text{X}$. Non-digits characters are kept. The first macro is the generic “localized” command.

[illegible]

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```

3449 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}%
3450 \ifx\\#1% % \\ before, in case #1 is multiletter
3451 \bbl@exp{%
3452 \def\\bbl@tempa####1{%
3453 \<ifcase>####1\space\the\toks@\<else>\\@ctrerrr\<fi>}}%
3454 \else
3455 \toks@\expandafter{\the\toks@\or #1}%
3456 \expandafter\bbl@buildifcase
3457 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before \@@ collect digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210.

Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```

3458 \newcommand\localenumberal[2]{%
3459   \bbl@ifunset{\bbl@cntr@#1@\language\language}%
3460   {#2}%
3461   {\bbl@cs{cntr@#1@\language\language}{#2}}}
3462 \def\bbl@localecntr#1#2{\localenumberal{#2}{#1}}
3463 \newcommand\localecounter[2]{%
3464   \expandafter\bbl@localecntr
3465   \expandafter{\number\csname c@#2\endcsname}{#1}}
3466 \def\bbl@alphnumer#1#2{%
3467   \expandafter\bbl@alphnumer@i\number#2 76543210\@@{#1}}
3468 \def\bbl@alphnumer@i#1#2#3#4#5#6#7#8\@@#9{%
3469   \ifcase\@car#8\@nil\or    % Currently <10000, but prepared for bigger
3470   \bbl@alphnumer@ii{#9}00000#1\or
3471   \bbl@alphnumer@ii{#9}00000#1#2\or
3472   \bbl@alphnumer@ii{#9}00000#1#2#3\or
3473   \bbl@alphnumer@ii{#9}00000#1#2#3#4\else
3474   \bbl@error{counter-too-large}{>9999}{}}%
3475   \fi}
3476 \def\bbl@alphnumer@ii#1#2#3#4#5#6#7#8{%
3477   \bbl@ifunset{\bbl@cntr@#1.F.\number#5#6#7#8@\language\language}%
3478   {\bbl@cs{cntr@#1.4@\language\language}{#5}%
3479   \bbl@cs{cntr@#1.3@\language\language}{#6}%
3480   \bbl@cs{cntr@#1.2@\language\language}{#7}%
3481   \bbl@cs{cntr@#1.1@\language\language}{#8}%
3482   \ifnum#6#7#8>\z@
3483   \bbl@ifunset{\bbl@cntr@#1.S.321@\language\language}{}}%
3484   {\bbl@cs{cntr@#1.S.321@\language\language}{}}%
3485   \fi}%
3486   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language\language}}}
```

Some Chinese counters follow special rules.

```

3487 \def\bbl@zhnumer#1#2{%
3488   \ifnum#1<10 \bbl@zhnum000#1{#2}%
3489   \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3490   \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3491   \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3492   \else\bbl@error{counter-too-large}{#1}{}}%
3493   \fi\fi\fi\fi}
3494 \def\bbl@zhnum#1#2#3#4#5{%
3495   \ifnum\numexpr#1#2#3#4\relax=\z@
3496   \localenumberal{#5.C}\@ne
3497   \else
3498   \ifnum#1>\z@\localenumberal{#5.D}{#1}\localenumberal{#5.C}5\fi % Thousands
3499   \ifnum#2>\z@\localenumberal{#5.D}{#2}\localenumberal{#5.C}4\fi % Hundreds
3500   \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumberal{#5.C}\@ne\fi
3501   \fi\fi
3502   \ifnum#3>\z@ % Tens
3503   \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumberal{#5.D}{#3}\fi % No  for 10-19:
3504   \else\localenumberal{#5.D}{#3}%
3505   \fi
3506   \localenumberal{#5.C}\thr@@
3507   \else
3508   \ifnum#2>\z@\ifnum#4>\z@\localenumberal{#5.C}\@ne\fi\fi
3509   \fi
3510   \ifnum#4>\z@\localenumberal{#5.D}{#4}\fi % Units
3511   \fi}
```

4.24. Casing

```

3512 \newcommand\BabelUppercaseMapping[3]{%
3513   \DeclareUppercaseMapping[\@nameuse{\bbl@casing@#1}]{#2}{#3}}
```

```

3514 \newcommand\BabelTitlecaseMapping[3]{%
3515   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3516 \newcommand\BabelLowercaseMapping[3]{%
3517   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

  The parser for casing and casing. (variant).
3518 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3519   \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3520 \else
3521   \def\bbl@uftocode#1{\expandafter`\string#1}
3522 \fi
3523 \def\bbl@casemapping#1#2#3{% 1:variant
3524   \def\bbl@tempa##1 ##2{% Loop
3525     \bbl@casemapping@i{##1}%
3526     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3527   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3528   \def\bbl@tempe{0}% Mode (upper/lower...)
3529   \def\bbl@tempc{#3}% Casing list
3530   \expandafter\bbl@tempa\bbl@tempc\@empty}
3531 \def\bbl@casemapping@i#1{%
3532   \def\bbl@tempb{#1}%
3533   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3534     \@nameuse{regex_replace_all:nnN}%
3535     {[{\x{c0}-\x{ff}}][\x{80}-\x{bf}]}*}{\@empty}\bbl@tempb
3536   \else
3537     \@nameuse{regex_replace_all:nnN}{.}{\@empty}\bbl@tempb
3538   \fi
3539   \expandafter\bbl@casemapping@ii\bbl@tempb\@empty}
3540 \def\bbl@casemapping@ii#1#2#3\@empty{%
3541   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3542   \ifin@
3543     \edef\bbl@tempe{%
3544       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3545   \else
3546     \ifcase\bbl@tempe\relax
3547       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3548       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3549     \or
3550       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3551     \or
3552       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3553     \or
3554       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3555     \fi
3556   \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3557 \def\bbl@localeinfo#1#2{%
3558   \bbl@ifunset{bbl@info@#2}{#1}%
3559   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3560    {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3561 \newcommand\localeinfo[1]{%
3562   \ifx*#1\@empty
3563     \bbl@afterelse\bbl@localeinfo{}%
3564   \else
3565     \bbl@localeinfo
3566     {\bbl@error{no-ini-info}}{}}}%
3567   {#1}%
3568 \fi}
3569 % \@namedef{bbl@info@name.locale}{lcname}
3570 \@namedef{bbl@info@tag.ini}{lini}

```

```

3571 \@namedef{bbl@info@name.english}{elname}
3572 \@namedef{bbl@info@name.opentype}{lname}
3573 \@namedef{bbl@info@tag.bcp47}{tbc}
3574 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3575 \@namedef{bbl@info@tag.opentype}{lotf}
3576 \@namedef{bbl@info@script.name}{esname}
3577 \@namedef{bbl@info@script.name.opentype}{sname}
3578 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3579 \@namedef{bbl@info@script.tag.opentype}{sotf}
3580 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3581 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3582 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3583 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3584 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3585 ({*More package options}) ≡
3586 \DeclareOption{ensureinfo=off}{}
3587 (/More package options)
3588 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3589 \newcommand\getlocaleproperty{%
3590   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3591 \def\bbl@getproperty@s#1#2#3{%
3592   \let#1\relax
3593   \def\bbl@elt##1##2##3{%
3594     \bbl@ifsamestring{##1/##2}{#3}%
3595     {\providecommand#1{##3}%
3596     \def\bbl@elt####1####2####3{}}}%
3597   {}}%
3598   \bbl@cs{inidata@#2}}%
3599 \def\bbl@getproperty@x#1#2#3{%
3600   \bbl@getproperty@s{#1}{#2}{#3}%
3601   \ifx#1\relax
3602     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3603   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3604 \let\bbl@ini@loaded\@empty
3605 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3606 \def\ShowLocaleProperties#1{%
3607   \typeout{}}%
3608   \typeout{*** Properties for language '#1' ***}%
3609   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3610   \@nameuse{bbl@inidata@#1}%
3611   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 47 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcptoname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3612 \newif\ifbbl@bcpallowed
3613 \bbl@bcpallowedfalse
3614 \def\bbl@autoload@options{@import}
3615 \def\bbl@provide@locale{%

```

```

3616 \ifx\babelprovide\undefined
3617   \bbl@error{base-on-the-fly}{\}\}%
3618 \fi
3619 \let\bbl@auxname\language
3620 \ifbbl@bcptoname
3621   \bbl@ifunset{bbl@bcp@map@\language}{\}% Move uplevel??
3622   {\edef\language{\@nameuse{bbl@bcp@map@\language}}\%
3623     \let\localname\language}%
3624 \fi
3625 \ifbbl@bcpallowed
3626   \expandafter\ifx\csname date\language\endcsname\relax
3627     \expandafter
3628     \bbl@bcplookup{\language}%
3629     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3630       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3631       \let\localname\language
3632       \expandafter\ifx\csname date\language\endcsname\relax
3633         \let\bbl@initoload\bbl@bcp
3634         \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3635         \let\bbl@initoload\relax
3636       \fi
3637       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localname}%
3638     \fi
3639   \fi
3640 \fi
3641 \expandafter\ifx\csname date\language\endcsname\relax
3642   \IfFileExists{babel-\language.tex}%
3643   {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3644   {\}%
3645 \fi}

```

$\TeX$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension `.<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to tag.bcp47.

```

3646 \providecommand\BCPdata{}
3647 \ifx\renewcommand\undefined\else
3648   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3649   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3650     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3651     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3652     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3653   \def\bbl@bcpdata@ii#1#2{%
3654     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3655     {\bbl@error{unknown-ini-field}{#1}{\}\}%
3656     {\bbl@ifunset{bbl@\csname bbl@info@#1.tag.bcp47\endcsname @#2}{\}%
3657       {\bbl@cs{\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}}
3658 \fi
3659 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3660 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3661 \newcommand\babeladjust[1]{%
3662   \bbl@forkv{#1}{%
3663     \bbl@ifunset{bbl@ADJ@##1@##2}%
3664     {\bbl@cs{ADJ@##1}{##2}}%
3665     {\bbl@cs{ADJ@##1@##2}}}%
3666 %
3667 \def\bbl@adjust@lua#1#2{%

```

```

3668 \ifvmode
3669 \ifnum\currentgrouplevel=\z@
3670 \directlua{ Babel.#2 }%
3671 \expandafter\expandafter\expandafter\@gobble
3672 \fi
3673 \fi
3674 {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3675 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3676 \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3677 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3678 \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3679 \@namedef{bbl@ADJ@bidi.text@on}{%
3680 \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3681 \@namedef{bbl@ADJ@bidi.text@off}{%
3682 \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3683 \@namedef{bbl@ADJ@bidi.math@on}{%
3684 \let\bbl@noamsmath\@empty}
3685 \@namedef{bbl@ADJ@bidi.math@off}{%
3686 \let\bbl@noamsmath\relax}
3687 %
3688 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3689 \bbl@adjust@lua{bidi}{digits_mapped=true}}
3690 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3691 \bbl@adjust@lua{bidi}{digits_mapped=false}}
3692 %
3693 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3694 \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3695 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3696 \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3697 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3698 \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3699 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3700 \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3701 \@namedef{bbl@ADJ@justify.arabic@on}{%
3702 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3703 \@namedef{bbl@ADJ@justify.arabic@off}{%
3704 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3705 %
3706 \def\bbl@adjust@layout#1{%
3707 \ifvmode
3708 #1%
3709 \expandafter\@gobble
3710 \fi
3711 {\bbl@error{layout-only-vertical}{}}}% Gobbled if everything went ok.
3712 \@namedef{bbl@ADJ@layout.tabular@on}{%
3713 \ifnum\bbl@tabular@mode=\tw@
3714 \bbl@adjust@layout{\let\@tabular\bbl@NL@@tabular}%
3715 \else
3716 \chardef\bbl@tabular@mode\@ne
3717 \fi}
3718 \@namedef{bbl@ADJ@layout.tabular@off}{%
3719 \ifnum\bbl@tabular@mode=\tw@
3720 \bbl@adjust@layout{\let\@tabular\bbl@OL@@tabular}%
3721 \else
3722 \chardef\bbl@tabular@mode\z@
3723 \fi}
3724 \@namedef{bbl@ADJ@layout.lists@on}{%
3725 \bbl@adjust@layout{\let\list\bbl@NL@list}}
3726 \@namedef{bbl@ADJ@layout.lists@off}{%
3727 \bbl@adjust@layout{\let\list\bbl@OL@list}}
3728 %
3729 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3730 \bbl@bcpallowedtrue}

```

```

3731 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3732   \bbl@bcppallowedfalse}
3733 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3734   \def\bbl@bcp@prefix{#1}}
3735 \def\bbl@bcp@prefix{bcp47-}
3736 \@namedef{bbl@ADJ@autoload.options}#1{%
3737   \def\bbl@autoload@options{#1}}
3738 \def\bbl@autoload@bcptoptions{import}
3739 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3740   \def\bbl@autoload@bcptoptions{#1}}
3741 \newif\ifbbl@bcptname
3742 %
3743 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3744   \bbl@bcptnametrue}
3745 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3746   \bbl@bcptnamefalse}
3747 %
3748 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3749   \directlua{ Babel.ignore_pre_char = function(node)
3750     return (node.lang == \the\csname \@nohyphenation\endcsname)
3751   end }}
3752 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3753   \directlua{ Babel.ignore_pre_char = function(node)
3754     return false
3755   end }}
3756 %
3757 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3758   \def\bbl@ignoreinterchar{%
3759     \ifnum\language=\@nohyphenation
3760       \expandafter\@gobble
3761     \else
3762       \expandafter\@firstofone
3763     \fi}}
3764 \@namedef{bbl@ADJ@interchar.disable@off}{%
3765   \let\bbl@ignoreinterchar\@firstofone}
3766 %
3767 \@namedef{bbl@ADJ@select.write@shift}{%
3768   \let\bbl@restorelastskip\relax
3769   \def\bbl@savelastskip{%
3770     \let\bbl@restorelastskip\relax
3771     \ifvmode
3772       \ifdim\lastskip=\z@
3773         \let\bbl@restorelastskip\nobreak
3774       \else
3775         \bbl@exp{%
3776           \def\\bbl@restorelastskip{%
3777             \skip@=\the\lastskip
3778             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3779         \fi
3780       \fi}}
3781 \@namedef{bbl@ADJ@select.write@keep}{%
3782   \let\bbl@restorelastskip\relax
3783   \let\bbl@savelastskip\relax}
3784 \@namedef{bbl@ADJ@select.write@omit}{%
3785   \AddBabelHook{babel-select}{beforestart}{%
3786     \expandafter\babel@aux\expandafter{\bbl@main@language}{}}%
3787   \let\bbl@restorelastskip\relax
3788   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3789 \@namedef{bbl@ADJ@select.encoding@off}{%
3790   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The *key* argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```
3791 ((*More package options)) ≡
3792 \DeclareOption{safe=none}{\let\bbl@opt@safe\empty}
3793 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3794 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3795 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3796 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3797 (/More package options)
```

\@newl@bel First we open a new group to keep the changed setting of `\protect local` and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```
3798 \bbl@trace{Cross referencing macros}
3799 \ifx\bbl@opt@safe\empty\else % i.e., if 'ref' and/or 'bib'
3800   \def\@newl@bel#1#2#3{%
3801     {\@safe@activestrue
3802       \bbl@ifunset{#1@#2}%
3803         \relax
3804         {\gdef\@multiplelabels{%
3805           \@latex@warning@no@line{There were multiply-defined labels}}%
3806           \@latex@warning@no@line{Label `#2' multiply defined}}%
3807       \global\@namedef{#1@#2}{#3}}}
```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```
3808 \CheckCommand*\@testdef[3]{%
3809   \def\reserved@a{#3}%
3810   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3811   \else
3812     \@tempswatrue
3813   \fi}
```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```
3814 \def\@testdef#1#2#3{%
3815   \@safe@activestrue
3816   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3817   \def\bbl@tempb{#3}%
3818   \@safe@activesfalse
3819   \ifx\bbl@tempa\relax
3820   \else
3821     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3822   \fi
3823   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3824   \ifx\bbl@tempa\bbl@tempb
3825   \else
3826     \@tempswatrue
3827   \fi}
3828 \fi
```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```
3829 \bbl@xin@{R}\bbl@opt@safe
3830 \ifin@
3831 \edef\bbl@tempc{\expandafter\string\csname ref_code\endcsname}%
3832 \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3833 {\expandafter\strip@prefix\meaning\ref}%
3834 \ifin@
3835 \bbl@redefine\@kernel@ref#1{%
3836   \@safe@activetrue\org@@kernel@ref{#1}\@safe@activfalse}
3837 \bbl@redefine\@kernel@pageref#1{%
3838   \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activfalse}
3839 \bbl@redefine\@kernel@sref#1{%
3840   \@safe@activetrue\org@@kernel@sref{#1}\@safe@activfalse}
3841 \bbl@redefine\@kernel@spageref#1{%
3842   \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activfalse}
3843 \else
3844 \bbl@redefinero bust\ref#1{%
3845   \@safe@activetrue\org@ref{#1}\@safe@activfalse}
3846 \bbl@redefinero bust\pageref#1{%
3847   \@safe@activetrue\org@pageref{#1}\@safe@activfalse}
3848 \fi
3849 \else
3850 \let\org@ref\ref
3851 \let\org@pageref\pageref
3852 \fi
```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```
3853 \bbl@xin@{B}\bbl@opt@safe
3854 \ifin@
3855 \bbl@redefine\@citex[#1]#2{%
3856   \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activfalse
3857   \org@@citex[#1]{\bbl@tempa}}
```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```
3858 \AtBeginDocument{%
3859   \@ifpackageloaded{natbib}{%
3860     \def\@citex[#1][#2]#3{%
3861       \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activfalse
3862       \org@@citex[#1][#2]{\bbl@tempa}}%
3863   }}
```

The package `cite` has a definition of `\@citex` where the shorthands need to be turned off in both arguments.

```
3864 \AtBeginDocument{%
3865   \@ifpackageloaded{cite}{%
3866     \def\@citex[#1]#2{%
3867       \@safe@activetrue\org@@citex[#1][#2]\@safe@activfalse}%
3868   }}
```

\nocite The macro \nocite which is used to instruct BiB_T_EX to extract uncited references from the database.

```
3869 \bbl@redefine\nocite#1{%
3870   \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}
```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \bibcite is needed we define \bibcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \bibcite. This new definition is then activated.

```
3871 \bbl@redefine\bibcite{%
3872   \bbl@cite@choice
3873   \bibcite}
```

\bbl@bibcite The macro \bbl@bibcite holds the definition of \bibcite needed when neither natbib nor cite is loaded.

```
3874 \def\bbl@bibcite#1#2{%
3875   \org@bibcite{#1}{\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3876 \def\bbl@cite@choice{%
3877   \global\let\bibcite\bbl@bibcite
3878   \@ifpackageloaded{natbib}{\global\let\bibcite\org@bibcite}{}%
3879   \@ifpackageloaded{cite}{\global\let\bibcite\org@bibcite}{}%
3880   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3881 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal L^AT_EX macros called by \bibitem that write the citation label on the aux file.

```
3882 \bbl@redefine\@bibitem#1{%
3883   \@safe@activetrue\org@bibitem{#1}\@safe@activesfalse}
3884 \else
3885   \let\org@nocite\nocite
3886   \let\org@@citex\@citex
3887   \let\org@bibcite\bibcite
3888   \let\org@@bibitem\@bibitem
3889 \fi
```

5.2. Layout

```
3890 \newcommand\BabelPatchSection[1]{%
3891   \@ifundefined{#1}{}{%
3892     \bbl@exp{\let\<bbl@ss@#1>\<#1>}%
3893     \@namedef{#1}{%
3894       \@ifstar{\bbl@presec@#1}{%
3895         {\@dblarg{\bbl@presec@x{#1}}}}
3896 \def\bbl@presec@x#1[#2]#3{%
3897   \bbl@exp{%
3898     \\select@language@x{\bbl@main@language}%
3899     \\bbl@cs{sspre@#1}%
3900     \\bbl@cs{ss@#1}%
3901     [\\foreignlanguage{\language@name}{\unexpanded{#2}}}%
3902     {\\foreignlanguage{\language@name}{\unexpanded{#3}}\babelsublr{}}}%
3903     \\select@language@x{\language@name}}}
```

```

3904 \def\bbl@presec@s#1#2{%
3905   \bbl@exp{%
3906     \select@language{x{\bbl@main@language}%
3907     \bbl@cs{sspre@#1}%
3908     \bbl@cs{ss@#1}*\%
3909     {\foreignlanguage{\language}{\unexpanded{#2}}\babelsublr{}}}%
3910     \select@language{x{\language}}}%
3911 %
3912 \IfBabelLayout{sectioning}%
3913   {\BabelPatchSection{part}%
3914   \BabelPatchSection{chapter}%
3915   \BabelPatchSection{section}%
3916   \BabelPatchSection{subsection}%
3917   \BabelPatchSection{subsubsection}%
3918   \BabelPatchSection{paragraph}%
3919   \BabelPatchSection{subparagraph}%
3920   \def\babel@toc#1{%
3921     \select@language{x{\bbl@main@language}}}%
3922 \IfBabelLayout{captions}%
3923   {\BabelPatchSection{caption}}}%

```

\BabelFootnote Footnotes.

```

3924 \bbl@trace{Footnotes}
3925 \def\bbl@footnote#1#2#3{%
3926   \ifnextchar[%
3927     {\bbl@footnote@o{#1}{#2}{#3}}%
3928     {\bbl@footnote@x{#1}{#2}{#3}}}
3929 \long\def\bbl@footnote@x#1#2#3#4{%
3930   \bgroup
3931     \select@language{x{\bbl@main@language}%
3932     \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3933   \egroup}
3934 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3935   \bgroup
3936     \select@language{x{\bbl@main@language}%
3937     \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3938   \egroup}
3939 \def\bbl@footnotetext#1#2#3{%
3940   \ifnextchar[%
3941     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3942     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3943 \long\def\bbl@footnotetext@x#1#2#3#4{%
3944   \bgroup
3945     \select@language{x{\bbl@main@language}%
3946     \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3947   \egroup}
3948 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3949   \bgroup
3950     \select@language{x{\bbl@main@language}%
3951     \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3952   \egroup}
3953 \def\BabelFootnote#1#2#3#4{%
3954   \ifx\bbl@fn@footnote\undefined
3955     \let\bbl@fn@footnote\footnote
3956   \fi
3957   \ifx\bbl@fn@footnotetext\undefined
3958     \let\bbl@fn@footnotetext\footnotetext
3959   \fi
3960   \bbl@iifblank{#2}%
3961     {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3962     \namedef{\bbl@stripslash#1text}%
3963     {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3964   {\def#1{\bbl@exp{\bbl@footnote{\foreignlanguage{#2}}}{#3}{#4}}}%

```

```

3965 \namedef{\bbl@stripslash#1text}%
3966 {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}{#3}{#4}}}%
3967 \IfBabelLayout{footnotes}%
3968 {\let\bbl@OL@footnote\footnote
3969 \BabelFootnote\footnote\languagesname{}}%
3970 \BabelFootnote\localfootnote\languagesname{}}%
3971 \BabelFootnote\mainfootnote{}}%
3972 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3973 \bbl@trace{Marks}
3974 \IfBabelLayout{sectioning}
3975 {\ifx\bbl@opt@headfoot\@nnil
3976 \g@addto@macro\resetactivechars{%
3977 \set@typeset@protect
3978 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3979 \let\protect\noexpand
3980 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3981 \edef\thepage{%
3982 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3983 \fi}%
3984 \fi}
3985 {\ifbbl@single\else
3986 \bbl@ifunset{markright }{\bbl@redefine\bbl@redefineroobust
3987 \markright#1{%
3988 \bbl@ifblank{#1}%
3989 {\org@markright{}}}%
3990 {\toks@{#1}%
3991 \bbl@exp{%
3992 \org@markright{\protect\foreignlanguage{\languagesname}%
3993 \protect\bbl@restore@actives\the\toks@}}}%

```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, ~~La~~TeX stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3994 \ifx\@mkboth\markboth
3995 \def\bbl@tempc{\let\@mkboth\markboth}%
3996 \else
3997 \def\bbl@tempc{}%
3998 \fi
3999 \bbl@ifunset{markboth }{\bbl@redefine\bbl@redefineroobust
4000 \markboth#1#2{%
4001 \protected@edef\bbl@tempb##1{%
4002 \protect\foreignlanguage
4003 {\languagesname}{\protect\bbl@restore@actives##1}}%
4004 \bbl@ifblank{#1}%
4005 {\toks@{}}%
4006 {\toks@\expandafter{\bbl@tempb{#1}}}%
4007 \bbl@ifblank{#2}%
4008 {\@temptokena{}}%
4009 {\@temptokena\expandafter{\bbl@tempb{#2}}}%

```

```

4010      \bbl@exp{\org@markboth{\the\toks@}{\the\@temptokena}}}%
4011      \bbl@tempc
4012      \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}{
%      {code for odd pages}
%      {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

4013 \bbl@trace{Preventing clashes with other packages}
4014 \ifx\org@ref\undefined\else
4015   \bbl@xin@{R}\bbl@opt@safe
4016   \ifin@
4017     \AtBeginDocument{%
4018       \@ifpackageloaded{ifthen}{%
4019         \bbl@redefine@long\ifthenelse#1#2#3{%
4020           \let\bbl@temp@pref\pageref
4021           \let\pageref\org@pageref
4022           \let\bbl@temp@ref\ref
4023           \let\ref\org@ref
4024           \@safe@activestrue
4025           \org@ifthenelse{#1}%
4026             {\let\pageref\bbl@temp@pref
4027              \let\ref\bbl@temp@ref
4028              \@safe@activesfalse
4029              #2}%
4030             {\let\pageref\bbl@temp@pref
4031              \let\ref\bbl@temp@ref
4032              \@safe@activesfalse
4033              #3}%
4034           }%
4035         }{}%
4036       }
4037 \fi

```

5.4.2. varioref

\@@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

4038 \AtBeginDocument{%
4039   \@ifpackageloaded{varioref}{%
4040     \bbl@redefine\@@vpageref#1[#2]#3{%
4041       \@safe@activestrue
4042       \org@@@vpageref{#1}[#2]{#3}%
4043       \@safe@activesfalse}%

```

```

4044 \bbl@redefine\hrefpagenum#1#2{%
4045 \@safe@activestrue
4046 \org\hrefpagenum{#1}{#2}%
4047 \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

4048 \expandafter\def\csname Ref \endcsname#1{%
4049 \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4050 }{}%
4051 }
4052 \fi

```

5.4.3. `hhline`

`\hhline` Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the “`’`” character which is made active by the french support in `babel`. Therefore we need to *reload* the package when the “`’`” is an active character. Note that this happens *after* the category code of the `@`-sign has been changed to other, so we need to temporarily change it to letter again.

```

4053 \AtEndOfPackage{%
4054 \AtBeginDocument{%
4055 \ifpackageloaded{hhline}%
4056 {\expandafter\ifx\csname normal@char\string\endcsname\relax
4057 \else
4058 \makeatletter
4059 \def\@currname{hhline}\input{hhline.sty}\makeatother
4060 \fi}%
4061 {}}}

```

`\substitutefontfamily` *Deprecated.* It creates an `fd` file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by $\TeX$ (`\DeclareFontFamilySubstitution`).

```

4062 \def\substitutefontfamily#1#2#3{%
4063 \lowercase{\immediate\openout15=#1#2.fd\relax}%
4064 \immediate\write15{%
4065 \string\ProvidesFile{#1#2.fd}%
4066 [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4067 \space generated font description file]^J
4068 \string\DeclareFontFamily{#1}{#2}{}}^J
4069 \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
4070 \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
4071 \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
4072 \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
4073 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
4074 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
4075 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
4076 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
4077 }%
4078 \closeout15
4079 }
4080 \onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of `\TeX` and `\LaTeX` for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```
4081 \bbl@trace{Encoding and fonts}
4082 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4083 \newcommand\BabelNonText{TS1,T3,TS3}
4084 \let\org@TeX\TeX
4085 \let\org@LaTeX\LaTeX
4086 \let\ensureascii\@firstofone
4087 \let\asciienencoding\@empty
4088 \AtBeginDocument{%
4089   \def\@elt#1{,#1,}%
4090   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4091   \let\@elt\relax
4092   \let\bbl@tempb\@empty
4093   \def\bbl@tempc{OT1}%
4094   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4095     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4096   \bbl@foreach\bbl@tempa{%
4097     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4098     \ifin@
4099       \def\bbl@tempb{#1}% Store last non-ascii
4100     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4101       \ifin@
4102         \def\bbl@tempc{#1}% Store last ascii
4103       \fi
4104     \fi}%
4105   \ifx\bbl@tempb\@empty\else
4106     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4107     \ifin@
4108       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4109     \fi
4110     \let\asciienencoding\bbl@tempc
4111     \renewcommand\ensureascii[1]{%
4112       {\fontencoding{\asciienencoding}\selectfont#1}}%
4113     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4114     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4115   \fi}
```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than 'latin' (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```
4116 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}
```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\@ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```
4117 \AtBeginDocument{%
4118   \@ifpackageloaded{fontspec}%
4119   {\xdef\latinencoding{%
4120     \ifx\UTFencname\undefined
4121       EU\ifcase\bbl@engine\or2\or1\fi
4122     \else
4123       \UTFencname
4124     \fi}}%
4125   {\gdef\latinencoding{OT1}%
4126     \ifx\cf@encoding\bbl@t@one
4127       \xdef\latinencoding{\bbl@t@one}%
4128     \else
4129       \def\@elt#1{,#1,}%
4130       \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4131     \fi}
```

```

4131      \let\@elt\relax
4132      \bbl@xin@{,Tl,}\bbl@tempa
4133      \ifin@
4134      \xdef\latinencoding{\bbl@t@one}%
4135      \fi
4136      \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4137 \DeclareRobustCommand{\latintext}{%
4138   \fontencoding{\latinencoding}\selectfont
4139   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4140 \ifx\@undefined\DeclareTextFontCommand
4141   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4142 \else
4143   \DeclareTextFontCommand{\textlatin}{\latintext}
4144 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\text{\LaTeX}$ 2021-06-01, there is a hook for this purpose.

```

4145 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at ARABI (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdfTeX` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\text{\TeX}$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\text{\TeX}$ -ja` shows, vertical typesetting is possible, too.

```

4146 \bbl@trace{Loading basic (internal) bidi support}
4147 \ifodd\bbl@engine
4148 \else % Any xe+lua bidi
4149   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4150     \bbl@error{bidi-only-lua}{\}\}\}\}%
4151     \let\bbl@beforeforeign\leavevmode
4152     \AtEndOfPackage{%
4153       \EnableBabelHook{babel-bidi}%
4154       \bbl@xebidipar}
4155   \fi\fi
4156   \def\bbl@loadxebidi#1{%
4157     \ifx\RTLfootnotetext\@undefined
4158       \AtEndOfPackage{%
4159         \EnableBabelHook{babel-bidi}%

```

```

4160 \ifx\fontspec\undefined
4161 \usepackage{fontspec}% bidi needs fontspec
4162 \fi
4163 \usepackage#1{bidi}%
4164 \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4165 \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4166 \ifnum\@nameuse{bbl@wdir\language}\tw@ % 'AL' bidi
4167 \bbl@digitsdotdash % So ignore in 'R' bidi
4168 \fi}%
4169 \fi}
4170 \ifnum\bbl@bidimode>200 % Any xe bidi=
4171 \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4172 \bbl@tentative{bidi=bidi}
4173 \bbl@loadxebidi{}
4174 \or
4175 \bbl@loadxebidi{[rldocument]}
4176 \or
4177 \bbl@loadxebidi{}
4178 \fi
4179 \fi
4180 \fi
4181 \ifnum\bbl@bidimode=\@ne % bidi=default
4182 \let\bbl@beforeforeign\leavevmode
4183 \ifodd\bbl@engine % lua
4184 \newattribute\bbl@attr@dir
4185 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4186 \bbl@exp{\output{\bodydir\pagedir\the\output}}
4187 \fi
4188 \AtEndOfPackage{%
4189 \EnableBabelHook{babel-bidi}% pdf/lua/xe
4190 \ifodd\bbl@engine\else % pdf/xe
4191 \bbl@xebidipar
4192 \fi}
4193 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4194 \bbl@trace{Macros to switch the text direction}
4195 \def\bbl@alscripts{%
4196 ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4197 \def\bbl@rscripts{%
4198 Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4199 Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4200 Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4201 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4202 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4203 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4204 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4205 Meroitic,N'Ko,Orkhon,Todhri}
4206 %
4207 \def\bbl@provide@dirs#1{%
4208 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4209 \ifin@
4210 \global\bbl@csarg\chardef{wdir@#1}\@ne
4211 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4212 \ifin@
4213 \global\bbl@csarg\chardef{wdir@#1}\tw@
4214 \fi
4215 \else
4216 \global\bbl@csarg\chardef{wdir@#1}\z@
4217 \fi
4218 \ifodd\bbl@engine

```

```

4219 \bbl@csarg\ifcase{wdir@#1}%
4220 \directlua{ Babel.locale_props[\the\localeid].textdir = 'l' }%
4221 \or
4222 \directlua{ Babel.locale_props[\the\localeid].textdir = 'r' }%
4223 \or
4224 \directlua{ Babel.locale_props[\the\localeid].textdir = 'al' }%
4225 \fi
4226 \fi}
4227 %
4228 \def\bbl@switchdir{%
4229 \bbl@ifunset{\bbl@sys{\languagename}}{\bbl@provide@sys{\languagename}}{}%
4230 \bbl@ifunset{\bbl@wdir{\languagename}}{\bbl@provide@dirs{\languagename}}{}%
4231 \bbl@exp{\bbl@setdirs\bbl@cl{wdir}}}%
4232 \def\bbl@setdirs#1{%
4233 \ifcase\bbl@select@type
4234 \bbl@bodydir{#1}%
4235 \bbl@pardir{#1}% <- Must precede \bbl@textdir
4236 \fi
4237 \bbl@textdir{#1}}
4238 \ifnum\bbl@bidimode>\z@
4239 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4240 \DisableBabelHook{babel-bidi}
4241 \fi

```

Now the engine-dependent macros.

```

4242 \ifodd\bbl@engine % luatex=1
4243 \else % pdftex=0, xetex=2
4244 \newcount\bbl@dirlevel
4245 \chardef\bbl@thetextdir\z@
4246 \chardef\bbl@thepardir\z@
4247 \def\bbl@textdir#1{%
4248 \ifcase#1\relax
4249 \chardef\bbl@thetextdir\z@
4250 \@nameuse{setlatin}%
4251 \bbl@textdir@i\beginL\endL
4252 \else
4253 \chardef\bbl@thetextdir@ne
4254 \@nameuse{setnonlatin}%
4255 \bbl@textdir@i\beginR\endR
4256 \fi}
4257 \def\bbl@textdir@i#1#2{%
4258 \ifhmode
4259 \ifnum\currentgrouplevel>\z@
4260 \ifnum\currentgrouplevel=\bbl@dirlevel
4261 \bbl@error{multiple-bidi}{}{}%
4262 \bgroup\aftergroup#2\aftergroup\egroup
4263 \else
4264 \ifcase\currentgrouptype\or % 0 bottom
4265 \aftergroup#2% 1 simple {}
4266 \or
4267 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4268 \or
4269 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4270 \or\or\or % vbox vtop align
4271 \or
4272 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4273 \or\or\or\or\or\or % output math disc insert vcent mathchoice
4274 \or
4275 \aftergroup#2% 14 \begingroup
4276 \else
4277 \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4278 \fi
4279 \fi

```

```

4280      \bbl@dirlevel\currentgrouplevel
4281      \fi
4282      #1%
4283      \fi}
4284      \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4285      \let\bbl@bodydir\@gobble
4286      \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4287      \def\bbl@xebidipar{%
4288        \let\bbl@xebidipar\relax
4289        \TeXeTstate\@ne
4290        \def\bbl@xeeverypar{%
4291          \ifcase\bbl@thepardir
4292            \ifcase\bbl@thetextdir\else\beginR\fi
4293            \else
4294              {\setbox\z@\lastbox\beginR\box\z@}%
4295              \fi}%
4296          \AddToHook{para/begin}{\bbl@xeeverypar}}
4297      \ifnum\bbl@bidimode>200 % Any xe bidi=
4298        \let\bbl@textdir\i\@gobbletwo
4299        \let\bbl@xebidipar\@empty
4300        \AddBabelHook{bidi}{foreign}{%
4301          \ifcase\bbl@thetextdir
4302            \BabelWrapText{\LR{##1}}%
4303            \else
4304              \BabelWrapText{\RL{##1}}%
4305          \fi}
4306        \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4307      \fi
4308      \fi

```

A tool for weak L (mainly digits). We also disable warnings with `hyperref`. There is a variant for `bidi=unibidi`, but using formatting characters is discouraged by Unicode UAX9, so it must be revised.

```

4309      \ifnum\bbl@bidimode>300
4310        \DeclareRobustCommand\babelsublr[1]{\leavevmode{^^^200f#1}}
4311      \else
4312        \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4313      \fi
4314      \AtBeginDocument{%
4315        \ifx\pdfstringdefDisableCommands\@undefined\else
4316          \ifx\pdfstringdefDisableCommands\relax\else
4317            \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4318          \fi
4319        \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4320      \bbl@trace{Local Language Configuration}
4321      \ifx\loadlocalcfg\@undefined
4322        \ifpackagewith{babel}{noconfigs}%
4323          {\let\loadlocalcfg\@gobble}%
4324          {\def\loadlocalcfg#1{%
4325            \InputIfFileExists{#1.cfg}%

```

```

4326      {\typeout{*****^J%
4327              * Local config file #1.cfg used^^J%
4328              *}}%
4329      \@empty}}
4330 \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```

4331 \bbl@trace{Language options}
4332 \def\BabelDefinitionFile#1#2#3{
4333 \let\bbl@afterlang\relax
4334 \let\BabelModifiers\relax
4335 \let\bbl@loaded\@empty
4336 \def\bbl@load@language#1{%
4337   \InputIfFileExists{#1.ldf}%
4338   {\edef\bbl@loaded{\CurrentOption
4339     \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4340     \expandafter\let\expandafter\bbl@afterlang
4341     \csname\CurrentOption. ldf-h@@k\endcsname
4342     \expandafter\let\expandafter\BabelModifiers
4343     \csname bbl@mod@\CurrentOption\endcsname
4344     \bbl@exp{\AtBeginDocument{%
4345       \bbl@usehooks@lang{\CurrentOption}{\begindocument}{\CurrentOption}}}%
4346     {\bbl@error{unknown-package-option}}{}}}%

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language as been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetada we also force it with `\foreignlanguage` (this is also done in bidi texts).

```

4347 \ifx\GetDocumentProperties\undefined\else
4348   \let\bbl@beforeforeign\leavevmode
4349   \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4350   \let\bbl@collect@other\@empty
4351   \bbl@foreach\bbl@tempa{%
4352     \edef\bbl@metalang{#1}%
4353     \ifx\bbl@metalang\@empty\else
4354       \begingroup
4355         \expandafter
4356         \bbl@bcplookup{\bbl@metalang}%
4357         \ifx\bbl@bcp\relax
4358           \ifx\bbl@opt@main\@nnil
4359             \bbl@error{no-locale-for-meta}{\bbl@metalang}}}%
4360       \fi
4361     \else
4362       \bbl@read@ini{\bbl@bcp}\m@ne
4363       \xdef\bbl@collect@other{\bbl@collect@other,\language#1}%
4364       \GenericInfo{(babel) \spaces\@spaces\@spaces
4365         Passing '\language#1' to babel\@gobble}%
4366     \fi
4367   \endgroup
4368 \fi}
4369 \ifx\bbl@collect@other\@empty\else
4370   \xdef\bbl@language@opts{\bbl@collect@other,\bbl@language@opts}%
4371 \fi

```

```

4372 \edef\bbl@metalang{\GetDocumentProperties{document/language}}%
4373 \ifx\bbl@metalang\empty\else
4374   \begingroup
4375     \expandafter
4376     \bbl@bcpllookup{\bbl@metalang}%
4377     \ifx\bbl@bcplrelax
4378       \ifx\bbl@opt@main\@nnil
4379         \bbl@error{no-locale-for-meta}{\bbl@metalang}{}{}%
4380       \fi
4381     \else
4382       \bbl@read@ini{\bbl@bcpl}\m@ne
4383       \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4384       \ifx\bbl@opt@main\@nnil
4385         \global\let\bbl@opt@main\language
4386       \fi
4387       \GenericInfo{}{(babel) \@spaces\@spaces\@spaces
4388         Passing '\language' to babel\@gobble}%
4389     \fi
4390   \endgroup
4391 \fi
4392 \fi
4393 \ifx\bbl@opt@config\@nnil
4394   \@ifpackagewith{babel}{noconfigs}{}%
4395   {\InputIfFileExists{bblopts.cfg}%
4396     {\bbl@info{Configuration files are deprecated, as\\%
4397       they can break document portability.\\%
4398       Reported}%
4399     \typeout{*****^J%
4400       * Local config file bblopts.cfg used^^J%
4401       *}%
4402     }{}%
4403   \else
4404     \InputIfFileExists{\bbl@opt@config.cfg}%
4405     {\bbl@info{Configuration files are deprecated, as\\%
4406       they can break document portability.\\%
4407       Reported}%
4408     \typeout{*****^J%
4409       * Local config file \bbl@opt@config.cfg used^^J%
4410       *}%
4411     {\bbl@error{config-not-found}{}{}{}%
4412   \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may content a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `@import`).

```

4413 \def\BabelBeforeIni#1#2{%
4414   \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4415   \let\bbl@tempb\empty
4416   #2%
4417   \edef\bbl@toload{%
4418     \ifx\bbl@toload\empty\else\bbl@toload,\fi
4419     \bbl@toload@last}%
4420   \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}

```

```

4421 \def\BabelDefinitionFile#1#2#3{%
4422   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4423   \namedef\bbl@preldf@{\CurrentOption}{#3}%
4424   \endinput}%

```

For efficiency, first preprocess the class options to remove those with =, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\text{\LaTeX}$ 3 layer.

```

4425 \def\bbl@tempf{,}
4426 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4427   \in@{=}{#1}%
4428   \ifin@ \else
4429     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4430   \fi}

```

Store the class/package options in a list. If there is an explicit main, it's placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4431 \let\bbl@toload\@empty
4432 \let\bbl@toload@last\@empty
4433 \let\bbl@unkopt\@gobble %% <- Ugly
4434 \edef\bbl@tempc{%
4435   \bbl@tempf,@,\bbl@language@opts
4436   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4437 \let\BabelLocalesTentative\bbl@tempc
4438 %
4439 \bbl@foreach\bbl@tempc{%
4440   \in@{@@}{#1}% <- Ugly
4441   \ifin@
4442     \def\bbl@unkopt##1{%
4443       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}{}}%
4444   \else
4445     \def\CurrentOption{#1}%
4446     \bbl@xin@{//#1//}{\bbl@toload@last}% Collapse consecutive
4447     \ifin@ \else
4448       \lowercase{\InputIfFileExists{babel-#1.tex}}{ }{%
4449         \IfFileExists{#1.ldf}%
4450         {\edef\bbl@toload{%
4451           \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4452           \bbl@toload@last}%
4453           \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4454         {\bbl@unkopt{#1}}}%
4455     \fi
4456   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4457 \ifx\bbl@opt@main\@nnil
4458   \ifx\bbl@toload@last\@empty
4459     \def\bbl@toload@last{0/0//nil//und-x-nil-nil}
4460     \bbl@info{%
4461       You haven't specified a language as a class or package\\%
4462       option. I'll load 'nil'. Reported}
4463   \fi
4464 \else
4465   \let\bbl@tempc\@empty
4466   \bbl@foreach\bbl@toload{%
4467     \bbl@xin@{//#1//}{\bbl@opt@main//}{#1}%
4468     \ifin@ \else
4469       \bbl@add@list\bbl@tempc{#1}%
4470     \fi}

```

```

4471 \let\bbl@toload\bbl@tempc
4472 \fi
4473 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4474 \def\AfterBabelLanguage#1{%
4475 \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang}{}}
4476 \NewHook{babel/presets}
4477 \UseHook{babel/presets}
4478 %
4479 \let\bbl@tempb\@empty
4480 \def\bbl@tempc#1/#2//#3//#4/#5\@@{%
4481 \count@\z@
4482 \ifnum#2=\@m % if no \BabelDefinitionFile
4483 \ifnum#1=\z@ % not main. -- % if provide+=!, provide*=!
4484 \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@@
4485 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4486 \fi
4487 \else % 10 = main -- % if provide+=!, provide*=!
4488 \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@@
4489 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4490 \fi
4491 \fi
4492 \else
4493 \ifnum#1=\z@ % not main
4494 \ifnum\bbl@iniflag>\@ne\else % if ø, provide
4495 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4496 \fi
4497 \else % 10 = main
4498 \ifodd\bbl@iniflag\else % if provide+, provide*
4499 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4500 \fi
4501 \fi
4502 \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4503 \fi}

```

Based on the value of \count@, do the actual loading. If ‘ldf’, we load the basic info from the ‘ini’ file before.

```

4504 \def\bbl@tempd#1#2#3#4#5{%
4505 \DeclareOption{#3}{}%
4506 \ifcase\count@
4507 \bbl@exp{\bbl@add\bbl@tempb{%
4508 \global\let\bbl@afterload\@empty
4509 \@@nameuse\bbl@preini@#3}%
4510 \bbl@ldfinit
4511 \def\CurrentOption{#3}%
4512 \bbl@babelprovide[import=#4,\ifnum#1=\z@\else\bbl@opt@provide,main\fi]{#3}%
4513 \bbl@afterldf
4514 \bbl@afterload}}%
4515 \else
4516 \bbl@add\bbl@tempb{%
4517 \global\let\bbl@afterload\@empty
4518 \def\CurrentOption{#3}%
4519 \let\localename\CurrentOption
4520 \let\languagename\localename
4521 \def\BabelIniTag{#4}%
4522 \@@nameuse\bbl@preldf@#3}%
4523 \begingroup
4524 \bbl@id@assign
4525 \bbl@read@ini{\BabelIniTag}0%
4526 \endgroup

```

```

4527 \bbl@load@language{#5}%
4528 \bbl@afterload}%
4529 \fi}
4530 %
4531 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4532 \bbl@tempb
4533 \DeclareOption*{}
4534 \ProcessOptions
4535 %
4536 \bbl@exp{%
4537 \\\AtBeginDocument{\bbl@usehooks@lang/{\begin{document}}{}}}%
4538 \def\AfterBabelLanguage{\bbl@error{late-after-babel}}{}{}
4539 \AddToHook{begin{document}/end}{%
4540 \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4541 (/package)

```

6. Tagging

Preliminary and experimental. Only `\foreignlanguage`, for the moment.

```

4542 (*tagging)
4543 \let\bbl@ifmetadata\IfDocumentMetadataT
4544 \ExplSyntaxOn
4545 \NewTaggingSocket{babel/foreign/begin}{0}
4546 \NewTaggingSocket{babel/foreign/end}{0}
4547 \NewTaggingSocketPlug{babel/foreign/begin}{default}{
4548 \tag_mc_end_push:
4549 \tagstructbegin{tag=Span, lang=\BCPdata{tag}}
4550 \tagmcbegin {} }
4551 \NewTaggingSocketPlug{babel/foreign/end}{default}{
4552 \tagmcend
4553 \tagstructend
4554 \tag_mc_begin_pop:n {}}
4555 \AssignTaggingSocketPlug{babel/foreign/begin}{default}
4556 \AssignTaggingSocketPlug{babel/foreign/end}{default}
4557 \ExplSyntaxOff
4558 (/tagging)

```

7. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4559 (*kernel)
4560 \let\bbl@onlyswitch\@empty
4561 \input babel.def
4562 \let\bbl@onlyswitch\@undefined
4563 (/kernel)

```

8. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make

sure some catcodes have the right value, although those for \, `, ^M, % and = are reset before loading the file.

```

4564 (*errors)
4565 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4566 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4567 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4568 \catcode`\@=11 \catcode`\^=7
4569 %
4570 \ifx\MessageBreak\undefined
4571 \gdef\bbl@error@i#1#2{%
4572     \beginingroup
4573     \newlinechar=`^^J
4574     \def\{^^J(babel) }%
4575     \errhelp{#2}\errmessage{\{#1}%
4576     \endgroup}
4577 \else
4578 \gdef\bbl@error@i#1#2{%
4579     \beginingroup
4580     \def\{\MessageBreak}%
4581     \PackageError{babel}{#1}{#2}%
4582     \endgroup}
4583 \fi
4584 \def\bbl@errmessage#1#2#3{%
4585     \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4586         \bbl@error@i{#2}{#3}}
4587 % Implicit #2#3#4:
4588 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4589 %
4590 \bbl@errmessage{not-yet-available}
4591     {Not yet available}%
4592     {Find an armchair, sit down and wait}
4593 \bbl@errmessage{bad-package-option}%
4594     {Bad option '#1=#2'. Either you have misspelled the\\%
4595     key or there is a previous setting of '#1'. Valid\\%
4596     keys are, among others, 'shorthands', 'main', 'bidi',\\%
4597     'strings', 'config', 'headfoot', 'safe', 'math'.}%
4598     {See the manual for further details.}
4599 \bbl@errmessage{base-on-the-fly}
4600     {For a language to be defined on the fly 'base'\\%
4601     is not enough, and the whole package must be\\%
4602     loaded. Either delete the 'base' option or\\%
4603     request the languages explicitly}%
4604     {See the manual for further details.}
4605 \bbl@errmessage{undefined-language}
4606     {You haven't defined the language '#1' yet.\\%
4607     Perhaps you misspelled it or your installation\\%
4608     is not complete}%
4609     {Your command will be ignored, type <return> to proceed}
4610 \bbl@errmessage{invalid-ini-name}
4611     {'#1' not valid with the 'ini' mechanism.\\%
4612     I think you want '#2' instead. You may continue,\\%
4613     but you should fix the name. See the babel manual\\%
4614     for the available locales with 'provide'}%
4615     {See the manual for further details.}
4616 \bbl@errmessage{shorthand-is-off}
4617     {I can't declare a shorthand turned off (\string#2)}
4618     {Sorry, but you can't use shorthands which have been\\%
4619     turned off in the package options}
4620 \bbl@errmessage{not-a-shorthand}
4621     {The character '\string #1' should be made a shorthand character;\\%
4622     add the command \string\usesshorthands\string{#1\string} to
4623     the preamble.\\%
4624     I will ignore your instruction}%

```

```

4625 {You may proceed, but expect unexpected results}
4626 \bbl@errmessage{not-a-shorthand-b}
4627 {I can't switch '\string#2' on or off--not a shorthand\\%
4628 This character is not a shorthand. Maybe you made\\%
4629 a typing mistake?}%
4630 {I will ignore your instruction.}
4631 \bbl@errmessage{unknown-attribute}
4632 {The attribute #2 is unknown for language #1.}%
4633 {Your command will be ignored, type <return> to proceed}
4634 \bbl@errmessage{missing-group}
4635 {Missing group for string \string#1}%
4636 {You must assign strings to some category, typically\\%
4637 captions or extras, but you set none}
4638 \bbl@errmessage{only-lua-xe}
4639 {This macro is available only in LuaLaTeX and XeLaTeX.}%
4640 {Consider switching to these engines.}
4641 \bbl@errmessage{only-lua}
4642 {This macro is available only in LuaLaTeX}%
4643 {Consider switching to that engine.}
4644 \bbl@errmessage{unknown-provide-key}
4645 {Unknown key '#1' in \string\babelprovide}%
4646 {See the manual for valid keys}%
4647 \bbl@errmessage{unknown-mapfont}
4648 {Option '\bbl@KVP@mapfont' unknown for\\%
4649 mapfont. Use 'direction'}%
4650 {See the manual for details.}
4651 \bbl@errmessage{no-ini-file}
4652 {There is no ini file for the requested language\\%
4653 (#1: \language). Perhaps you misspelled it or your\\%
4654 installation is not complete}%
4655 {Fix the name or reinstall babel.}
4656 \bbl@errmessage{digits-is-reserved}
4657 {The counter name 'digits' is reserved for mapping\\%
4658 decimal digits}%
4659 {Use another name.}
4660 \bbl@errmessage{limit-two-digits}
4661 {Currently two-digit years are restricted to the\\5
4662 range 0-9999}%
4663 {There is little you can do. Sorry.}
4664 \bbl@errmessage{counter-too-large}
4665 {Counter too large (#1)}%
4666 {Currently this is the limit.}
4667 \bbl@errmessage{no-ini-info}
4668 {I've found no info for the current locale.\\%
4669 The corresponding ini file has not been loaded\\%
4670 Perhaps it doesn't exist}%
4671 {See the manual for details.}
4672 \bbl@errmessage{unknown-ini-field}
4673 {Unknown field '#1' in \string\BCPdata.\\%
4674 Perhaps you misspelled it}%
4675 {See the manual for details.}
4676 \bbl@errmessage{unknown-locale-key}
4677 {Unknown key for locale '#2':\\%
4678 #3\\%
4679 \string#1 will be set to \string\relax}%
4680 {Perhaps you misspelled it.}%
4681 \bbl@errmessage{adjust-only-vertical}
4682 {Currently, #1 related features can be adjusted only\\%
4683 in the main vertical list}%
4684 {Maybe things change in the future, but this is what it is.}
4685 \bbl@errmessage{layout-only-vertical}
4686 {Currently, layout related features can be adjusted only\\%
4687 in vertical mode}%

```

```

4688 {Maybe things change in the future, but this is what it is.}
4689 \bbl@errmessage{bidi-only-lua}
4690 {The bidi method 'basic' is available only in\\%
4691   luatex. I'll continue with 'bidi=default', so\\%
4692   expect wrong results.\\%
4693   Suggested actions:\\%
4694   * If possible, switch to luatex, as xetex is not\\%
4695     recommend anymore.\\
4696   * If you can't, try 'bidi=bidi' with xetex.\\%
4697   * With pdftex, only 'bidi=default' is available.}%
4698 {See the manual for further details.}
4699 \bbl@errmessage{multiple-bidi}
4700 {Multiple bidi settings inside a group\\%
4701   I'll insert a new group, but expect wrong results.\\%
4702   Suggested action:\\%
4703   * Add a new group where appropriate.}
4704 {See the manual for further details.}
4705 \bbl@errmessage{unknown-package-option}
4706 {Unknown option '\CurrentOption'.\\%
4707   Suggested actions:\\%
4708   * Make sure you haven't misspelled it\\%
4709   * Check in the babel manual that it's supported\\%
4710   * If supported and it's a language, you may\\%
4711     \space\space need in some distributions a separate\\%
4712     \space\space installation\\%
4713   * If installed, check there isn't an old\\%
4714     \space\space version of the required files in your system\\%
4715   * If it's an unsupported language, create it with\\%
4716     \space\space \string\babelprovide\space (see the manual)}
4717 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4718   activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4719   headfoot=, strings=, config=, hyphenmap=, or a language name.}
4720 \bbl@errmessage{config-not-found}
4721 {Local config file '\bbl@opt@config.cfg' not found.\\%
4722   Suggested actions:\\%
4723   * Make sure you haven't misspelled it in config=\\%
4724   * Check it exists and it's in the correct path}%
4725 {Perhaps you misspelled it.}
4726 \bbl@errmessage{late-after-babel}
4727 {Too late for \string\AfterBabelLanguage}%
4728 {Languages have been loaded, so I can do nothing}
4729 \bbl@errmessage{double-hyphens-class}
4730 {Double hyphens aren't allowed in \string\babelcharclass\\%
4731   because it's potentially ambiguous}%
4732 {See the manual for further info}
4733 \bbl@errmessage{unknown-interchar}
4734 {'#1' for '\language' cannot be enabled.\\%
4735   Maybe there is a typo}%
4736 {See the manual for further details.}
4737 \bbl@errmessage{unknown-interchar-b}
4738 {'#1' for '\language' cannot be disabled.\\%
4739   Maybe there is a typo}%
4740 {See the manual for further details.}
4741 \bbl@errmessage{charproperty-only-vertical}
4742 {\string\babelcharproperty\space can be used only in\\%
4743   vertical mode (preamble or between paragraphs)}%
4744 {See the manual for further info}
4745 \bbl@errmessage{unknown-char-property}
4746 {No property named '#2'. Allowed values are\\%
4747   direction (bc), mirror (bmg), and linebreak (lb)}%
4748 {See the manual for further info}
4749 \bbl@errmessage{bad-transform-option}
4750 {Bad option '#1' in a transform.\\%

```

```

4751 I'll ignore it but expect more errors}%
4752 {See the manual for further info.}
4753 \bbl@errmessage{font-conflict-transforms}
4754 {Transforms cannot be re-assigned to different\\%
4755 fonts. The conflict is in '\bbl@kv@label'.\\%
4756 Apply the same fonts or use a different label}%
4757 {See the manual for further details.}
4758 \bbl@errmessage{transform-not-available}
4759 {'#1' for '\language' cannot be enabled.\\%
4760 Maybe there is a typo or it's a font-dependent transform}%
4761 {See the manual for further details.}
4762 \bbl@errmessage{transform-not-available-b}
4763 {'#1' for '\language' cannot be disabled.\\%
4764 Maybe there is a typo or it's a font-dependent transform}%
4765 {See the manual for further details.}
4766 \bbl@errmessage{year-out-range}
4767 {Year out of range.\\%
4768 The allowed range is #1}%
4769 {See the manual for further details.}
4770 \bbl@errmessage{only-pdftex-lang}
4771 {The '#1' ldf style doesn't work with #2,\\%
4772 but you can use the ini locale instead.\\%
4773 Try adding 'provide=' to the option list. You may\\%
4774 also want to set 'bidi=' to some value}%
4775 {See the manual for further details.}
4776 \bbl@errmessage{hyphenmins-args}
4777 {\string\babelhyphenmins\ accepts either the optional\\%
4778 argument or the star, but not both at the same time}%
4779 {See the manual for further details.}
4780 \bbl@errmessage{no-locale-for-meta}
4781 {There isn't currently a locale for the 'lang' requested\\%
4782 in the PDF metadata ('#1'). To fix it, you can\\%
4783 set explicitly a similar language (using the same\\%
4784 script) with the key main= when loading babel. If you\\%
4785 continue, I'll fallback to the 'nil' language, with\\%
4786 tag 'und' and script 'Latn', but expect a bad font\\%
4787 rendering with other scripts. You may also need set\\%
4788 explicitly captions and date, too}%
4789 {See the manual for further details.}
4790 (/errors)
4791 (*patterns)

```

9. Loading hyphenation patterns

The following code is meant to be read by `iniTeX` because it should instruct `TeX` to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4792 <@Make sure ProvidesFile is defined@>
4793 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4794 \xdef\bbl@format{\jobname}
4795 \def\bbl@version{<@version@>}
4796 \def\bbl@date{<@date@>}
4797 \ifx\AtBeginDocument\undefined
4798 \def\empty{}
4799 \fi
4800 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```

4801 \def\process@line#1#2 #3 #4 {%

```

```

4802 \ifx=#1%
4803   \process@synonym{#2}%
4804 \else
4805   \process@language{#1#2}{#3}{#4}%
4806 \fi
4807 \ignorespaces}

```

\process@synonym This macro takes care of the lines which start with an =. It needs an empty token register to begin with. \bbl@languages is also set to empty.

```

4808 \toks@{}
4809 \def\bbl@languages{}

```

When no languages have been loaded yet, the name following the = will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The \relax just helps to the \if below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the hyphenmin parameters for the synonym.

```

4810 \def\process@synonym#1{%
4811   \ifnum\last@language=\m@ne
4812     \toks@{\expandafter\the\toks@\relax\process@synonym{#1}}%
4813   \else
4814     \expandafter\chardef\csname l@#1\endcsname\last@language
4815     \wlog{\string\l@#1=\string\language\the\last@language}%
4816     \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4817       \csname\language\endcsname hyphenmins\endcsname
4818     \let\bbl@elt\relax
4819     \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%
4820 \fi}

```

\process@language The macro \process@language is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call \addlanguage to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file language.dat by adding for instance ‘:T1’ to the name of the language. The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to \lefthyphenmin and \righthyphenmin. T_EX does not keep track of these assignments. Therefore we try to detect such assignments and store them in the \<language>hyphenmins macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the \lccode en \uccode arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the \patterns command acts globally so its effect will be remembered.

Then we globally store the settings of \lefthyphenmin and \righthyphenmin and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

\bbl@languages saves a snapshot of the loaded languages in the form \bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}. Note the last 2 arguments are empty in ‘dialects’ defined in language.dat with =. Note also the language name can have encoding info.

Finally, if the counter \language is equal to zero we execute the synonyms stored.

```

4821 \def\process@language#1#2#3{%
4822   \expandafter\addlanguage\csname l@#1\endcsname
4823   \expandafter\language\csname l@#1\endcsname
4824   \edef\language{#1}%
4825   \bbl@hook@everylanguage{#1}%
4826   % > luatex

```

```

4827 \bbl@get@enc#1:.\@@@
4828 \begingroup
4829 \lefthyphenmin\m@ne
4830 \bbl@hook@loadpatterns{#2}%
4831 % > luatex
4832 \ifnum\lefthyphenmin=\m@ne
4833 \else
4834 \expandafter\xdef\csname #1hyphenmins\endcsname{%
4835 \the\lefthyphenmin\the\righthyphenmin}%
4836 \fi
4837 \endgroup
4838 \def\bbl@tempa{#3}%
4839 \ifx\bbl@tempa\@empty\else
4840 \bbl@hook@loadexceptions{#3}%
4841 % > luatex
4842 \fi
4843 \let\bbl@elt\relax
4844 \edef\bbl@languages{%
4845 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4846 \ifnum\the\language=\z@
4847 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4848 \set@hyphenmins\tw@\thr@@\relax
4849 \else
4850 \expandafter\expandafter\expandafter\set@hyphenmins
4851 \csname #1hyphenmins\endcsname
4852 \fi
4853 \the\toks@
4854 \toks@{}%
4855 \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```

4856 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4857 \def\bbl@hook@everylanguage#1{}
4858 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4859 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4860 \def\bbl@hook@loadkernel#1{%
4861 \def\addlanguage{\csname newlanguage\endcsname}%
4862 \def\adddialect##1##2{%
4863 \global\chardef##1##2\relax
4864 \wlog{\string##1 = a dialect from \string\language##2}}%
4865 \def\iflanguage##1{%
4866 \expandafter\ifx\csname l@##1\endcsname\relax
4867 \nol@nerr{##1}%
4868 \else
4869 \ifnum\csname l@##1\endcsname=\language
4870 \expandafter\expandafter\expandafter\@firstoftwo
4871 \else
4872 \expandafter\expandafter\expandafter\@secondoftwo
4873 \fi
4874 \fi}%
4875 \def\providehyphenmins##1##2{%
4876 \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4877 \namedef{##1hyphenmins}{##2}%
4878 \fi}%
4879 \def\set@hyphenmins##1##2{%
4880 \lefthyphenmin##1\relax
4881 \righthyphenmin##2\relax}%

```

```

4882 \def\selectlanguage{%
4883   \errhelp{Selecting a language requires a package supporting it}%
4884   \errmessage{No multilingual package has been loaded}}%
4885 \let\foreignlanguage\selectlanguage
4886 \let\otherlanguage\selectlanguage
4887 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4888 \def\bbl@usehooks##1##2{%
4889 \def\setlocale{%
4890   \errhelp{Find an armchair, sit down and wait}%
4891   \errmessage{(babel) Not yet available}}%
4892 \let\uselocale\setlocale
4893 \let\locale\setlocale
4894 \let\selectlocale\setlocale
4895 \let\localename\setlocale
4896 \let\textlocale\setlocale
4897 \let\textlanguage\setlocale
4898 \let\languagetext\setlocale}
4899 \begingroup
4900 \def\AddBabelHook#1#2{%
4901   \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4902     \def\next{\toks1}%
4903   \else
4904     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname###1}%
4905   \fi
4906   \next}
4907 \ifx\directlua@undefined
4908   \ifx\XeTeXinputencoding\undefined\else
4909     \input xebabel.def
4910   \fi
4911 \else
4912   \input luababel.def
4913 \fi
4914 \openin1 = babel-\bbl@format.cfg
4915 \ifeof1
4916 \else
4917   \input babel-\bbl@format.cfg\relax
4918 \fi
4919 \closein1
4920 \endgroup
4921 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4922 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4923 \def\language{english}%
4924 \ifeof1
4925   \message{I couldn't find the file language.dat,\space
4926           I will try the file hyphen.tex}
4927   \input hyphen.tex\relax
4928   \chardef\l@english\z@
4929 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```

4930 \last@language@m@ne

```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```

4931 \loop
4932   \endlinechar\m@ne
4933   \readl to \bbl@line
4934   \endlinechar\^^M

```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```

4935   \if T\ifeoflF\fi T\relax
4936   \ifx\bbl@line\empty\else
4937     \edef\bbl@line{\bbl@line\space\space\space}%
4938     \expandafter\process@line\bbl@line\relax
4939   \fi
4940 \repeat

```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```

4941 \begingroup
4942   \def\bbl@elt#1#2#3#4{%
4943     \global\language=#2\relax
4944     \gdef\language#1}%
4945   \def\bbl@elt##1##2##3##4{}}%
4946   \bbl@languages
4947 \endgroup
4948 \fi
4949 \closeinl

```

We add a message about the fact that babel is loaded in the format and with which language patterns to the `\everyjob` register.

```

4950 \if/\the\toks@/\else
4951   \errhelp{language.dat loads no language, only synonyms}
4952   \errmessage{Orphan language synonym}
4953 \fi

```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```

4954 \let\bbl@line\undefined
4955 \let\process@line\undefined
4956 \let\process@synonym\undefined
4957 \let\process@language\undefined
4958 \let\bbl@get@enc\undefined
4959 \let\bbl@hyph@enc\undefined
4960 \let\bbl@tempa\undefined
4961 \let\bbl@hook@loadkernel\undefined
4962 \let\bbl@hook@everylanguage\undefined
4963 \let\bbl@hook@loadpatterns\undefined
4964 \let\bbl@hook@loadexceptions\undefined
4965 (/patterns)

```

Here the code for `iniTEX` ends.

10. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdftex`).

```

4966 ({*More package options}) ≡
4967 \chardef\bbl@bidimode\z@
4968 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4969 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4970 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4971 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4972 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }

```

```

4973 \DeclareOption{bidi= bidi-l}{\chardef\bb@bidimode=203 }
4974 \DeclareOption{bidi=unibidi}{\chardef\bb@bidimode=301 }
4975 {/More package options}

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `\bb@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```

4976 {(*Font selection)} ≡
4977 \bb@trace{Font handling with fontspec}
4978 \AddBabelHook{babel-fontspec}{afterextras}{\bb@switchfont}
4979 \AddBabelHook{babel-fontspec}{beforestart}{\bb@ckckstdfonts}
4980 \DisableBabelHook{babel-fontspec}
4981 \@onlypreamble\babelfont
4982 \ifx\NewDocumentCommand\@undefined\else % Not plain
4983   \NewDocumentCommand\babelfont{0}{m0}{m0}{m0}{m0}{%
4984     \bb@bblfont@o[#1]{#2}[#3,#5]{#4}}
4985 \fi
4986 \newcommand\bb@bblfont@o[2][l]{% 1=langs/scripts 2=fam
4987   \ifx\fontspec\@undefined
4988     \usepackage{fontspec}%
4989   \fi
4990   \EnableBabelHook{babel-fontspec}%
4991   \edef\bb@tempa{#1}%
4992   \def\bb@tempb{#2}% Used by \bb@bblfont
4993   \bb@bblfont}
4994 \newcommand\bb@bblfont[2][l]{% 1=features 2=fontname, @font=rm|sf|tt
4995   \bb@ifunset{\bb@tempb family}%
4996     {\bb@providefam{\bb@tempb}}%
4997   }%
4998   % For the default font, just in case:
4999   \bb@ifunset{\bb@lsys{\languagenome}}{\bb@provide{\languagenome}}{%
5000     \expandafter\bb@ifblank\expandafter{\bb@tempa}%
5001     {\bb@csarg\edef{\bb@tempb dflt@}{<{#1}{#2}}% save \bb@rmdflt@
5002     \bb@exp{%
5003       \let<\bb@tempb dflt@\languagenome>\bb@tempb dflt@>%
5004       \bb@font@set<\bb@tempb dflt@\languagenome>%
5005       \bb@tempb default>\bb@tempb family>}}%
5006     {\bb@foreach\bb@tempa{% i.e., \bb@rmdflt@lang / *scrt
5007       \bb@csarg\def{\bb@tempb dflt@##1}{<{#1}{#2}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

5008 \def\bb@providefam#1{%
5009   \bb@exp{%
5010     \newcommand<#1default>{}% Just define it
5011     \bb@add@list{\bb@font@fams{#1}}%
5012     \NewHook{#1family}%
5013     \DeclareRobustCommand<#1family>{%
5014       \not@math@alphabet<#1family>\relax
5015       % \prepare@family@series@update{#1}<#1default>% TODO. Fails
5016       \fontfamily<#1default>%
5017       \UseHook{#1family}%
5018       \selectfont}%
5019     \DeclareTextFontCommand{\text{#1}}{<#1family>}}

```

The following macro is activated when the hook `babel-fontspec` is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

5020 \def\bb@nostdfont#1{%
5021   \bb@once{nostdfam-\f@family}%
5022   {\bb@infowarn{The current font is not a babel standard family:\%
5023     #1%
5024     \fontname\font\%
5025     There is nothing intrinsically wrong, and you can\%,
5026     ignore this message altogether if you do not need\%

```

```

5027     this font. If they are used in the document, be aware\\%
5028     'babel' will not set Script and Language for it, so\\%
5029     you may consider defining a new family with \string\babelfont.\\%
5030     See the manual for further details about \string\babelfont.
5031     Reported}}}%
5032   {}}%
5033 \gdef\babel@switchfont{%
5034   \babel@ifunset{\babel@sys@\language}\babel@provide@sys@\language}}}%
5035   \babel@exp% e.g., Arabic -> arabic
5036   \lowercase{\edef\\babel@tempa{\babel@cl{sname}}}}}%
5037   \babel@foreach\babel@font@fams{%
5038     \babel@ifunset{\babel@##1dflt@\language}% (1) language?
5039     {\babel@ifunset{\babel@##1dflt@*\babel@tempa}% (2) from script?
5040       {\babel@ifunset{\babel@##1dflt@}% 2=F - (3) from generic?
5041         {}% 123=F - nothing!
5042         {\babel@exp% 3=T - from generic
5043           \global\let<\babel@##1dflt@\language>%
5044           \<\babel@##1dflt@>}}}%
5045       {\babel@exp% 2=T - from script
5046         \global\let<\babel@##1dflt@\language>%
5047         \<\babel@##1dflt@*\babel@tempa>}}}%
5048     {}}% 1=T - language, already defined
5049   \def\babel@tempa{\babel@nostdfont{}}}%
5050   \babel@foreach\babel@font@fams{% don't gather with prev for
5051     \babel@ifunset{\babel@##1dflt@\language}%
5052     {\babel@cs{famrst@##1}%
5053       \global\babel@csarg\let{famrst@##1}\relax}%
5054     {\babel@exp% order is relevant.
5055       \\babel@add\\originalTeX{%
5056         \\babel@font@rst{\babel@cl{##1dflt}}}%
5057         \<##1default>\<##1family>{##1}}}%
5058       \\babel@font@set<\babel@##1dflt@\language>% the main part!
5059       \<##1default>\<##1family>}}}%
5060   \babel@ifrestoring{\babel@tempa}}}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with \babelfont.

```

5061 \ifx\f@family\undefined\else % if latex
5062   \ifcase\babel@engine % if pdftex
5063     \let\babel@ckeckstdfonts\relax
5064   \else
5065     \def\babel@ckeckstdfonts{%
5066       \begingroup
5067       \global\let\babel@ckeckstdfonts\relax
5068       \let\babel@tempa\empty
5069       \babel@foreach\babel@font@fams{%
5070         \babel@ifunset{\babel@##1dflt@}%
5071         {\@nameuse{##1family}%
5072           \babel@csarg\gdef\WFF@f@family{}}}% Flag
5073           \babel@exp{\\babel@add\\babel@tempa{* \<##1family>= \f@family\\}%
5074             \space\space\fontname\font\\}%
5075             \babel@csarg\xdef{##1dflt@}{\f@family}%
5076             \expandafter\xdef\csname ##1default\endcsname{\f@family}}}%
5077         {}}%
5078       \ifx\babel@tempa\empty\else
5079         \babel@infowarn{The following font families will use the default\\%
5080           settings for all or some languages:\\%
5081           \babel@tempa
5082           There is nothing intrinsically wrong with it, but\\%
5083           'babel' will no set Script and Language, which could\\%
5084           be relevant in some languages. If your document uses\\%
5085           these families, consider redefining them with \string\babelfont.\\%
5086           Reported}%

```

```

5087      \fi
5088      \endgroup}
5089      \fi
5090 \fi

```

Now the macros defining the font with fontspec.

When there are repeated keys in fontspec, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes bx/sc is the correct font, but sometimes points to b/n, even if b/sc exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5091 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
5092   \bbl@xin@{<>}{#1}%
5093   \ifin@
5094     \bbl@exp{\bbl@fontspec@set\#1\expandafter\@gobbletwo#1\#3}%
5095   \fi
5096   \bbl@exp{%
5097     \def\#2{#1}% e.g., \rmdefault{\bbl@rmdflt@lang}
5098     \bbl@ifsamestring{#2}{\f@family}%
5099     {\#3%
5100      \bbl@ifsamestring{\f@series}{\bfdefault}{\bfseries}}}%
5101   \let\bbl@tempa\relax}%
5102   {}%

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5103 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5104   \let\bbl@tempa\bbl@mapselect
5105   \edef\bbl@tempb{\bbl@stripslash#4}% Catcodes hack (better pass it).
5106   \bbl@exp{\bbl@replace\bbl@tempb{\bbl@stripslash\family/}}}%
5107   \let\bbl@mapselect\relax
5108   \let\bbl@tempa@fam#4% e.g., '\rmfamily', to be restored below
5109   \let#4@empty % Make sure \renewfontfamily is valid
5110   \bbl@set@renderer
5111   \bbl@exp{%
5112     \let\bbl@tempa@pfam<\bbl@stripslash#4\space>% e.g., '\rmfamily '
5113     \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}}%
5114     {\newfontscript{\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5115     \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}}%
5116     {\newfontlanguage{\bbl@cl{lname}}{\bbl@cl{lotf}}}%
5117     \renewfontfamily\#4%
5118     [\bbl@cl{lsys},% xetex removes unknown features :-(
5119     \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5120     #2}{#3}% i.e., \bbl@exp{.}{#3}
5121   \bbl@unset@renderer
5122   \beginngroup
5123     #4%
5124     \xdef#1{\f@family}% e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5125   \endgroup
5126   \bbl@xin@{\detokenize{>ssub*}}}%
5127   {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5128   \ifin@
5129     \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5130   \fi
5131   \bbl@xin@{\detokenize{>ssub*}}}%
5132   {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5133   \ifin@

```

```

5134 \global\bbL@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5135 \fi
5136 \let#4\bbL@temp@fam
5137 \bbL@exp{\let<\bbL@stripslash#4\space>}\bbL@temp@pfam
5138 \let\bbL@mapselect\bbL@tempe}%

```

font@rst and famrst are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5139 \def\bbL@font@rst#1#2#3#4{%
5140 \bbL@csarg\def{famrst@#4}{\bbL@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with \babelfont.

```

5141 \def\bbL@font@fams{rm,sf,tt}
5142 <</Font selection>>

```

11. Hooks for XeTeX and LuaTeX

11.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to utf8, which seems a sensible default.

Now, the code.

```

5143 (*xetex)
5144 \def\BabelStringsDefault{unicode}
5145 \let\xebbl@stop\relax
5146 \AddBabelHook{xetex}{encodedcommands}{%
5147 \def\bbL@tempa{#1}%
5148 \ifx\bbL@tempa@empty
5149 \XeTeXinputencoding"bytes"%
5150 \else
5151 \XeTeXinputencoding"#1"%
5152 \fi
5153 \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5154 \AddBabelHook{xetex}{stopcommands}{%
5155 \xebbl@stop
5156 \let\xebbl@stop\relax}
5157 \def\bbL@input@classes{% Used in CJK intraspaces
5158 \input{load-unicode-xetex-classes.tex}%
5159 \let\bbL@input@classes\relax}
5160 \def\bbL@intraspace#1 #2 #3\@@{%
5161 \bbL@csarg\gdef{xeisp@\languagename}%
5162 {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5163 \def\bbL@intrapenalty#1\@@{%
5164 \bbL@csarg\gdef{xeipn@\languagename}%
5165 {\XeTeXlinebreakpenalty #1\relax}}
5166 \def\bbL@provide@intraspace{%
5167 \bbL@xin@{/s}{\bbL@cl{lnbrk}}}%
5168 \ifin@else\bbL@xin@{/c}{\bbL@cl{lnbrk}}\fi
5169 \ifin@
5170 \bbL@ifunset{bbL@intsp@\languagename}{}%
5171 {\expandafter\ifx\csname bbL@intsp@\languagename\endcsname\@empty\else
5172 \ifx\bbL@KVP@intraspace\@nnil
5173 \bbL@exp{%
5174 \bbL@intraspace\bbL@cl{intsp}\@@}%
5175 \fi
5176 \ifx\bbL@KVP@intrapenalty\@nnil
5177 \bbL@intrapenalty0\@@
5178 \fi
5179 \fi
5180 \ifx\bbL@KVP@intraspace\@nnil\else % We may override the ini
5181 \expandafter\bbL@intraspace\bbL@KVP@intraspace\@@

```

```

5182 \fi
5183 \ifx\bbbl@KVP@intrapenalty\@nnil\else
5184 \expandafter\bbbl@intrapenalty\bbbl@KVP@intrapenalty\@@
5185 \fi
5186 \bbbl@exp{%
5187 \\\bbbl@add<extras\language>{%
5188 \XeTeXlinebreaklocale "\bbbl@cl{tbc}"%
5189 \<bbbl@xeisp@language>%
5190 \<bbbl@xeipn@language>%
5191 \\\bbbl@tglobal\<extras\language>%
5192 \\\bbbl@add<noextras\language>{%
5193 \XeTeXlinebreaklocale ""}%
5194 \\\bbbl@tglobal\<noextras\language>%
5195 \ifx\bbbl@ispace\@undefined
5196 \gdef\bbbl@ispace{\bbbl@cl{xeisp}}%
5197 \ifx\AtBeginDocument\@notprerr
5198 \expandafter\@secondoftwo % to execute right now
5199 \fi
5200 \AtBeginDocument{\bbbl@patchfont{\bbbl@ispace}}%
5201 \fi}%
5202 \fi}
5203 \ifx\DisableBabelHook\@undefined\endinput\fi
5204 \let\bbbl@set@renderer\relax
5205 \let\bbbl@unset@renderer\relax
5206 <@Font selection@>
5207 \def\bbbl@provide@extra#1{}

Hack for unhyphenated line breaking. See \bbbl@provide@lsys in the common code.

5208 \def\bbbl@xenohyph@{%
5209 \bbbl@ifset{\bbbl@prehc@language}%
5210 {\ifnum\hyphenchar\font=\defaultthyphenchar
5211 \iffontchar\font\bbbl@cl{prehc}\relax
5212 \hyphenchar\font\bbbl@cl{prehc}\relax
5213 \else\iffontchar\font"200B
5214 \hyphenchar\font"200B
5215 \else
5216 \bbbl@warning
5217 {Neither 0 nor ZERO WIDTH SPACE are available\\%
5218 in the current font, and therefore the hyphen\\%
5219 will be printed. Try changing the fontspec's\\%
5220 'HyphenChar' to another value, but be aware\\%
5221 this setting is not safe (see the manual).\\%
5222 Reported}%
5223 \hyphenchar\font\defaultthyphenchar
5224 \fi\fi
5225 \fi}%
5226 {\hyphenchar\font\defaultthyphenchar}}

```

11.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5227 \ifnum\xe@alloc@intercharclass<\thr@@
5228 \xe@alloc@intercharclass\thr@@
5229 \fi
5230 \chardef\bbbl@xe@class@default@=\z@
5231 \chardef\bbbl@xe@class@cjkideogram@=\@ne
5232 \chardef\bbbl@xe@class@cjkleftpunctuation@=\tw@
5233 \chardef\bbbl@xe@class@cjkrightpunctuation@=\thr@@
5234 \chardef\bbbl@xe@class@boundary@=4095
5235 \chardef\bbbl@xe@class@ignore@=4096

```

The machinery is activated with a hook (enabled only if actually used). Here \bbbl@tempc is pre-set with \bbbl@usingxe@class, defined below. The standard mechanism based on \originalTeX to save,

set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```

5236 \AddBabelHook{babel-interchar}{beforeextras}{%
5237   \@nameuse{bbl@xechars@\language\name}}
5238 \DisableBabelHook{babel-interchar}
5239 \protected\def\bbl@charclass#1{%
5240   \ifnum\count@<\z@
5241     \count@-\count@
5242     \loop
5243       \bbl@exp{%
5244         \\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5245         \XeTeXcharclass\count@ \bbl@tempc
5246         \ifnum\count@<`#1\relax
5247           \advance\count@\@ne
5248         \repeat
5249   \else
5250     \babel@savevariable{\XeTeXcharclass`#1}%
5251     \XeTeXcharclass`#1 \bbl@tempc
5252   \fi
5253   \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above has internally the form \bbl@usingxechar\bbl@xechar@punct@english\bbl@charclass{.} \bbl@charclass{,} (etc.), where \bbl@usingxechar stores the class to be applied to the subsequent characters. The \ifcat part deals with the alternative way to enter characters as macros (e.g., \). As a special case, hyphens are stored as \bbl@upto, to deal with ranges.

```

5254 \newcommand\bbl@ifinterchar[1]{%
5255   \let\bbl@tempa\@gobble % Assume to ignore
5256   \edef\bbl@tempb{\zap@space#1 \@empty}%
5257   \ifx\bbl@KVP@interchar\@nnil\else
5258     \bbl@replace\bbl@KVP@interchar{ }{,}%
5259     \bbl@foreach\bbl@tempb{%
5260       \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5261       \ifin@
5262         \let\bbl@tempa\@firstofone
5263       \fi}%
5264   \fi
5265   \bbl@tempa}
5266 \newcommand\IfBabelIntercharT[2]{%
5267   \bbl@carg\bbl@add{\bbl@icsave@\CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5268 \newcommand\babelcharclass[3]{%
5269   \EnableBabelHook{babel-interchar}%
5270   \bbl@csarg\newXeTeXintercharclass{xechar@#2@#1}%
5271   \def\bbl@tempb##1{%
5272     \ifx##1\@empty\else
5273       \ifx##1-%
5274         \bbl@upto
5275       \else
5276         \bbl@charclass{%
5277           \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5278         \fi
5279         \expandafter\bbl@tempb
5280       \fi}%
5281   \bbl@ifunset{\bbl@xechars@#1}%
5282   {\toks@{%
5283     \babel@savevariable\XeTeXinterchartokenstate
5284     \XeTeXinterchartokenstate\@ne
5285   }}%
5286   {\toks@\expandafter\expandafter\expandafter{%
5287     \csname bbl@xechars@#1\endcsname}}}%
5288   \bbl@csarg\edef{xechars@#1}{%
5289     \the\toks@

```

```

5290 \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5291 \bbl@tempb#3\@empty}}
5292 \protected\def\bbl@usingxeclass#1{\count@ \z@ \let\bbl@tempc#1}
5293 \protected\def\bbl@upto{%
5294 \ifnum\count@>\z@
5295 \advance\count@\@ne
5296 \count@-\count@
5297 \else\ifnum\count@=\z@
5298 \bbl@charclass{-}%
5299 \else
5300 \bbl@error{double-hyphens-class}{}}{}%
5301 \fi\fi}

And finally, the command with the code to be inserted. If the language doesn't define a class, then
use the global one, as defined above. For the definition there is a intermediate macro, which can be
'disabled' with \bbl@ic@<label>@<language>.

5302 \def\bbl@ignoreinterchar{%
5303 \ifnum\language=\@nohyphenation
5304 \expandafter\@gobble
5305 \else
5306 \expandafter\@firstofone
5307 \fi}
5308 \newcommand\babelinterchar[5][{}]{%
5309 \let\bbl@kv@label\@empty
5310 \bbl@forkv{#1}{\bbl@csarg\edef{kv@##1}{##2}}%
5311 \@namedef{\zap@space bbl@xeinter@\bbl@kv@label @#3@#4@#2 \@empty}%
5312 {\bbl@ignoreinterchar{#5}}%
5313 \bbl@csarg\let{ic@\bbl@kv@label @#2}\@firstofone
5314 \bbl@exp{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}}%
5315 \bbl@exp{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}}%
5316 \XeTeXinterchartoks
5317 \@nameuse{bbl@xeclass@\bbl@tempa @#2}
5318 \bbl@ifunset{bbl@xeclass@\bbl@tempa @#2}{#2} %
5319 \@nameuse{bbl@xeclass@\bbl@tempb @#2}
5320 \bbl@ifunset{bbl@xeclass@\bbl@tempb @#2}{#2} %
5321 = \expandafter{%
5322 \csname bbl@ic@\bbl@kv@label @#2\expandafter\endcsname
5323 \csname\zap@space bbl@xeinter@\bbl@kv@label
5324 @#3@#4@#2 \@empty\endcsname}}}%
5325 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5326 \bbl@ifunset{bbl@ic@#1@language}%
5327 {\bbl@error{unknown-interchar}{#1}}{}%
5328 {\bbl@csarg\let{ic@#1@language}\@firstofone}}
5329 \DeclareRobustCommand\disablelocaleinterchar[1]{%
5330 \bbl@ifunset{bbl@ic@#1@language}%
5331 {\bbl@error{unknown-interchar-b}{#1}}{}%
5332 {\bbl@csarg\let{ic@#1@language}\@gobble}}
5333 (/xetex)

```

11.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titles, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdftex and xetex.

```

5334 (*xetex|texet)
5335 \providecommand\bbl@provide@intraspace{}
5336 \bbl@trace{Redefinitions for bidi layout}

```

Finish here if there is no layout.

```

5337 \ifx\bb@opt@layout\@nnil\else % if layout=..
5338 \ifx\bb@opt@layout\@undefined\else % Plain
5339 \IfBabelLayout{nopars}
5340 {}
5341 {\edef\bb@opt@layout{\bb@opt@layout.pars.}}%
5342 \fi
5343 \def\bb@startskip{\ifcase\bb@thepardir\leftskip\else\rightskip\fi}
5344 \def\bb@endskip{\ifcase\bb@thepardir\rightskip\else\leftskip\fi}
5345 \ifnum\bb@bidimode>\z@
5346 \IfBabelLayout{pars}
5347 {\def\@hangfrom#1{%
5348 \setbox\@tempboxa\hbox{#1}}%
5349 \hangindent\ifcase\bb@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5350 \noindent\box\@tempboxa}
5351 \def\raggedright{%
5352 \let\@centercr
5353 \bb@startskip\z@skip
5354 \@rightskip\@flushglue
5355 \bb@endskip\@rightskip
5356 \parindent\z@
5357 \parfillskip\bb@startskip}
5358 \def\raggedleft{%
5359 \let\@centercr
5360 \bb@startskip\@flushglue
5361 \bb@endskip\z@skip
5362 \parindent\z@
5363 \parfillskip\bb@endskip}}
5364 {}
5365 \fi
5366 \IfBabelLayout{lists}
5367 {\bb@sreplace\list
5368 {\@totalleftmargin\leftmargin}{\@totalleftmargin\bb@listleftmargin}%
5369 \def\bb@listleftmargin{%
5370 \ifcase\bb@thepardir\leftmargin\else\rightmargin\fi}%
5371 \ifcase\bb@engine
5372 \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5373 \def\p@enumiii{\p@enumii}\theenumii{}\fi
5374 \fi
5375 \bb@sreplace\@verbatim
5376 {\leftskip\@totalleftmargin}%
5377 {\bb@startskip\textwidth
5378 \advance\bb@startskip-\linewidth}%
5379 \bb@sreplace\@verbatim
5380 {\rightskip\z@skip}%
5381 {\bb@endskip\z@skip}}%
5382 {}
5383 \IfBabelLayout{contents}
5384 {\bb@sreplace\@dottedtocline{\leftskip}{\bb@startskip}%
5385 \bb@sreplace\@dottedtocline{\rightskip}{\bb@endskip}}
5386 {}
5387 \IfBabelLayout{columns}
5388 {\bb@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bb@outputbox}%
5389 \def\bb@outputbox#1{%
5390 \hb@xt@\textwidth{%
5391 \hskip\columnwidth
5392 \hfil
5393 {\normalcolor\vrule \@width\columnseprule}%
5394 \hfil
5395 \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5396 \hskip-\textwidth
5397 \hb@xt@\columnwidth{\box\@outputbox \hss}%
5398 \hskip\columnsep
5399 \hskip\columnwidth}}}%

```

5400 {}

Implicitly reverses sectioning labels in bidi=basic, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5401 \IfBabelLayout{counters*}%
5402 {\bbl@add\bbl@opt@layout{.counters.}%
5403 \AddToHook{shipout/before}{%
5404 \let\bbl@tempa\babelsublr
5405 \let\babelsublr\@firstofone
5406 \let\bbl@save@thepage\thepage
5407 \protected@edef\thepage{\thepage}%
5408 \let\babelsublr\bbl@tempa}%
5409 \AddToHook{shipout/after}{%
5410 \let\thepage\bbl@save@thepage}}{}
5411 \IfBabelLayout{counters*}%
5412 {\let\bbl@latinarabic=\@arabic
5413 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5414 \let\bbl@asciroman=\@roman
5415 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5416 \let\bbl@asciiRoman=\@Roman
5417 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5418 \fi % end if layout
5419 (/xetex|texxet)

```

11.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```

5420 (*texxet)
5421 \def\bbl@provide@extra#1{%
5422 % == auto-select encoding ==
5423 \ifx\bbl@encoding@select@off\@empty\else
5424 \bbl@ifunset{\bbl@encoding@#1}%
5425 {\def\@elt##1{,##1,}%
5426 \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5427 \count@z@
5428 \bbl@foreach\bbl@tempe{%
5429 \def\bbl@tempd{##1}% Save last declared
5430 \advance\count@z@\@ne}%
5431 \ifnum\count@z@>\@ne % (1)
5432 \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5433 \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5434 \bbl@replace\bbl@tempa{ },}%
5435 \global\bbl@csarg\let{encoding@#1}\@empty
5436 \bbl@xin@{,\bbl@tempd,},{,\bbl@tempa,}%
5437 \ifin@Velse % if main encoding included in ini, do nothing
5438 \let\bbl@tempb\relax
5439 \bbl@foreach\bbl@tempa{%
5440 \ifx\bbl@tempb\relax
5441 \bbl@xin@{,##1,},{,\bbl@tempe,}%
5442 \ifin@\def\bbl@tempb{##1}\fi
5443 \fi}%
5444 \ifx\bbl@tempb\relax\else
5445 \bbl@exp{%
5446 \global\<\bbl@add>\<\bbl@preextras@#1>{\<\bbl@encoding@#1>}%
5447 \gdef\<\bbl@encoding@#1>{%
5448 \\\babel@save\\f@encoding
5449 \\\bbl@add\\originalTeX{\\selectfont}%
5450 \\\fontencoding{\bbl@tempb}%
5451 \\\selectfont}}%
5452 \fi
5453 \fi
5454 \fi}%

```

```

5455      {}%
5456    \fi}
5457 (/texet)

```

11.5. LuaTeX

The loader for luatex is based solely on `language.dat`, which is read on the fly. The code shouldn't be executed when the format is build, so we check if `\AddBabelHook` is defined. Then comes a modified version of the loader in `hyphen.cfg` (without the `hyphenmins` stuff, which is under the direct control of `babel`).

The names `\l@<language>` are defined and take some value from the beginning because all `ldf` files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the `ldf` finishes). If a language has been loaded, `\bbl@hyphendata@<num>` exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in `language.dat` have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, lua(e)tex is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on `babel`, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format `language.dat` is used (under the principle of a single source), instead of `language.def`.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by `babel`) provide a command to allocate them (although there are packages like `ctablestack`). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, `etex.sty` changes the way languages are allocated.

This files is read at three places: (1) when `plain.def`, `babel.sty` starts, to read the list of available languages from `language.dat` (for the base option); (2) at `hyphen.cfg`, to modify some macros; (3) in the middle of `plain.def` and `babel.sty`, by `babel.def`, with the commands and other definitions for luatex (e.g., `\babelpatterns`).

```

5458 (*luatex)
5459 \directlua{ Babel = Babel or {} } % DL2
5460 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5461 \bbl@trace{Read language.dat}
5462 \ifx\bbl@readstream\undefined
5463   \csname newread\endcsname\bbl@readstream
5464 \fi
5465 \begingroup
5466   \toks@{}
5467   \count@ \z@ % 0=start, 1=0th, 2=normal
5468   \def\bbl@process@line#1#2 #3 #4 {%
5469     \ifx=#1%
5470       \bbl@process@synonym{#2}%
5471     \else
5472       \bbl@process@language{#1#2}{#3}{#4}%
5473     \fi
5474     \ignorespaces}
5475   \def\bbl@manylang{%
5476     \ifnum\bbl@last>\@ne
5477       \bbl@info{Non-standard hyphenation setup}%
5478     \fi
5479     \let\bbl@manylang\relax}
5480   \def\bbl@process@language#1#2#3{%
5481     \ifcase\count@
5482       \@ifundefined{zth@#1}{\count@\tw@}{\count@\@ne}%

```

```

5483 \or
5484 \count@ \tw@
5485 \fi
5486 \ifnum \count@ = \tw@
5487 \expandafter \addlanguage \csname l@#1 \endcsname
5488 \language \allocationnumber
5489 \chardef \bbl@last \allocationnumber
5490 \bbl@many lang
5491 \let \bbl@elt \relax
5492 \xdef \bbl@languages {%
5493 \bbl@languages \bbl@elt{#1}{\the \language}{#2}{#3}}%
5494 \fi
5495 \the \toks@
5496 \toks@ {}
5497 \def \bbl@process@synonym@aux#1#2{%
5498 \global \expandafter \chardef \csname l@#1 \endcsname #2 \relax
5499 \let \bbl@elt \relax
5500 \xdef \bbl@languages {%
5501 \bbl@languages \bbl@elt{#1}{#2}{}}}%
5502 \def \bbl@process@synonym#1{%
5503 \ifcase \count@
5504 \toks@ \expandafter {\the \toks@ \relax \bbl@process@synonym{#1}}%
5505 \or
5506 \ifundefined{zth@#1}{\bbl@process@synonym@aux{#1}{0}}}%
5507 \else
5508 \bbl@process@synonym@aux{#1}{\the \bbl@last}%
5509 \fi}
5510 \ifx \bbl@languages \@undefined % Just a (sensible?) guess
5511 \chardef \l@english \z@
5512 \chardef \l@USenglish \z@
5513 \chardef \bbl@last \z@
5514 \global \@namedef{bbl@hyphendata@0}{{\hyphen.tex}}
5515 \gdef \bbl@languages {%
5516 \bbl@elt{english}{0}{\hyphen.tex}}%
5517 \bbl@elt{USenglish}{0}{}}
5518 \else
5519 \global \let \bbl@languages @format \bbl@languages
5520 \def \bbl@elt#1#2#3#4{% Remove all except language 0
5521 \ifnum #2 > \z@ \else
5522 \noexpand \bbl@elt{#1}{#2}{#3}{#4}%
5523 \fi}%
5524 \xdef \bbl@languages {\bbl@languages}%
5525 \fi
5526 \def \bbl@elt#1#2#3#4{\@namedef{zth@#1}} % Define flags
5527 \bbl@languages
5528 \openin \bbl@readstream = language.dat
5529 \ifeof \bbl@readstream
5530 \bbl@warning{I couldn't find language.dat. No additional\\%
5531 patterns loaded. Reported}%
5532 \else
5533 \loop
5534 \endlinechar \m@ne
5535 \read \bbl@readstream to \bbl@line
5536 \endlinechar ``^M
5537 \if T \ifeof \bbl@readstream \fi T \relax
5538 \ifx \bbl@line \@empty \else
5539 \edef \bbl@line {\bbl@line \space \space \space}%
5540 \expandafter \bbl@process@line \bbl@line \relax
5541 \fi
5542 \repeat
5543 \fi
5544 \closein \bbl@readstream
5545 \endgroup

```

```

5546 \bbl@trace{Macros for reading patterns files}
5547 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5548 \ifx\babelcatcodetablenum\undefined
5549 \ifx\newcatcodetable\undefined
5550 \def\babelcatcodetablenum{5211}
5551 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5552 \else
5553 \newcatcodetable\babelcatcodetablenum
5554 \newcatcodetable\bbl@pattcodes
5555 \fi
5556 \else
5557 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5558 \fi
5559 \def\bbl@luapatterns#1#2{%
5560 \bbl@get@enc#1:.\@@@
5561 \setbox\z@\hbox\bgroup
5562 \begingroup
5563 \savecatcodetable\babelcatcodetablenum\relax
5564 \initcatcodetable\bbl@pattcodes\relax
5565 \catcodetable\bbl@pattcodes\relax
5566 \catcode\#=6 \catcode\$=3 \catcode\&=4 \catcode\^=7
5567 \catcode\_ =8 \catcode\{=1 \catcode\}=2 \catcode\~=13
5568 \catcode\@=11 \catcode\^^I=10 \catcode\^^J=12
5569 \catcode\<=12 \catcode\>=12 \catcode\*=12 \catcode\.=12
5570 \catcode\-=12 \catcode\/=12 \catcode\[=12 \catcode\]=12
5571 \catcode\`=12 \catcode\'=12 \catcode\"=12
5572 \input #1\relax
5573 \catcodetable\babelcatcodetablenum\relax
5574 \endgroup
5575 \def\bbl@tempa{#2}%
5576 \ifx\bbl@tempa\empty\else
5577 \input #2\relax
5578 \fi
5579 \egroup}%
5580 \def\bbl@patterns@lua#1{%
5581 \language=\expandafter\ifx\csname l@#1:f@encoding\endcsname\relax
5582 \csname l@#1\endcsname
5583 \edef\bbl@tempa{#1}%
5584 \else
5585 \csname l@#1:f@encoding\endcsname
5586 \edef\bbl@tempa{#1:f@encoding}%
5587 \fi\relax
5588 \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5589 \@ifundefined{bbl@hyphendata@the\language}%
5590 {\def\bbl@elt##1##2###4{%
5591 \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5592 \def\bbl@tempb{##3}%
5593 \ifx\bbl@tempb\empty\else % if not a synonymous
5594 \def\bbl@tempc{{##3}{##4}}%
5595 \fi
5596 \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5597 \fi}%
5598 \bbl@languages
5599 \@ifundefined{bbl@hyphendata@the\language}%
5600 {\bbl@info{No hyphenation patterns were set for\%
5601 language '\bbl@tempa'. Reported}}%
5602 {\expandafter\expandafter\expandafter\bbl@luapatterns
5603 \csname bbl@hyphendata@the\language\endcsname}}}%
5604 \endinput\fi

Here ends \ifx\AddBabelHook\undefined. A few lines are only read by HYPHEN.CFG.

5605 \ifx\DisableBabelHook\undefined
5606 \AddBabelHook{luatex}{everylanguage}{%

```

```

5607 \def\process@language##1##2##3{%
5608 \def\process@line####1####2 ####3 ####4 {}}
5609 \AddBabelHook{luatex}{loadpatterns}{%
5610 \input #1\relax
5611 \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5612 {{#1}}}}
5613 \AddBabelHook{luatex}{loadexceptions}{%
5614 \input #1\relax
5615 \def\bbl@tempb##1##2{{#1}{#1}}%
5616 \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5617 {\expandafter\expandafter\expandafter\bbl@tempb
5618 \csname bbl@hyphendata@the\language\endcsname}}
5619 \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5620 \begingroup
5621 \catcode`\%=12
5622 \catcode`\'=12
5623 \catcode`\|=12
5624 \catcode`\:=12
5625 \directlua{
5626 Babel.locale_props = Babel.locale_props or {}
5627
5628 function Babel.lua_error(e, a)
5629 tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5630 e .. '}' .. (a or '') .. '}{'}])
5631 end
5632
5633 function Babel.bytes(line)
5634 return line:gsub(".",
5635 function (chr) return unicode.utf8.char(string.byte(chr)) end)
5636 end
5637
5638 function Babel.priority_in_callback(name,description)
5639 for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5640 if v == description then return i end
5641 end
5642 return false
5643 end
5644
5645 function Babel.begin_process_input()
5646 if luatexbase and luatexbase.add_to_callback then
5647 luatexbase.add_to_callback('process_input_buffer',
5648 Babel.bytes,'Babel.bytes')
5649 else
5650 Babel.callback = callback.find('process_input_buffer')
5651 callback.register('process_input_buffer',Babel.bytes)
5652 end
5653 end
5654 function Babel.end_process_input ()
5655 if luatexbase and luatexbase.remove_from_callback then
5656 luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5657 else
5658 callback.register('process_input_buffer',Babel.callback)
5659 end
5660 end
5661
5662 function Babel.str_to_nodes(fn, matches, base)
5663 local n, head, last
5664 if fn == nil then return nil end
5665 for s in string.utfvalues(fn(matches)) do
5666 if base.id == 7 then

```

```

5667     base = base.replace
5668 end
5669 n = node.copy(base)
5670 n.char = s
5671 if not head then
5672     head = n
5673 else
5674     last.next = n
5675 end
5676 last = n
5677 end
5678 return head
5679 end
5680
5681 Babel.linebreaking = Babel.linebreaking or {}
5682 Babel.linebreaking.before = {}
5683 Babel.linebreaking.after = {}
5684 Babel.locale = {}
5685 function Babel.linebreaking.add_before(func, pos)
5686     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5687     if pos == nil then
5688         table.insert(Babel.linebreaking.before, func)
5689     else
5690         table.insert(Babel.linebreaking.before, pos, func)
5691     end
5692 end
5693 function Babel.linebreaking.add_after(func)
5694     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5695     table.insert(Babel.linebreaking.after, func)
5696 end
5697
5698 function Babel.addpatterns(pp, lg)
5699     local lg = lang.new(lg)
5700     local pats = lang.patterns(lg) or ''
5701     lang.clear_patterns(lg)
5702     for p in pp:gmatch('[^%s]+') do
5703         ss = ''
5704         for i in string.utfcharacters(p:gsub('%d', '')) do
5705             ss = ss .. '%d?' .. i
5706         end
5707         ss = ss:gsub('^%d%?%', '%%.') .. '%d?'
5708         ss = ss:gsub('%.%d%?$', '%%.')
5709         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5710         if n == 0 then
5711             tex.sprint(
5712                 [[\string\csname\space bbl@info\endcsname{New pattern: }]]
5713                 .. p .. [[{}]])
5714             pats = pats .. ' ' .. p
5715         else
5716             tex.sprint(
5717                 [[\string\csname\space bbl@info\endcsname{Renew pattern: }]]
5718                 .. p .. [[{}]])
5719         end
5720     end
5721     lang.patterns(lg, pats)
5722 end
5723
5724 Babel.characters = Babel.characters or {}
5725 Babel.ranges = Babel.ranges or {}
5726 function Babel.hlist_has_bidi(head)
5727     local has_bidi = false
5728     local ranges = Babel.ranges
5729     for item in node.traverse(head) do

```

```

5730     if item.id == node.id'glyph' then
5731         local itemchar = item.char
5732         local chardata = Babel.characters[itemchar]
5733         local dir = chardata and chardata.d or nil
5734         if not dir then
5735             for nn, et in ipairs(ranges) do
5736                 if itemchar < et[1] then
5737                     break
5738                 elseif itemchar <= et[2] then
5739                     dir = et[3]
5740                     break
5741                 end
5742             end
5743         end
5744         if dir and (dir == 'al' or dir == 'r') then
5745             has_bidi = true
5746         end
5747     end
5748 end
5749 return has_bidi
5750 end
5751 function Babel.set_chranges_b (script, chrng)
5752     if chrng == '' then return end
5753     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')
5754     Babel.script_blocks[script] = {}
5755     for s, e in string.gmatch(chrng..' ', '(.)%.%.(.)%s') do
5756         table.insert(
5757             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5758     end
5759 end
5760
5761 function Babel.discard_sublr(str)
5762     if str:find( [[\string\indexentry]] ) and
5763        str:find( [[\string\babelsublr]] ) then
5764         str = str:gsub( [[\string\babelsublr%s*(%b{})]],
5765                        function(m) return m:sub(2,-2) end )
5766     end
5767     return str
5768 end
5769 }
5770 \endgroup
5771 \ifx\newattribute\@undefined\else % Test for plain
5772     \newattribute\bbl@attr@locale % DL4
5773     \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5774     \AddBabelHook{luatex}{beforeextras}{%
5775         \setattribute\bbl@attr@locale\localeid}
5776 \fi
5777 %
5778 \def\BabelStringsDefault{unicode}
5779 \let\luabbl@stop\relax
5780 \AddBabelHook{luatex}{encodedcommands}{%
5781     \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5782     \ifx\bbl@tempa\bbl@tempb\else
5783         \directlua{Babel.begin_process_input()}%
5784         \def\luabbl@stop{%
5785             \directlua{Babel.end_process_input()}}%
5786     \fi}%
5787 \AddBabelHook{luatex}{stopcommands}{%
5788     \luabbl@stop
5789     \let\luabbl@stop\relax}
5790 %
5791 \AddBabelHook{luatex}{patterns}{%
5792     \@ifundefined{bbl@hyphendata@the\language}%

```

```

5793 {\def\bbl@elt##1##2##3##4{%
5794   \ifnum##2=\csname l@#2\endcsname % #2=spanish, dutch:OT1...
5795   \def\bbl@tempb{##3}%
5796   \ifx\bbl@tempb\@empty\else % if not a synonymous
5797     \def\bbl@tempc{##3}{##4}%
5798   \fi
5799   \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5800   \fi}%
5801 \bbl@languages
5802 \@ifundefined{bbl@hyphendata@the\language}%
5803   {\bbl@info{No hyphenation patterns were set for\%
5804     language '#2'. Reported}}%
5805   {\expandafter\expandafter\expandafter\bbl@luapatterns
5806     \csname bbl@hyphendata@the\language\endcsname}}}%
5807 \@ifundefined{bbl@patterns@}{}%
5808   \begingroup
5809     \bbl@xin@{, \number\language,}{, \bbl@pttnlist}%
5810     \ifin@else
5811       \ifx\bbl@patterns@\@empty\else
5812         \directlua{ Babel.addpatterns(
5813           [[\bbl@patterns@]], \number\language) }%
5814       \fi
5815       \@ifundefined{bbl@patterns@#1}%
5816       \@empty
5817       {\directlua{ Babel.addpatterns(
5818         [[\space\csname bbl@patterns@#1\endcsname]],
5819         \number\language) }}%
5820       \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5821     \fi
5822   \endgroup}%
5823 \bbl@exp{%
5824   \bbl@ifunset{bbl@prehc@\language\name}{}%
5825   {\bbl@ifblank{\bbl@cs{prehc@\language\name}}{}}%
5826   {\prehyphenchar=\bbl@cl{prehc}\relax}}}%

```

\babelpatterns This macro adds patterns. Two macros are used to store them: \bbl@patterns@ for the global ones and \bbl@patterns@(*language*) for language ones. We make sure there is a space between words when multiple commands are used.

```

5827 \@onlypreamble\babelpatterns
5828 \AtEndOfPackage{%
5829   \newcommand\babelpatterns[2][\@empty]{%
5830     \ifx\bbl@patterns@\relax
5831       \let\bbl@patterns@\@empty
5832     \fi
5833     \ifx\bbl@pttnlist@\@empty\else
5834       \bbl@warning{%
5835         You must not intermingle \string\selectlanguage\space and\%
5836         \string\babelpatterns\space or some patterns will not\%
5837         be taken into account. Reported}%
5838       \fi
5839       \ifx\@empty#1%
5840         \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5841       \else
5842         \edef\bbl@tempb{\zap@space#1 \@empty}%
5843         \bbl@for\bbl@tempa\bbl@tempb{%
5844           \bbl@fixname\bbl@tempa
5845           \bbl@iflanguage\bbl@tempa{%
5846             \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5847               \@ifundefined{bbl@patterns@\bbl@tempa}%
5848               \@empty
5849               {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5850               #2}}}%
5851         \fi}}

```

11.6. Southeast Asian scripts

First, some general code for line breaking, used by `\babelposthyphenation`.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```
5852 \def\bbl@intraspace#1 #2 #3\@@{%
5853   \directlua{
5854     Babel.intraspaces = Babel.intraspaces or {}
5855     Babel.intraspaces['\csname bbl@sbc@language\endcsname'] = %
5856       {b = #1, p = #2, m = #3}
5857     Babel.locale_props[\the\localeid].intraspace = %
5858       {b = #1, p = #2, m = #3}
5859   }}
5860 \def\bbl@intrapenalty#1\@@{%
5861   \directlua{
5862     Babel.intrapenalties = Babel.intrapenalties or {}
5863     Babel.intrapenalties['\csname bbl@sbc@language\endcsname'] = #1
5864     Babel.locale_props[\the\localeid].intrapenalty = #1
5865   }}
5866 \begingroup
5867 \catcode`\%=12
5868 \catcode`\&=14
5869 \catcode`\'=12
5870 \catcode`\-=12
5871 \gdef\bbl@seaintraspace{&
5872   \let\bbl@seaintraspace\relax
5873   \directlua{
5874     Babel.sea_enabled = true
5875     Babel.sea_ranges = Babel.sea_ranges or {}
5876     function Babel.set_chranges (script, chrng)
5877       local c = 0
5878       for s, e in string.gmatch(chrng..' ', '(.-%.%.(-)%s') do
5879         Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5880         c = c + 1
5881       end
5882     end
5883     function Babel.sea_disc_to_space (head)
5884       local sea_ranges = Babel.sea_ranges
5885       local last_char = nil
5886       local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5887       for item in node.traverse(head) do
5888         local i = item.id
5889         if i == node.id'glyph' then
5890           last_char = item
5891         elseif i == 7 and item.subtype == 3 and last_char
5892           and last_char.char > 0x0C99 then
5893           quad = font.getfont(last_char.font).size
5894           for lg, rg in pairs(sea_ranges) do
5895             if last_char.char > rg[1] and last_char.char < rg[2] then
5896               lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril1
5897               local intraspace = Babel.intraspaces[lg]
5898               local intrapenalty = Babel.intrapenalties[lg]
5899               local n
5900               if intrapenalty ~= 0 then
5901                 n = node.new(14, 0)      &% penalty
5902                 n.penalty = intrapenalty
5903                 node.insert_before(head, item, n)
5904               end
5905               n = node.new(12, 13)      &% (glue, spaceskip)
5906               node.setglue(n, intraspace.b * quad,
5907                 intraspace.p * quad,
5908                 intraspace.m * quad)
```

```

5909         node.insert_before(head, item, n)
5910         node.remove(head, item)
5911     end
5912 end
5913 end
5914 end
5915 end
5916 }&
5917 \bbl@luahyphenate}

```

11.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5918 \catcode`\%=14
5919 \gdef\bbl@cjkintraspacespace{%
5920   \let\bbl@cjkintraspacespace\relax
5921   \directlua{
5922     require('babel-data-cjk.lua')
5923     Babel.cjk_enabled = true
5924     function Babel.cjk_linebreak(head)
5925       local GLYPH = node.id'glyph'
5926       local last_char = nil
5927       local quad = 655360      % 10 pt = 655360 = 10 * 65536
5928       local last_class = nil
5929       local last_lang = nil
5930       for item in node.traverse(head) do
5931         if item.id == GLYPH then
5932           local lang = item.lang
5933           local LOCALE = node.get_attribute(item,
5934             Babel.attr_locale)
5935           local props = Babel.locale_props[LOCALE] or {}
5936           local class = Babel.cjk_class[item.char].c
5937           if props.cjk_quotes and props.cjk_quotes[item.char] then
5938             class = props.cjk_quotes[item.char]
5939           end
5940           if class == 'cp' then class = 'cl' % )] as CL
5941           elseif class == 'id' then class = 'I'
5942           elseif class == 'cj' then class = 'I' % loose
5943           end
5944           local br = 0
5945           if class and last_class and Babel.cjk_breaks[last_class][class] then
5946             br = Babel.cjk_breaks[last_class][class]
5947           end
5948           if br == 1 and props.linebreak == 'c' and
5949             lang ~= \the\l@nohyphenation\space and
5950             last_lang ~= \the\l@nohyphenation then
5951             local intrapenalty = props.intrapenalty
5952             if intrapenalty ~= 0 then
5953               local n = node.new(14, 0)      % penalty
5954               n.penalty = intrapenalty
5955               node.insert_before(head, item, n)
5956             end
5957             local intraspacespace = props.intraspacespace
5958             local n = node.new(12, 13)      % (glue, spaceskip)
5959             node.setglue(n, intraspacespace.b * quad,
5960               intraspacespace.p * quad,
5961               intraspacespace.m * quad)
5962             node.insert_before(head, item, n)

```

```

5963         end
5964         if font.getfont(item.font) then
5965             quad = font.getfont(item.font).size
5966         end
5967         last_class = class
5968         last_lang = lang
5969         else % if penalty, glue or anything else
5970             last_class = nil
5971         end
5972     end
5973     lang.hyphenate(head)
5974 end
5975 }%
5976 \bbl@luahyphenate}
5977 \gdef\bbl@luahyphenate{%
5978 \let\bbl@luahyphenate\relax
5979 \directlua{
5980     luatexbase.add_to_callback('hyphenate',
5981     function (head, tail)
5982         if Babel.linebreaking.before then
5983             for k, func in ipairs(Babel.linebreaking.before) do
5984                 func(head)
5985             end
5986         end
5987         lang.hyphenate(head)
5988         if Babel.cjk_enabled then
5989             Babel.cjk_linebreak(head)
5990         end
5991         if Babel.linebreaking.after then
5992             for k, func in ipairs(Babel.linebreaking.after) do
5993                 func(head)
5994             end
5995         end
5996         if Babel.set_hboxed then
5997             Babel.set_hboxed(head)
5998         end
5999         if Babel.sea_enabled then
6000             Babel.sea_disc_to_space(head)
6001         end
6002     end,
6003     'Babel.hyphenate')
6004 }}
6005 \endgroup
6006 %
6007 \def\bbl@provide@intraspace{%
6008 \bbl@ifunset\bbl@intsp@\languagename}{}%
6009 {\expandafter\ifx\csname bbl@intsp@\languagename\endcsname\@empty\else
6010 \bbl@xin@{/c}{/bbl@c{l}{lnbrk}}}%
6011 \ifin@ % cjk
6012 \bbl@cjk@intraspace
6013 \directlua{
6014     Babel.locale_props = Babel.locale_props or {}
6015     Babel.locale_props[\the\localeid].linebreak = 'c'
6016 }%
6017 \bbl@exp{\bbl@intraspace\bbl@c{l}{intsp}}\@}%
6018 \ifx\bbl@KVP@intrapenalty\@nnil
6019 \bbl@intrapenalty0\@@
6020 \fi
6021 \else % sea
6022 \bbl@sea@intraspace
6023 \bbl@exp{\bbl@intraspace\bbl@c{l}{intsp}}\@}%
6024 \directlua{
6025     Babel.sea_ranges = Babel.sea_ranges or {}

```

```

6026         Babel.set_chranges('\bbl@cl{sbcpr}',
6027                               '\bbl@cl{chrng}')
6028     }%
6029     \ifx\bbl@KVP@intrapenalty\@nnil
6030         \bbl@intrapenalty0\@@
6031     \fi
6032 \fi
6033 \fi
6034 \ifx\bbl@KVP@intrapenalty\@nnil\else
6035     \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
6036 \fi}}

```

11.8. Arabic justification

WIP. \bbl@arabicjust is executed with both elongated and kashida. This must be fine tuned. The attribute kashida is set by transforms with kashida.

```

6037 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
6038 \def\bblar@chars{%
6039     0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
6040     0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
6041     0640,0641,0642,0643,0644,0645,0646,0647,0649}
6042 \def\bblar@elongated{%
6043     0626,0628,062A,062B,0633,0634,0635,0636,063B,%
6044     063C,063D,063E,063F,0641,0642,0643,0644,0646,%
6045     0649,064A}
6046 \begingroup
6047 \catcode\_:=11 \catcode\:=11
6048 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6049 \endgroup
6050 \gdef\bbl@arabicjust{%
6051     \let\bbl@arabicjust\relax
6052     \newattribute\bblar@kashida
6053     \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6054     \bblar@kashida=\z@
6055     \bbl@patchfont{\bbl@parsejalt}}%
6056 \directlua{
6057     Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6058     Babel.arabic.elong_map[\the\localeid] = {}
6059     luatexbase.add_to_callback('post_linebreak_filter',
6060         Babel.arabic.justify, 'Babel.arabic.justify')
6061     luatexbase.add_to_callback('hpack_filter',
6062         Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6063 }}%

```

Save both node lists to make replacement.

```

6064 \def\bblar@fetchjalt#1#2#3#4{%
6065     \bbl@exp{\bbl@foreach{#1}}{%
6066         \bbl@ifunset{bblar@JE@##1}%
6067         {\setbox\z@\hbox{\textdir TRT ^^^200d\char"##1#2}}%
6068         {\setbox\z@\hbox{\textdir TRT ^^^200d\char"\@nameuse{bblar@JE@##1}#2}}%
6069     \directlua{%
6070         local last = nil
6071         for item in node.traverse(tex.box[0].head) do
6072             if item.id == node.id'glyph' and item.char > 0x600 and
6073                 not (item.char == 0x200D) then
6074                 last = item
6075             end
6076         end
6077         Babel.arabic.#3['##1#4'] = last.char
6078     }}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, csw?). What about kaf? And diacritic positioning?

```

6079 \gdef\bbl@parsejalt{%
6080   \ifx\addfontfeature\undefined\else
6081     \bbl@xin@{/e}/{/bbl@cl{lbrk}}}%
6082   \ifin@
6083     \directlua{%
6084       if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
6085         Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6086         tex.print([[string\csname\space bbl@parsejalti\endcsname]])
6087       end
6088     }%
6089   \fi
6090 \fi}
6091 \gdef\bbl@parsejalti{%
6092   \begingroup
6093     \let\bbl@parsejalt\relax % To avoid infinite loop
6094     \edef\bbl@tempb{\fontid\font}%
6095     \bblar@nofswarn
6096     \@nameuse{tracinglostchars}\z@
6097     \bblar@fetchjalt\bblar@elongated{}{from}{}%
6098     \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6099     \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6100     \addfontfeature{RawFeature+=jalt}%
6101     % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf
6102     \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6103     \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6104     \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6105     \directlua{%
6106       for k, v in pairs(Babel.arabic.from) do
6107         if Babel.arabic.dest[k] and
6108            not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6109           Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6110             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6111         end
6112       end
6113     }%
6114   \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6115 \begingroup
6116 \catcode`#=11
6117 \catcode`~=11
6118 \directlua{
6119
6120 Babel.arabic = Babel.arabic or {}
6121 Babel.arabic.from = {}
6122 Babel.arabic.dest = {}
6123 Babel.arabic.justify_factor = 0.95
6124 Babel.arabic.justify_enabled = true
6125 Babel.arabic.kashida_limit = -1
6126
6127 function Babel.arabic.justify(head)
6128   if not Babel.arabic.justify_enabled then return head end
6129   for line in node.traverse_id(node.id'hlist', head) do
6130     Babel.arabic.justify_hlist(head, line)
6131   end
6132   % In case the very first item is a line (eg, in \vbox):
6133   while head.prev do head = head.prev end
6134   return head
6135 end
6136
6137 function Babel.arabic.justify_hbox(head, gc, size, pack)
6138   local has_inf = false
6139   if Babel.arabic.justify_enabled and pack == 'exactly' then

```

```

6140     for n in node.traverse_id(12, head) do
6141         if n.stretch_order > 0 then has_inf = true end
6142     end
6143     if not has_inf then
6144         Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6145     end
6146 end
6147 return head
6148 end
6149
6150 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6151     local d, new
6152     local k_list, k_item, pos_inline
6153     local width, width_new, full, k_curr, wt_pos, goal, shift
6154     local subst_done = false
6155     local elong_map = Babel.arabic.elong_map
6156     local cnt
6157     local last_line
6158     local GLYPH = node.id'glyph'
6159     local KASHIDA = Babel.attr_kashida
6160     local LOCALE = Babel.attr_locale
6161
6162     if line == nil then
6163         line = {}
6164         line.glue_sign = 1
6165         line.glue_order = 0
6166         line.head = head
6167         line.shift = 0
6168         line.width = size
6169     end
6170
6171     % Exclude last line.
6172     if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6173         elongs = {}      % Stores elongated candidates of each line
6174         k_list = {}      % And all letters with kashida
6175         pos_inline = 0   % Not yet used
6176
6177         for n in node.traverse_id(GLYPH, line.head) do
6178             pos_inline = pos_inline + 1 % To find where it is. Not used.
6179
6180             % Elongated glyphs
6181             if elong_map then
6182                 local locale = node.get_attribute(n, LOCALE)
6183                 if elong_map[locale] and elong_map[locale][n.font] and
6184                     elong_map[locale][n.font][n.char] then
6185                     table.insert(elongs, {node = n, locale = locale} )
6186                     node.set_attribute(n.prev, KASHIDA, 0)
6187                 end
6188             end
6189
6190             % Tatwil. First create a list of nodes marked with kashida. The
6191             % rest of nodes can be ignored. The list of used weights is build
6192             % when transforms with the key kashida= are declared.
6193             if Babel.kashida_wts then
6194                 local k_wt = node.get_attribute(n, KASHIDA)
6195                 if k_wt > 0 then % todo. parameter for multi inserts
6196                     table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6197                 end
6198             end
6199
6200         end % of node.traverse_id
6201
6202         if #elongs == 0 and #k_list == 0 then goto next_line end

```

```

6203     full = line.width
6204     shift = line.shift
6205     goal = full * Babel.arabic.justify_factor % A bit crude
6206     width = node.dimensions(line.head) % The 'natural' width
6207
6208     % == Elongated ==
6209     % Original idea taken from 'chickenize'
6210     while (#elongs > 0 and width < goal) do
6211         subst_done = true
6212         local x = #elongs
6213         local curr = elongs[x].node
6214         local oldchar = curr.char
6215         curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6216         width = node.dimensions(line.head) % Check if the line is too wide
6217         % Substitute back if the line would be too wide and break:
6218         if width > goal then
6219             curr.char = oldchar
6220             break
6221         end
6222         % If continue, pop the just substituted node from the list:
6223         table.remove(elongs, x)
6224     end
6225
6226     % == Tatwil ==
6227     % Traverse the kashida node list so many times as required, until
6228     % the line is filled. The first pass adds a tatweel after each
6229     % node with kashida in the line, the second pass adds another one,
6230     % and so on. In each pass, add first the kashida with the highest
6231     % weight, then with lower weight and so on.
6232     if #k_list == 0 then goto next_line end
6233
6234     width = node.dimensions(line.head) % The 'natural' width
6235     k_curr = #k_list % Traverse backwards, from the end
6236     wt_pos = 1
6237
6238     while width < goal do
6239         subst_done = true
6240         k_item = k_list[k_curr].node
6241         if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6242             d = node.copy(k_item)
6243             d.char = 0x0640
6244             d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6245             d.xoffset = 0
6246             line.head, new = node.insert_after(line.head, k_item, d)
6247             width_new = node.dimensions(line.head)
6248             if width > goal or width == width_new then
6249                 node.remove(line.head, new) % Better compute before
6250                 break
6251             end
6252             if Babel.fix_diacr then
6253                 Babel.fix_diacr(k_item.next)
6254             end
6255             width = width_new
6256         end
6257         if k_curr == 1 then
6258             k_curr = #k_list
6259             wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6260         else
6261             k_curr = k_curr - 1
6262         end
6263     end
6264
6265     % Limit the number of tatweel by removing them. Not very efficient,

```

```

6266 % but it does the job in a quite predictable way.
6267 if Babel.arabic.kashida_limit > -1 then
6268   cnt = 0
6269   for n in node.traverse_id(GLYPH, line.head) do
6270     if n.char == 0x0640 then
6271       cnt = cnt + 1
6272       if cnt > Babel.arabic.kashida_limit then
6273         node.remove(line.head, n)
6274       end
6275     else
6276       cnt = 0
6277     end
6278   end
6279 end
6280
6281 ::next_line::
6282
6283 % Must take into account marks and ins, see luatex manual.
6284 % Have to be executed only if there are changes. Investigate
6285 % what's going on exactly.
6286 if subst_done and not gc then
6287   d = node.hpack(line.head, full, 'exactly')
6288   d.shift = shift
6289   node.insert_before(head, line, d)
6290   node.remove(head, line)
6291 end
6292 end % if process line
6293 end
6294 }
6295 \endgroup
6296 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

11.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6297 \def\bbl@scr@node@list{%
6298   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6299   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6300 \ifnum\bbl@bidimode=102 % bidi-r
6301   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6302 \fi
6303 \def\bbl@set@renderer{%
6304   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6305   \ifin@
6306     \let\bbl@unset@renderer\relax
6307   \else
6308     \bbl@exp{%
6309       \def\\bbl@unset@renderer{%
6310         \def<g__fontspec_default_fontopts_clist>{%
6311           \[g__fontspec_default_fontopts_clist]}%
6312         \def<g__fontspec_default_fontopts_clist>{%
6313           Renderer=Harfbuzz,\[g__fontspec_default_fontopts_clist]}%
6314       \fi}
6315 <@Font selection@>

```

11.10. Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the

replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6316 \directlua{% DL6
6317 Babel.script_blocks = {
6318   ['dflt'] = {},
6319   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6320               {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6321   ['Armn'] = {{0x0530, 0x058F}},
6322   ['Beng'] = {{0x0980, 0x09FF}},
6323   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6324   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6325   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6326              {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6327   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6328   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6329              {0xAB00, 0xAB2F}},
6330   ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6331   % Don't follow strictly Unicode, which places some Coptic letters in
6332   % the 'Greek and Coptic' block
6333   ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6334   ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6335              {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6336              {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6337              {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6338              {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6339              {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6340   ['Hebr'] = {{0x0590, 0x05FF},
6341              {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6342   ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6343              {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6344   ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6345   ['Knda'] = {{0x0C80, 0x0CFF}},
6346   ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6347              {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6348              {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6349   ['Lao'] = {{0x0E80, 0x0EFF}},
6350   ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6351              {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6352              {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6353   ['Mahj'] = {{0x11150, 0x1117F}},
6354   ['Mlym'] = {{0x0D00, 0x0D7F}},
6355   ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6356   ['Orya'] = {{0x0B00, 0x0B7F}},
6357   ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6358   ['Syr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6359   ['Taml'] = {{0x0B80, 0x0BFF}},
6360   ['Telu'] = {{0x0C00, 0x0C7F}},
6361   ['Tfng'] = {{0x2D30, 0x2D7F}},
6362   ['Thai'] = {{0x0E00, 0x0E7F}},
6363   ['Tibt'] = {{0x0F00, 0x0FFF}},
6364   ['Vaii'] = {{0xA500, 0xA63F}},
6365   ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6366 }
6367
6368 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6369 Babel.script_blocks.Hant = Babel.script_blocks.Hans

```

```

6370 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6371
6372 function Babel.locale_map(head)
6373   if not Babel.locale_mapped then return head end
6374
6375   local LOCALE = Babel.attr_locale
6376   local GLYPH = node.id('glyph')
6377   local inmath = false
6378   local toloc_save
6379   for item in node.traverse(head) do
6380     local toloc
6381     if not inmath and item.id == GLYPH then
6382       % Optimization: build a table with the chars found
6383       if Babel.chr_to_loc[item.char] then
6384         toloc = Babel.chr_to_loc[item.char]
6385       else
6386         for lc, maps in pairs(Babel.loc_to_scr) do
6387           for rg in pairs(maps) do
6388             if item.char >= rg[1] and item.char <= rg[2] then
6389               Babel.chr_to_loc[item.char] = lc
6390               toloc = lc
6391               break
6392             end
6393           end
6394         end
6395         % Treat composite chars in a different fashion, because they
6396         % 'inherit' the previous locale.
6397         if (item.char >= 0x0300 and item.char <= 0x036F) or
6398            (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6399            (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6400           Babel.chr_to_loc[item.char] = -2000
6401           toloc = -2000
6402         end
6403         if not toloc then
6404           Babel.chr_to_loc[item.char] = -1000
6405         end
6406       end
6407       if toloc == -2000 then
6408         toloc = toloc_save
6409       elseif toloc == -1000 then
6410         toloc = nil
6411       end
6412       if toloc and Babel.locale_props[toloc] and
6413          Babel.locale_props[toloc].letters and
6414          tex.getcatcode(item.char) \string~= 11 then
6415         toloc = nil
6416       end
6417       if toloc and Babel.locale_props[toloc].script
6418          and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6419          and Babel.locale_props[toloc].script ==
6420          Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6421         toloc = nil
6422       end
6423       if toloc then
6424         if Babel.locale_props[toloc].lg then
6425           item.lang = Babel.locale_props[toloc].lg
6426           node.set_attribute(item, LOCALE, toloc)
6427         end
6428         if Babel.locale_props[toloc]['/'..item.font] then
6429           item.font = Babel.locale_props[toloc]['/'..item.font]
6430         end
6431       end
6432       toloc_save = toloc

```

```

6433 elseif not inmath and item.id == 7 then % Apply recursively
6434   item.replace = item.replace and Babel.locale_map(item.replace)
6435   item.pre      = item.pre and Babel.locale_map(item.pre)
6436   item.post     = item.post and Babel.locale_map(item.post)
6437 elseif item.id == node.id'math' then
6438   inmath = (item.subtype == 0)
6439 end
6440 end
6441 return head
6442 end
6443 }

```

The code for `\babelcharproperty` is straightforward. Just note the modified lua table can be different.

```

6444 \newcommand\babelcharproperty[1]{%
6445   \count@=#1\relax
6446   \ifvmode
6447     \expandafter\bbl@chprop
6448   \else
6449     \bbl@error{charproperty-only-vertical}{#1}%
6450   \fi}
6451 \newcommand\bbl@chprop[3][\the\count@]{%
6452   \@tempcnta=#1\relax
6453   \bbl@ifunset{bbl@chprop@#2}% {unknown-char-property}
6454   {\bbl@error{unknown-char-property}{#2}}%
6455   {%
6456     \loop
6457       \bbl@cs{chprop@#2}{#3}%
6458     \ifnum\count@<\@tempcnta
6459       \advance\count@\@ne
6460     \repeat}
6461 %
6462 \def\bbl@chprop@direction#1{%
6463   \directlua{
6464     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6465     Babel.characters[\the\count@]['d'] = '#1'
6466   }}
6467 \let\bbl@chprop@bc\bbl@chprop@direction
6468 %
6469 \def\bbl@chprop@mirror#1{%
6470   \directlua{
6471     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6472     Babel.characters[\the\count@]['m'] = '\number#1'
6473   }}
6474 \let\bbl@chprop@bmg\bbl@chprop@mirror
6475 %
6476 \def\bbl@chprop@linebreak#1{%
6477   \directlua{
6478     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6479     Babel.cjk_characters[\the\count@]['c'] = '#1'
6480   }}
6481 \let\bbl@chprop@lb\bbl@chprop@linebreak
6482 %
6483 \def\bbl@chprop@locale#1{%
6484   \directlua{
6485     Babel.chr_to_loc = Babel.chr_to_loc or {}
6486     Babel.chr_to_loc[\the\count@] =
6487       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6488   }}

```

Post-handling hyphenation patterns for non-standard rules, like `ff` to `ff-f`. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6489 \directlua{% DL7
6490   Babel.nohyphenation = \the\l@nohyphenation

```

6491 }

Now the $\TeX$ high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ becomes `function(m) return m[1]..m[1]..'-' end`, where m are the matches returned after applying the pattern. With a mapped capture the functions are similar to `function(m) return Babel.capt_map(m[1],1) end`, where the last argument identifies the mapping to be applied to $m[1]$. The way it is carried out is somewhat tricky, but the effect is not dissimilar to `lua load` – save the code as string in a $\TeX$ macro, and expand this macro at the appropriate place. As `\directlua` does not take into account the current catcode of `@`, we just avoid this character in macro names (which explains the internal group, too).

```

6492 \begingroup
6493 \catcode\~ =12
6494 \catcode\% =12
6495 \catcode\& =14
6496 \catcode\| =12
6497 \gdef\babelprehyphenation{%&
6498   \ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}[]}]
6499 \gdef\babelposthyphenation{%&
6500   \ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}[]}]
6501 %
6502 \gdef\bbl@settransform#1[#2]#3#4#5{%&
6503   \ifcase#1
6504     \bbl@activateprehyphen
6505   \or
6506     \bbl@activateposthyphen
6507   \fi
6508   \begingroup
6509     \def\babeltempa{\bbl@add@list\babeltempb}%&
6510     \let\babeltempb\empty
6511     \def\bbl@tempa{#5}%&
6512     \bbl@replace\bbl@tempa{,}{,}%& TODO. Ugly trick to preserve {}
6513     \expandafter\bbl@foreach\expandafter{\bbl@tempa}{%&
6514       \bbl@ifsamestring{##1}{remove}%&
6515       {\bbl@add@list\babeltempb{nil}}}%&
6516     {\directlua{
6517       local rep = [=##1]=]
6518       local three_args = '%s*=%s*([%-d%.a{}|]+)%s+([%-d%.a{}|]+)%s+([%-d%.a{}|]+)'
6519       &% Numeric passes directly: kern, penalty...
6520       rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6521       rep = rep:gsub('^%s*(insert)%s*', 'insert = true, ')
6522       rep = rep:gsub('^%s*(after)%s*', 'after = true, ')
6523       rep = rep:gsub('(string)%s*=%s*([%^s,]*)', Babel.capture_func)
6524       rep = rep:gsub('node%s*=%s*([%^a+)%s*([%^a]*)', Babel.capture_node)
6525       rep = rep:gsub(' (norule)' .. three_args,
6526         'norule = {' .. '%2, %3, %4' .. '}')
6527       if #1 == 0 or #1 == 2 then
6528         rep = rep:gsub(' (space)' .. three_args,
6529           'space = {' .. '%2, %3, %4' .. '}')
6530         rep = rep:gsub(' (spacefactor)' .. three_args,
6531           'spacefactor = {' .. '%2, %3, %4' .. '}')
6532         rep = rep:gsub(' (kashida)%s*=%s*([%^s,]*)', Babel.capture_kashida)
6533       &% Transform values
6534       rep, n = rep:gsub(' {[[%a%-%.]+}|([[%a%_%.]+)}',
6535         function(v,d)
6536           return string.format (
6537             '{\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6538             v,
6539             load( 'return Babel.locale_props'..
6540               '[\the\csname bbl@id@@#3\endcsname].' .. d)() )
6541           end )
6542       rep, n = rep:gsub(' {[[%a%-%.]+}|([[%a%-d%.]+)}',
6543         '{\the\csname bbl@id@@#3\endcsname,"%1",%2}')

```

```

6544         end
6545         if #1 == 1 then
6546             rep = rep:gsub( ' (no)%s*=%s*([^\s,]*) ', Babel.capture_func)
6547             rep = rep:gsub( ' (pre)%s*=%s*([^\s,]*) ', Babel.capture_func)
6548             rep = rep:gsub( ' (post)%s*=%s*([^\s,]*) ', Babel.capture_func)
6549         end
6550         tex.print([[string\babeltempa{[] .. rep .. []]])
6551     }&%
6552 \bbl@foreach\babeltempb{&%
6553     \bbl@forkv{###1}{&%
6554         \in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6555             post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6556         \ifin@else
6557             \bbl@error{bad-transform-option}{###1}{}&%
6558         \fi}&%
6559 \let\bbl@kv@attribute\relax
6560 \let\bbl@kv@label\relax
6561 \let\bbl@kv@fonts@empty
6562 \let\bbl@kv@prepend\relax
6563 \bbl@forkv{#2}{\bbl@csarg\edef{kv###1}{##2}}&%
6564 \ifx\bbl@kv@fonts@empty\else\bbl@settransfont\fi
6565 \ifx\bbl@kv@attribute\relax
6566     \ifx\bbl@kv@label\relax\else
6567         \bbl@exp{\bbl@trim\def\bbl@kv@fonts{\bbl@kv@fonts}}&%
6568         \bbl@replace\bbl@kv@fonts{ }{,}&%
6569         \edef\bbl@kv@attribute{\bbl@ATR@\bbl@kv@label @#3@\bbl@kv@fonts}&%
6570         \count@ \z@
6571         \def\bbl@elt##1##2##3{&%
6572             \bbl@ifsamestring{#3,\bbl@kv@label}{##1,##2}&%
6573             {\bbl@ifsamestring{\bbl@kv@fonts}{##3}&%
6574                 {\count@ \@ne}&%
6575                 {\bbl@error{font-conflict-transforms}{}}{}}}&%
6576             {}}&%
6577         \bbl@transfont@list
6578         \ifnum\count@=\z@
6579             \bbl@exp{\global\bbl@add\bbl@transfont@list
6580                 {\bbl@elt{#3}{\bbl@kv@label}{\bbl@kv@fonts}}}&%
6581         \fi
6582         \bbl@ifunset{\bbl@kv@attribute}&%
6583         {\global\bbl@carg\newattribute{\bbl@kv@attribute}}&%
6584         {}&%
6585         \global\bbl@carg\setattribute{\bbl@kv@attribute}\@ne
6586     \fi
6587 \else
6588     \edef\bbl@kv@attribute{\expandafter\bbl@stripslash\bbl@kv@attribute}&%
6589 \fi
6590 \directlua{
6591     local lbr = Babel.linebreaking.replacements[#1]
6592     local u = unicode.utf8
6593     local id, attr, label
6594     if #1 == 0 then
6595         id = \the\csname bbl@id@@#3\endcsname\space
6596     else
6597         id = \the\csname l@#3\endcsname\space
6598     end
6599     \ifx\bbl@kv@attribute\relax
6600         attr = -1
6601     \else
6602         attr = luatexbase.registernumber'\bbl@kv@attribute'
6603     \fi
6604     \ifx\bbl@kv@label\relax\else &% Same refs:
6605         label = [==[\bbl@kv@label]==]
6606     \fi

```

```

6607      &% Convert pattern:
6608      local patt = string.gsub([==[#4]==], '%s', '')
6609      if #1 == 0 then
6610          patt = string.gsub(patt, '|', ' ')
6611      end
6612      if not u.find(patt, '()', nil, true) then
6613          patt = '()' .. patt .. '()'
6614      end
6615      patt = string.gsub(patt, '%(%)^', '^()')
6616      patt = string.gsub(patt, '%$%(%)', '()$')
6617      patt = u.gsub(patt, '{(.)}',
6618          function (n)
6619              return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6620          end)
6621      patt = u.gsub(patt, '{(%x%x%x%x+)}',
6622          function (n)
6623              return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6624          end)
6625      lbkr[id] = lbkr[id] or {}
6626      table.insert(lbkr[id], \ifx\bbl@kv@prepend\relax\else 1,\fi
6627          { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6628      }&%
6629      \endgroup}
6630 \endgroup
6631 %
6632 \let\bbl@transfont@list@empty
6633 \def\bbl@settransfont{%
6634     \global\let\bbl@settransfont\relax % Execute only once
6635     \gdef\bbl@transfont{%
6636         \def\bbl@elt####1####2####3{%
6637             \bbl@ifblank{####3}%
6638             {\count@tw@}% Do nothing if no fonts
6639             {\count@z@
6640             \bbl@vforeach{####3}{%
6641                 \def\bbl@tempd{#####1}%
6642                 \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6643                 \ifx\bbl@tempd\bbl@tempe
6644                     \count@\@ne
6645                 \else\ifx\bbl@tempd\bbl@transfam
6646                     \count@\@ne
6647                 \fi\fi}%
6648             \ifcase\count@
6649                 \bbl@csarg\unsetattribute{ATR@####2@####1@####3}%
6650             \or
6651                 \bbl@csarg\setattribute{ATR@####2@####1@####3}\@ne
6652             \fi}}%
6653             \bbl@transfont@list}%
6654     \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6655     \gdef\bbl@transfam{-unknown-}%
6656     \bbl@foreach\bbl@font@fams{%
6657         \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6658         \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6659         {\xdef\bbl@transfam{##1}}%
6660     }}}}
6661 %
6662 \DeclareRobustCommand\enablelocaletransform[1]{%
6663     \bbl@ifunset\bbl@ATR@#1@\language @}%
6664     {\bbl@error{transform-not-available}{#1}}}%
6665     {\bbl@csarg\setattribute{ATR@#1@\language @}\@ne}}
6666 \DeclareRobustCommand\disablelocaletransform[1]{%
6667     \bbl@ifunset\bbl@ATR@#1@\language @}%
6668     {\bbl@error{transform-not-available-b}{#1}}}%
6669     {\bbl@csarg\unsetattribute{ATR@#1@\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6670 \def\bbl@activateposthyphen{%
6671   \let\bbl@activateposthyphen\relax
6672   \ifx\bbl@attr@hboxed\undefined
6673     \newattribute\bbl@attr@hboxed
6674   \fi
6675   \directlua{
6676     require('babel-transforms.lua')
6677     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6678   }}
6679 \def\bbl@activateprehyphen{%
6680   \let\bbl@activateprehyphen\relax
6681   \ifx\bbl@attr@hboxed\undefined
6682     \newattribute\bbl@attr@hboxed
6683   \fi
6684   \directlua{
6685     require('babel-transforms.lua')
6686     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6687   }}
6688 \newcommand\SetTransformValue[3]{%
6689   \directlua{
6690     Babel.locale_props[\the\csname bbl@id@@#1\endcsname].vars["#2"] = #3
6691   }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6692 \newcommand\ShowBabelTransforms[1]{%
6693   \bbl@activateprehyphen
6694   \bbl@activateposthyphen
6695   \begingroup
6696     \directlua{ Babel.show_transforms = true }%
6697     \setbox\z@\vbox{#1}%
6698     \directlua{ Babel.show_transforms = false }%
6699   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]==]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6700 \newcommand\localeprehyphenation[1]{%
6701   \directlua{ Babel.string_prehyphenation([==[#1]==], \the\localeid) }}

```

11.11.Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```

6702 \AtBeginDocument{\pagedir\bodydir}

```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by $\TeX$. Just in case, consider the possibility it has not been loaded.

```

6703 \def\bbl@activate@preotf{%
6704   \let\bbl@activate@preotf\relax % only once
6705   \directlua{
6706     function Babel.pre_otfload_v(head)
6707       if Babel.numbers and Babel.digits_mapped then
6708         head = Babel.numbers(head)
6709       end
6710       if Babel.bidi_enabled then
6711         head = Babel.bidi(head, false, dir)

```

```

6712     end
6713     return head
6714 end
6715 %
6716 function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6717     if Babel.numbers and Babel.digits_mapped then
6718         head = Babel.numbers(head)
6719     end
6720     if Babel.bidi_enabled then
6721         head = Babel.bidi(head, false, dir)
6722     end
6723     return head
6724 end
6725 %
6726 luatexbase.add_to_callback('pre_linebreak_filter',
6727     Babel.pre_otfload_v,
6728     'Babel.pre_otfload_v',
6729     Babel.priority_in_callback('pre_linebreak_filter',
6730         'luaotfload.node_processor') or nil)
6731 %
6732 luatexbase.add_to_callback('hpack_filter',
6733     Babel.pre_otfload_h,
6734     'Babel.pre_otfload_h',
6735     Babel.priority_in_callback('hpack_filter',
6736         'luaotfload.node_processor') or nil)
6737 }}

```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in `basic-r`.

```

6738 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6739 \let\bbl@beforeforeign\leavevmode
6740 \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6741 \ifnum\bbl@bidimode>300
6742 \RequirePackage{unibidi-lua}
6743 \else
6744 \RequirePackage{luatexbase}
6745 \bbl@activate@preotf
6746 \directlua{
6747     require('babel-data-bidi.lua')
6748     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6749         require('babel-bidi-basic.lua')
6750     \or
6751         require('babel-bidi-basic-r.lua')
6752         table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6753         table.insert(Babel.ranges, {0xF000, 0xFFFFD, 'on'})
6754         table.insert(Babel.ranges, {0x10000, 0x10FFFD, 'on'})
6755     \fi}
6756 \fi
6757 \newattribute\bbl@attr@dir
6758 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6759 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6760 \fi
6761 %
6762 \chardef\bbl@thetextdir\z@
6763 \chardef\bbl@thepardir\z@
6764 \def\bbl@setluadir#1#2{% 1=\text/pardirection 2= 0:l 1:r 2:al
6765     \ifcase#2\relax
6766         \ifcase#1\else#1=\z@\fi
6767     \else
6768         \ifcase#1#1=\@ne\fi
6769     \fi}

```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (`PP` is the `par dir`) and `0x3` (`TT` is the `text dir`). These macro names are shared by the 3 engines, with different definitions.

```

6770 \def\bbl@thedir{0}
6771 \def\bbl@textdir#1{%
6772   \bbl@setluadir\textdirection{#1}%
6773   \chardef\bbl@thetextdir#1\relax
6774   \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6775   \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6776 \def\bbl@pardir#1{% Used twice
6777   \bbl@setluadir\pardirection{#1}%
6778   \chardef\bbl@thepardir#1\relax}
6779 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6780 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

```

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to ‘`tabular`’, which is based on a fake math.

```

6781 \ifnum\bbl@bidimode>\z@ % Any bidi=
6782   \def\bbl@insidemath{0}%
6783   \def\bbl@everymath{\def\bbl@insidemath{1}}
6784   \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6785   \frozen@everymath\expandafter{%
6786     \expandafter\bbl@everymath\the\frozen@everymath}
6787   \frozen@everydisplay\expandafter{%
6788     \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6789   \AtBeginDocument{
6790     \directlua{
6791       function Babel.math_box_dir(head)
6792         if not (token.get_macro('bbl@insidemath') == '0') then
6793           if Babel.hlist_has_bidi(head) then
6794             local d = node.new(node.id'dir')
6795             d.dir = '+TRT'
6796             for item in node.traverse(head) do
6797               if item.id == 11 or item.id == node.id'glyph' then
6798                 head = node.insert_before(head, item, d)
6799                 break
6800             end
6801           end
6802           local inmath = false
6803           for item in node.traverse(head) do
6804             if item.id == 11 then
6805               inmath = (item.subtype == 0)
6806             elseif not inmath then
6807               node.set_attribute(item,
6808                 Babel.attr_dir, token.get_macro('bbl@thedir'))
6809             end
6810           end
6811         end
6812       end
6813       return head
6814     end
6815     luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6816       "Babel.math_box_dir", 0)
6817     if Babel.unset_atdir then
6818       luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6819         "Babel.unset_atdir")
6820       luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6821         "Babel.unset_atdir")
6822     end
6823     luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6824       local dir = tex.mathdirection
6825       local n = node.new("dir")
6826       n.direction = dir
6827       head = node.insert_before(head,head,n)

```

```

6828         n = node.new("dir",1)
6829         n.direction = dir
6830         head = node.insert_after(head,node.tail(head),n)
6831         return head
6832     end,"Babel.ensure_math_dir")
6833 }}%
6834 \fi

```

Experimental. Tentative name.

```

6835 \DeclareRobustCommand\localebox[1]{%
6836   {\def\bbl@insidemath{0}%
6837     \mbox{\foreignlanguage{\language}{#1}}}}

```

11.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidibasic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I’ve made some progress in graphics, but they’re essentially hacks; I’ve also made some progress in ‘tabular’, but when I decided to tackle math (both standard math and ‘amsmath’) the nightmare began. I’m still not sure how ‘amsmath’ should be modified, but the main problem is that, boxes are “generic” containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with ‘math’ (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hhline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolumn` still fails.

```

6838 \bbl@trace{Redefinitions for bidi layout}
6839 %
6840 ({*More package options}) ≡
6841 \chardef\bbl@eqnpos\z@
6842 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6843 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6844 ({/More package options})

```

Fixes in luatex are sometimes done with flags. We first activate all all them.

```

6845 \ifnum\bbl@bidimode>\z@ % Any bidi=
6846   \matheqdirmode\@ne
6847   \matheptydisplaymode\@ne
6848   \breakafterdirmode\@ne
6849   \fixupboxesmode\@ne
6850   \let\bbl@eqnudir\relax
6851   \def\bbl@eqdel{()}
6852   \def\bbl@eqnum{%
6853     {\normalfont\normalcolor
6854       \expandafter\@firstoftwo\bbl@eqdel
6855       \theequation
6856       \expandafter\@secondoftwo\bbl@eqdel}}
6857   \def\bbl@puteqno#1{\eqno\hbox{#1}}
6858   \def\bbl@putleqno#1{\leqno\hbox{#1}}
6859   \def\bbl@eqno@flip#1{% So
6860     \ifdim\predisplaywidth<\maxdimen % For consecutive displays
6861       \eqno
6862       \hb@xt@.01pt{%
6863         \hb@xt@\displaywidth{\hss#1\glet\bbl@upset\@currentlabel}}\hss}%
6864     \else

```

```

6865 \leqno\hbox{#1\glet\bb@upset\@currentlabel}%
6866 \fi
6867 \bb@exp{\def\\@currentlabel{\bb@upset}}}%
6868 \def\bb@leqno@flip#1{%
6869 \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6870 \leqno
6871 \hb@xt@.01pt{%
6872 \hss\hb@xt@\displaywidth{#1\glet\bb@upset\@currentlabel}\hss}}%
6873 \else
6874 \eqno\hbox{#1\glet\bb@upset\@currentlabel}%
6875 \fi
6876 \bb@exp{\def\\@currentlabel{\bb@upset}}}%
6877 %
6878 \AtBeginDocument{%
6879 \ifx\bb@noamsmath\relax\else
6880 \ifx\maketag@@@undefined % Normal equation, eqnarray
6881 \ifnum\bb@eqnpos=\tw@
6882 \bb@replace\equation{\hb@xt@\linewidth}%
6883 {\hbox bdir\mathdirection to\linewidth}%
6884 \bb@carg\bb@sreplace{[ ]}{\hb@xt@\linewidth}%
6885 {\hbox bdir\mathdirection to\linewidth}%
6886 \fi
6887 \AddToHook{env/equation/begin}{%
6888 \ifnum\bb@thetextdir>z@
6889 \let\@eqnnum\bb@eqnum
6890 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6891 \chardef\bb@thetextdirz@
6892 \bb@add\normalfont{\bb@eqnodir}%
6893 \ifcase\bb@eqnpos
6894 \let\bb@puteqno\bb@eqno@flip
6895 \or
6896 \let\bb@puteqno\bb@leqno@flip
6897 \fi
6898 \fi}%
6899 \ifnum\bb@eqnpos=\tw@\else
6900 \def\endequation{\bb@puteqno{\@eqnnum}$$\@ignoretrue}%
6901 \fi
6902 \AddToHook{env/eqnarray/begin}{%
6903 \ifnum\bb@thetextdir>z@
6904 \edef\bb@eqnodir{\noexpand\bb@textdir{\the\bb@thetextdir}}%
6905 \chardef\bb@thetextdirz@
6906 \bb@add\normalfont{\bb@eqnodir}%
6907 \ifnum\bb@eqnpos=\@ne
6908 \def\@eqnnum{%
6909 \setboxz@hbox{\bb@eqnum}%
6910 \hbox to0.01pt{\hss\hbox to\displaywidth{\boxz@\hss}}}%
6911 \else
6912 \let\@eqnnum\bb@eqnum
6913 \fi
6914 \fi}
6915 \else % amstex
6916 \bb@exp{% Hack to hide maybe undefined conditionals:
6917 \chardef\bb@eqnpos=0<iftagsleft>1<else><if@fleqn>2<fi><fi>\relax}%
6918 \ifnum\bb@eqnpos=\@ne
6919 \let\bb@ams@lap\hbox
6920 \else
6921 \let\bb@ams@lap\llap
6922 \fi
6923 \ExplSyntaxOn % Required by \bb@sreplace with \intertext@
6924 \bb@sreplace\intertext@{\normalbaselines}%
6925 {\normalbaselines
6926 \ifx\bb@eqnodir\relax\else\bb@pardir\@ne\bb@eqnodir\fi}%
6927 \ExplSyntaxOff

```

```

6928 \def\bbl@ams@tagbox#1#2{#1{\bbl@eqnodir#2}}% #1=hbox|@lap|flip
6929 \ifx\bbl@ams@lap\hbox % leqno
6930 \def\bbl@ams@flip#1{%
6931 \hbox to 0.01pt{\hss\hbox to\displaywidth{#{1}\hss}}}%
6932 \else % eqno
6933 \def\bbl@ams@flip#1{%
6934 \hbox to 0.01pt{\hbox to\displaywidth{\hss{#1}\hss}}}%
6935 \fi
6936 \def\bbl@ams@preset#1{%
6937 \ifnum\bbl@thetextdir>\z@
6938 \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6939 \bbl@sreplace\textdef@{\hbox}{\bbl@ams@tagbox\hbox}%
6940 \bbl@sreplace\maketag@@@{\hbox}{\bbl@ams@tagbox#1}%
6941 \fi}%
6942 \def\bbl@ams@equation{%
6943 \ifnum\bbl@thetextdir>\z@
6944 \bbl@ams@preset\hbox
6945 \chardef\bbl@thetextdir\z@
6946 \bbl@add\normalfont{\bbl@eqnodir}%
6947 \ifcase\bbl@eqnpos
6948 \def\veqno##1##2{\bbl@eqno@flip{##1##2}}%
6949 \or
6950 \def\veqno##1##2{\bbl@leqno@flip{##1##2}}%
6951 \fi
6952 \fi}%
6953 \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6954 \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6955 \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6956 \AddToHook{env/multline/begin}{\bbl@ams@preset\hbox}%
6957 \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6958 \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6959 \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6960 \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6961 \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6962 \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6963 \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\hbox}%
6964 % Hackish, for proper alignment. Don't ask me why it works!:
6965 \bbl@exp{% Avoid a 'visible' conditional
6966 \\\AddToHook{env/align*/end}{\<iftag@>\<else>\\tag*{\<fi>}}%
6967 \\\AddToHook{env/alignat*/end}{\<iftag@>\<else>\\tag*{\<fi>}}}%
6968 \AddToHook{env/flalign/begin}{\bbl@ams@preset\hbox}%
6969 \AddToHook{env/split/before}{%
6970 \ifnum\bbl@thetextdir>\z@
6971 \bbl@ifsamestring\@currentvir{equation}%
6972 {\ifx\bbl@ams@lap\hbox % leqno
6973 \def\bbl@ams@flip#1{%
6974 \hbox to 0.01pt{\hbox to\displaywidth{#{1}\hss}\hss}}}%
6975 \else
6976 \def\bbl@ams@flip#1{%
6977 \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}}}%
6978 \fi}%
6979 {}%
6980 \fi}%
6981 \fi\fi}
6982 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6983 \def\bbl@provide@extra#1{%
6984 % == onchar ==
6985 \ifx\bbl@KVP@onchar\@nnil\else
6986 \bbl@luahyphenate
6987 \bbl@exp{%
6988 \\\AddToHook{env/document/before}{%

```

```

6989         {\let\\bbl@ifrestoring\\@firstoftwo
6990         \\select@language{#1}{}}}%
6991 \directlua{
6992     if Babel.locale_mapped == nil then
6993         Babel.locale_mapped = true
6994         Babel.linebreaking.add_before(Babel.locale_map, 1)
6995         Babel.loc_to_scr = {}
6996         Babel.chr_to_loc = Babel.chr_to_loc or {}
6997     end
6998     Babel.locale_props[\the\localeid].letters = false
6999 }%
7000 \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
7001 \ifin@
7002     \directlua{
7003         Babel.locale_props[\the\localeid].letters = true
7004     }%
7005 \fi
7006 \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
7007 \ifin@
7008     \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
7009         \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}%
7010     \fi
7011     \bbl@exp{\bbl@add\bbl@starthyphens
7012     {\bbl@patterns@lua{\language}}}%
7013     \directlua{
7014         if Babel.script_blocks['\bbl@cl{sbc}'] then
7015             Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
7016             Babel.locale_props[\the\localeid].lg = \the\@nameuse{l@\language}\space
7017         end
7018     }%
7019 \fi
7020 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
7021 \ifin@
7022     \bbl@ifunset{bbl@lsys@\language}{\bbl@provide@lsys{\language}}}%
7023     \bbl@ifunset{bbl@wdir@\language}{\bbl@provide@dirs{\language}}}%
7024     \directlua{
7025         if Babel.script_blocks['\bbl@cl{sbc}'] then
7026             Babel.loc_to_scr[\the\localeid] =
7027             Babel.script_blocks['\bbl@cl{sbc}']
7028         end}%
7029     \ifx\bbl@mapselect\@undefined
7030         \AtBeginDocument{%
7031             \bbl@patchfont{\bbl@mapselect}}%
7032             {\selectfont}}%
7033         \def\bbl@mapselect{%
7034             \let\bbl@mapselect\relax
7035             \edef\bbl@prefontid{\fontid\font}}%
7036         \def\bbl@mapdir##1{%
7037             \begingroup
7038                 \setbox\z@\hbox{% Force text mode
7039                 \def\language{##1}%
7040                 \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
7041                 \bbl@switchfont
7042                 \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
7043                     \directlua{
7044                         Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
7045                         ['/bbl@prefontid'] = \fontid\font\space}%
7046                     \fi}%
7047                 \endgroup}%
7048     \fi
7049     \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\language}}}%
7050 \fi
7051 \fi

```

```

7052 % == mapfont ==
7053 % For bidi texts, to switch the font based on direction. Deprecated
7054 \ifx\bbk@KVP@mapfont\@nnil\else
7055   \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}}}%
7056   {\bbl@error{unknown-mapfont}}{}{}{}%
7057   \bbl@ifunset{\bbl@lsys@language}{\bbl@provide@lsys{language}}}%
7058   \bbl@ifunset{\bbl@wdir@language}{\bbl@provide@dirs{language}}}%
7059   \ifx\bbl@mapselect\undefined
7060     \AtBeginDocument{%
7061       \bbl@patchfont{\bbl@mapselect}}%
7062       {\selectfont}}%
7063     \def\bbl@mapselect{%
7064       \let\bbl@mapselect\relax
7065       \edef\bbl@prefontid{\fontid\font}}%
7066     \def\bbl@mapdir##1{%
7067       {\def\language{##1}%
7068         \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7069         \bbl@switchfont
7070         \directlua{Babel.fontmap
7071           [\the\csname bbl@wdir@##1\endcsname]%
7072           [\bbl@prefontid]=\fontid\font}}}%
7073     \fi
7074     \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{language}}}%
7075   \fi
7076 % == Line breaking: CJK quotes ==
7077 \ifcase\bbl@engine\or
7078   \bbl@xin@{/c}{/\bbl@cl{lnbrk}}%
7079   \ifin@
7080     \bbl@ifunset{\bbl@quote@language}{}%
7081     {\directlua{
7082       Babel.locale_props[\the\localeid].cjk_quotes = {}
7083       local cs = 'op'
7084       for c in string.utfvalues(
7085         [[\csname bbl@quote@language\endcsname]]) do
7086         if Babel.cjk_characters[c].c == 'qu' then
7087           Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7088         end
7089         cs = ( cs == 'op') and 'cl' or 'op'
7090       end
7091     }}%
7092   \fi
7093 \fi
7094 % == Counters: mapdigits ==
7095 % Native digits
7096 \ifx\bbk@KVP@mapdigits\@nnil\else
7097   \bbl@ifunset{\bbl@dgnat@language}{}%
7098   {\bbl@activate@preotf
7099     \directlua{
7100       Babel.digits_mapped = true
7101       Babel.digits = Babel.digits or {}
7102       Babel.digits[\the\localeid] =
7103         table.pack(string.utfvalue('\bbl@cl{dgnat}'))
7104       if not Babel.numbers then
7105         function Babel.numbers(head)
7106           local LOCALE = Babel.attr_locale
7107           local GLYPH = node.id'glyph'
7108           local inmath = false
7109           for item in node.traverse(head) do
7110             if not inmath and item.id == GLYPH then
7111               local temp = node.get_attribute(item, LOCALE)
7112               if Babel.digits[temp] then
7113                 local chr = item.char
7114                 if chr > 47 and chr < 58 then

```

```

7115             item.char = Babel.digits[temp][chr-47]
7116         end
7117     end
7118     elseif item.id == node.id'math' then
7119         inmath = (item.subtype == 0)
7120     end
7121 end
7122 return head
7123 end
7124 end
7125 }}%
7126 \fi
7127 % == transforms ==
7128 \ifx\bbbl@KVP@transforms\@nnil\else
7129   \def\bbbl@elt##1##2##3{%
7130     \in@{$transforms.}{##1}%
7131     \ifin@
7132       \def\bbbl@tempa{##1}%
7133       \bbbl@replace\bbbl@tempa{transforms.}{}%
7134       \bbbl@carg\bbbl@transforms{babel\bbbl@tempa}{##2}{##3}%
7135     \fi}%
7136 \bbbl@exp{%
7137   \\bbbl@ifblank{\bbbl@cl{dgnat}}}%
7138   {\let\\bbbl@tempa\relax}%
7139   {\def\\bbbl@tempa{%
7140     \\bbbl@elt{transforms.prehyphenation}%
7141     {digits.native.1.0}{([0-9])}%
7142     \\bbbl@elt{transforms.prehyphenation}%
7143     {digits.native.1.1}{string={\string|0123456789\string|\bbbl@cl{dgnat}}}}}%
7144 \ifx\bbbl@tempa\relax\else
7145   \toks@{\expandafter\expandafter\expandafter{%
7146     \csname bbl@inidata@\language\endcsname}%
7147     \bbbl@carg\edef{inidata@\language}{%
7148       \unexpanded\expandafter{\bbbl@tempa}%
7149       \the\toks@}%
7150   \fi
7151   \csname bbl@inidata@\language\endcsname
7152   \bbbl@release@transforms\relax % \relax closes the last item.
7153 \fi}

```

Start tabular here:

```

7154 \def\localerestoredirs{%
7155   \ifcase\bbbl@thetextdir
7156     \ifnum\textdirection=\z@\else\textdirection=\z@\fi
7157   \else
7158     \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi
7159   \fi
7160   \ifcase\bbbl@thepardir
7161     \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7162   \else
7163     \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7164   \fi}
7165 %
7166 \IfBabelLayout{tabular}%
7167   {\chardef\bbbl@tabular@mode\z@}% All RTL
7168   {\IfBabelLayout{lrtabular}%
7169     {\chardef\bbbl@tabular@mode\@ne}%
7170     {\chardef\bbbl@tabular@mode\z@}}% Mixed, with LTR cols
7171 %
7172 \ifnum\bbbl@bidimode>\@ne % Any lua bidi= except default=1
7173   % Redefine: vrules mess up dirs (why?).
7174   \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7175   \ifcase\bbbl@tabular@mode\else % 1 = Mixed

```

```

7176 \let\bbl@parabefore\relax
7177 \AddToHook{para/before}{\bbl@parabefore}
7178 \AtBeginDocument{%
7179   \bbl@replace@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7180   \ifnum\bbl@tabular@mode=\@ne
7181     \bbl@ifunset{@tabclassz}{\}%
7182     \bbl@exp{% Hide conditionals
7183       \\bbl@sreplace\\@tabclassz
7184         {\<ifcase>\\@chnum}%
7185         {\\localerestoredirs\<ifcase>\\@chnum}}}%
7186   \@ifpackageloaded{colortbl}%
7187     {\bbl@sreplace@classz
7188       {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7189   {\@ifpackageloaded{array}%
7190     {\bbl@exp{% Hide conditionals
7191       \\bbl@sreplace\\@classz
7192         {\<ifcase>\\@chnum}%
7193         {\bgroup\\localerestoredirs\<ifcase>\\@chnum}%
7194         \\bbl@sreplace\\@classz
7195         {\\do@row@strut\<fi>}{\\do@row@strut\<fi>\egroup}}}%
7196     {}}%
7197   \fi}%
7198 \fi

```

Very likely the `\output` routine must be patched in a quite general way to make sure the `\bodydir` is set to `\pagedir`. Note outside `\output` they can be different (and often are). For the moment, two *ad hoc* changes.

```

7199 \AtBeginDocument{%
7200   \@ifpackageloaded{multicol}%
7201     {\toks@expandafter{\multi@column@out}%
7202      \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7203     {}%
7204   \@ifpackageloaded{paracol}%
7205     {\edef\pcol@output{%
7206       \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7207     {}%
7208 \fi

```

Finish here if there in no layout.

```
7209 \ifx\bbl@opt@layout\@nnil\endinput\fi
```

`\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7210 \chardef\bbl@trace@vboxdir\z@
7211 \ifnum\bbl@bidimode>\z@ % Any bidi=
7212   \IfBabelLayout{nopars}{}
7213   {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7214   \IfBabelLayout{pars}
7215   {\chardef\bbl@trace@vboxdir\@ne
7216    \def\@hangfrom#1{%
7217      \setbox\@tempboxa\hbox{#1}%
7218      \hangindent\wd\@tempboxa
7219      \ifnum\bbl@vbox@bodydir=\pardirection\else
7220        \shapemode\@ne
7221        \fi
7222      \noindent\box\@tempboxa}}
7223   {}
7224 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7225 \IfBabelLayout{lists}
7226   {\chardef\bbl@trace@vboxdir\@ne
7227    \let\bbl@0L@list\list
7228    \bbl@sreplace\list{\parshape}{\bbl@listparshape}%

```

```

7229 \let\bbl@NL@list\list
7230 \def\bbl@listparshape#1#2#3{%
7231   \parshape #1 #2 #3 %
7232   \ifnum\bbl@vbox@bodydir=\pardirection\else
7233     \shapemode\tw@
7234   \fi}}
7235 {}
7236 %
7237 \ifcase\bbl@trace@vboxdir\else
7238 \AtBeginDocument{\chardef\bbl@vbox@bodydir\pagedirection}%
7239 \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7240 \let\bbl@bidi@vbox\everyvbox
7241 \@nameuse{newtoks}\everyvbox % \outer in Plain
7242 \everyvbox\expandafter{\the\bbl@bidi@vbox}%
7243 \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7244 \fi
7245 %
7246 \IfBabelLayout{graphics}
7247 {\let\bbl@pictresetdir\relax
7248 \def\bbl@pictsetdir#1{%
7249   \ifcase\bbl@thetextdir
7250     \let\bbl@pictresetdir\relax
7251   \else
7252     \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7253     \or\textdir TLT
7254     \else\bodydir TLT \textdir TLT
7255   \fi
7256   % \text\par\dir required in pgf:
7257   \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7258 \fi}%
7259 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7260 \directlua{
7261   Babel.get_picture_dir = true
7262   Babel.picture_has_bidi = 0
7263   %
7264   function Babel.picture_dir (head)
7265     if not Babel.get_picture_dir then return head end
7266     if Babel.hlist_has_bidi(head) then
7267       Babel.picture_has_bidi = 1
7268     end
7269     return head
7270   end
7271   luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7272     "Babel.picture_dir")
7273 }%
7274 \AddToHook{package/graphics/after}{%
7275   \bbl@exp{%
7276     \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7277     {\hbox bdir\z@{\hbox bdir<\ifcase>\bbl@thetextdir\z@<\else>\@ne<\fi>{\string#2}}}}
7278 \AddToHook{package/graphicx/after}{%
7279   \bbl@exp{%
7280     \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7281     {\hbox bdir\z@{\hbox bdir<\ifcase>\bbl@thetextdir\z@<\else>\@ne<\fi>{\string#2}}}%
7282     \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7283     {\hbox bdir\z@{\hbox bdir<\ifcase>\bbl@thetextdir\z@<\else>\@ne<\fi>{\string#3}}}%
7284     \\bbl@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}
7285 \AtBeginDocument{%
7286   \long\def\put(#1,#2)#3{%
7287     \@killglue
7288     % Try:
7289     \ifx\bbl@pictresetdir\relax
7290       \def\bbl@tempc{0}%
7291     \else

```

```

7292 \directlua{
7293   Babel.get_picture_dir = true
7294   Babel.picture_has_bidi = 0
7295 }%
7296 \setbox\z@\hb@xt@\z@{%
7297   \@defaultunitsset\@tempdimc{#1}\unitlength
7298   \kern\@tempdimc
7299   #3\hss}%
7300 \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7301 \fi
7302 % Do:
7303 \@defaultunitsset\@tempdimc{#2}\unitlength
7304 \raise\@tempdimc\hb@xt@\z@{%
7305   \@defaultunitsset\@tempdimc{#1}\unitlength
7306   \kern\@tempdimc
7307   {\ifnum\bbl@tempc>\z@\bbl@pictresetdir\fi#3}\hss}%
7308 \ignorespaces}%
7309 \MakeRobust\put}%
7310 \AtBeginDocument
7311 {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7312 \ifx\pgfpicture\undefined\else
7313   \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7314   \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7315   \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7316 \fi
7317 \ifx\tikzpicture\undefined\else
7318   \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw@}%
7319   \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7320   \bbl@sreplace\tikz{\begingroup}{\begingroup\bbl@pictsetdir\tw@}%
7321   \bbl@sreplace\tikzpicture{\begingroup}{\begingroup\bbl@pictsetdir\tw@}%
7322 \fi
7323 }}
7324 {}

```

Implicitly reverses sectioning labels in bidi=basic-r, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes bidi=basic, but there are some additional readjustments for bidi=default.

```

7325 \IfBabelLayout{counters*}%
7326 {\bbl@add\bbl@opt@layout{.counters.}%
7327 \directlua{
7328   luatexbase.add_to_callback("process_output_buffer",
7329     Babel.discard_sublr , "Babel.discard_sublr") }%
7330 {}%
7331 \IfBabelLayout{counters}%
7332 {\let\bbl@latinarabic=\@arabic
7333 \let\bbl@0L@arabic=\@arabic
7334 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7335 \@ifpackagewith{babel}{bidi=default}%
7336 {\let\bbl@asciroman=\@roman
7337 \let\bbl@0L@roman=\@roman
7338 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7339 \let\bbl@asciiRoman=\@Roman
7340 \let\bbl@0L@roman=\@Roman
7341 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7342 \let\bbl@0L@labelenumii\labelenumii
7343 \def\labelenumii{}\theenumii}%
7344 \let\bbl@0L@p@enumiii\p@enumiii
7345 \def\p@enumiii{\p@enumii}\theenumii}{}}{}%

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7346 \IfBabelLayout{extras}%
7347 {\bbl@ncarg\let\bbl@0L@underline{underline }%
7348 \bbl@carg\bbl@sreplace{underline }{\hbox}%

```

```

7349     {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7350     \let\bbl@0L@LaTeXe\LaTeXe
7351     \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7352       \if b\expandafter\@car\@fseries\@nil\boldmath\fi
7353       \babelsublr{\LaTeX\kern.15em2$_{\textstyle\varepsilon}$}}}
7354   {}
7355 (/luatex)

```

11.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7356 (*transforms)
7357 Babel.linebreaking.replacements = {}
7358 Babel.linebreaking.replacements[0] = {} -- pre
7359 Babel.linebreaking.replacements[1] = {} -- post
7360
7361 function Babel.tovalue(v)
7362   if type(v) == 'table' then
7363     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7364   else
7365     return v
7366   end
7367 end
7368
7369 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7370
7371 function Babel.set_hboxed(head, gc)
7372   for item in node.traverse(head) do
7373     node.set_attribute(item, Babel.attr_hboxed, 1)
7374   end
7375   return head
7376 end
7377
7378 Babel.fetch_subtext = {}
7379
7380 Babel.ignore_pre_char = function(node)
7381   return (node.lang == Babel.nohyphenation)
7382 end
7383
7384 Babel.show_transforms = false
7385
7386 -- Merging both functions doesn't seem feasible, because there are too
7387 -- many differences.
7388 Babel.fetch_subtext[0] = function(head)
7389   local word_string = ''
7390   local word_nodes = {}
7391   local lang
7392   local item = head
7393   local inmath = false
7394
7395   while item do
7396     if item.id == 11 then

```

```

7398     inmath = (item.subtype == 0)
7399 end
7400
7401 if inmath then
7402     -- pass
7403
7404 elseif item.id == 29 then
7405     local locale = node.get_attribute(item, Babel.attr_locale)
7406
7407     if lang == locale or lang == nil then
7408         lang = lang or locale
7409         if Babel.ignore_pre_char(item) then
7410             word_string = word_string .. Babel.us_char
7411         else
7412             if node.has_attribute(item, Babel.attr_hboxed) then
7413                 word_string = word_string .. Babel.us_char
7414             else
7415                 word_string = word_string .. unicode.utf8.char(item.char)
7416             end
7417         end
7418         word_nodes[#word_nodes+1] = item
7419     else
7420         break
7421     end
7422
7423 elseif item.id == 12 and item.subtype == 13 then
7424     if node.has_attribute(item, Babel.attr_hboxed) then
7425         word_string = word_string .. Babel.us_char
7426     else
7427         word_string = word_string .. ' '
7428     end
7429     word_nodes[#word_nodes+1] = item
7430
7431     -- Ignore leading unrecognized nodes, too.
7432 elseif word_string ~= '' then
7433     word_string = word_string .. Babel.us_char
7434     word_nodes[#word_nodes+1] = item -- Will be ignored
7435 end
7436
7437 item = item.next
7438 end
7439
7440 -- Here and above we remove some trailing chars but not the
7441 -- corresponding nodes. But they aren't accessed.
7442 if word_string:sub(-1) == ' ' then
7443     word_string = word_string:sub(1,-2)
7444 end
7445 if Babel.show_transforms then texio.write_nl(word_string) end
7446 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7447 return word_string, word_nodes, item, lang
7448 end
7449
7450 Babel.fetch_subtext[1] = function(head)
7451     local word_string = ''
7452     local word_nodes = {}
7453     local lang
7454     local item = head
7455     local inmath = false
7456
7457     while item do
7458
7459         if item.id == 11 then
7460             inmath = (item.subtype == 0)

```

```

7461     end
7462
7463     if inmath then
7464         -- pass
7465
7466     elseif item.id == 29 then
7467         if item.lang == lang or lang == nil then
7468             lang = lang or item.lang
7469             if node.has_attribute(item, Babel.attr_hboxed) then
7470                 word_string = word_string .. Babel.us_char
7471             elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7472                 word_string = word_string .. Babel.us_char
7473             else
7474                 word_string = word_string .. unicode.utf8.char(item.char)
7475             end
7476             word_nodes[#word_nodes+1] = item
7477         else
7478             break
7479         end
7480
7481     elseif item.id == 7 and item.subtype == 2 then
7482         if node.has_attribute(item, Babel.attr_hboxed) then
7483             word_string = word_string .. Babel.us_char
7484         else
7485             word_string = word_string .. '='
7486         end
7487         word_nodes[#word_nodes+1] = item
7488
7489     elseif item.id == 7 and item.subtype == 3 then
7490         if node.has_attribute(item, Babel.attr_hboxed) then
7491             word_string = word_string .. Babel.us_char
7492         else
7493             word_string = word_string .. '|'
7494         end
7495         word_nodes[#word_nodes+1] = item
7496
7497         -- (1) Go to next word if nothing was found, and (2) implicitly
7498         -- remove leading USs.
7499         elseif word_string == '' then
7500             -- pass
7501
7502         -- This is the responsible for splitting by words.
7503         elseif (item.id == 12 and item.subtype == 13) then
7504             break
7505
7506         else
7507             word_string = word_string .. Babel.us_char
7508             word_nodes[#word_nodes+1] = item -- Will be ignored
7509         end
7510
7511         item = item.next
7512     end
7513     if Babel.show_transforms then texio.write_nl(word_string) end
7514     word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7515     return word_string, word_nodes, item, lang
7516 end
7517
7518 function Babel.pre_hyphenate_replace(head)
7519     Babel.hyphenate_replace(head, 0)
7520 end
7521
7522 function Babel.post_hyphenate_replace(head)
7523     Babel.hyphenate_replace(head, 1)

```

```

7524 end
7525
7526 Babel.us_char = string.char(31)
7527
7528 function Babel.hyphenate_replace(head, mode)
7529     local u = unicode.utf8
7530     local lbkr = Babel.linebreaking.replacements[mode]
7531     local tovalue = Babel.tovalue
7532
7533     local word_head = head
7534
7535     if Babel.show_transforms then
7536         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7537     end
7538
7539     while true do -- for each subtext block
7540
7541         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7542
7543         if Babel.debug then
7544             print()
7545             print((mode == 0) and '@@@<' or '@@@>', w)
7546         end
7547
7548         if nw == nil and w == '' then break end
7549
7550         if not lang then goto next end
7551         if not lbkr[lang] then goto next end
7552
7553         -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7554         -- loops are nested.
7555         for k=1, #lbkr[lang] do
7556             local p = lbkr[lang][k].pattern
7557             local r = lbkr[lang][k].replace
7558             local attr = lbkr[lang][k].attr or -1
7559
7560             if Babel.debug then
7561                 print('*****', p, mode)
7562             end
7563
7564             -- This variable is set in some cases below to the first *byte*
7565             -- after the match, either as found by u.match (faster) or the
7566             -- computed position based on sc if w has changed.
7567             local last_match = 0
7568             local step = 0
7569
7570             -- For every match.
7571             while true do
7572                 if Babel.debug then
7573                     print('====')
7574                 end
7575                 local new -- used when inserting and removing nodes
7576                 local dummy_node -- used by after
7577
7578                 local matches = { u.match(w, p, last_match) }
7579
7580                 if #matches < 2 then break end
7581
7582                 -- Get and remove empty captures (with ()'s, which return a
7583                 -- number with the position), and keep actual captures
7584                 -- (from (...)), if any, in matches.
7585                 local first = table.remove(matches, 1)
7586                 local last = table.remove(matches, #matches)

```

```

7587 -- Non re-fetched substrings may contain \31, which separates
7588 -- subsubstrings.
7589 if string.find(w:sub(first, last-1), Babel.us_char) then break end
7590
7591 local save_last = last -- with A()BC()D, points to D
7592
7593 -- Fix offsets, from bytes to unicode. Explained above.
7594 first = u.len(w:sub(1, first-1)) + 1
7595 last = u.len(w:sub(1, last-1)) -- now last points to C
7596
7597 -- This loop stores in a small table the nodes
7598 -- corresponding to the pattern. Used by 'data' to provide a
7599 -- predictable behavior with 'insert' (w_nodes is modified on
7600 -- the fly), and also access to 'remove'd nodes.
7601 local sc = first-1 -- Used below, too
7602 local data_nodes = {}
7603
7604 local enabled = true
7605 for q = 1, last-first+1 do
7606     data_nodes[q] = w_nodes[sc+q]
7607     if enabled
7608         and attr > -1
7609         and not node.has_attribute(data_nodes[q], attr)
7610     then
7611         enabled = false
7612     end
7613 end
7614
7615 -- This loop traverses the matched substring and takes the
7616 -- corresponding action stored in the replacement list.
7617 -- sc = the position in substr nodes / string
7618 -- rc = the replacement table index
7619 local rc = 0
7620
7621 ----- TODO. dummy_node?
7622 while rc < last-first+1 or dummy_node do -- for each replacement
7623     if Babel.debug then
7624         print('.....', rc + 1)
7625     end
7626     sc = sc + 1
7627     rc = rc + 1
7628
7629     if Babel.debug then
7630         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7631         local ss = ''
7632         for itt in node.traverse(head) do
7633             if itt.id == 29 then
7634                 ss = ss .. unicode.utf8.char(itt.char)
7635             else
7636                 ss = ss .. '{' .. itt.id .. '}'
7637             end
7638         end
7639         print('*****', ss)
7640
7641     end
7642
7643     local crep = r[rc]
7644     local item = w_nodes[sc]
7645     local item_base = item
7646     local placeholder = Babel.us_char
7647     local d
7648
7649     if crep and crep.data then

```

```

7650         item_base = data_nodes[crep.data]
7651     end
7652
7653     if crep then
7654         step = crep.step or step
7655     end
7656
7657     if crep and crep.after then
7658         crep.insert = true
7659         if dummy_node then
7660             item = dummy_node
7661         else -- TODO. if there is a node after?
7662             d = node.copy(item_base)
7663             head, item = node.insert_after(head, item, d)
7664             dummy_node = item
7665         end
7666     end
7667
7668     if crep and not crep.after and dummy_node then
7669         node.remove(head, dummy_node)
7670         dummy_node = nil
7671     end
7672
7673     if not enabled then
7674         last_match = save_last
7675         goto next
7676
7677     elseif crep and next(crep) == nil then -- = {}
7678         if step == 0 then
7679             last_match = save_last -- Optimization
7680         else
7681             last_match = utf8.offset(w, sc+step)
7682         end
7683         goto next
7684
7685     elseif crep == nil or crep.remove then
7686         node.remove(head, item)
7687         table.remove(w_nodes, sc)
7688         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7689         sc = sc - 1 -- Nothing has been inserted.
7690         last_match = utf8.offset(w, sc+1+step)
7691         goto next
7692
7693     elseif crep and crep.kashida then
7694         node.set_attribute(item,
7695             Babel.attr_kashida,
7696             crep.kashida)
7697         last_match = utf8.offset(w, sc+1+step)
7698         goto next
7699
7700     elseif crep and crep.string then
7701         local str = crep.string(matches)
7702         if str == '' then -- Gather with nil
7703             node.remove(head, item)
7704             table.remove(w_nodes, sc)
7705             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7706             sc = sc - 1 -- Nothing has been inserted.
7707         else
7708             local loop_first = true
7709             for s in string.utfvalues(str) do
7710                 d = node.copy(item_base)
7711                 d.char = s
7712                 if loop_first then

```

```

7713         loop_first = false
7714         head, new = node.insert_before(head, item, d)
7715         if sc == 1 then
7716             word_head = head
7717         end
7718         w_nodes[sc] = d
7719         w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7720     else
7721         sc = sc + 1
7722         head, new = node.insert_before(head, item, d)
7723         table.insert(w_nodes, sc, new)
7724         w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7725     end
7726     if Babel.debug then
7727         print('.....', 'str')
7728         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7729     end
7730     end -- for
7731     node.remove(head, item)
7732 end -- if ''
7733 last_match = utf8.offset(w, sc+1+step)
7734 goto next
7735
7736 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7737     d = node.new(7, 3) -- (disc, regular)
7738     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7739     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7740     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7741     d.attr = item_base.attr
7742     if crep.pre == nil then -- TeXbook p96
7743         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7744     else
7745         d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7746     end
7747     placeholder = '|'
7748     head, new = node.insert_before(head, item, d)
7749
7750 elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7751     -- ERROR
7752
7753 elseif crep and crep.penalty then
7754     d = node.new(14, 0) -- (penalty, userpenalty)
7755     d.attr = item_base.attr
7756     d.penalty = tovalue(crep.penalty)
7757     head, new = node.insert_before(head, item, d)
7758
7759 elseif crep and crep.space then
7760     -- 655360 = 10 pt = 10 * 65536 sp
7761     d = node.new(12, 13) -- (glue, spaceskip)
7762     local quad = font.getfont(item_base.font).size or 655360
7763     node.setglue(d, tovalue(crep.space[1]) * quad,
7764                 tovalue(crep.space[2]) * quad,
7765                 tovalue(crep.space[3]) * quad)
7766     if mode == 0 then
7767         placeholder = ' '
7768     end
7769     head, new = node.insert_before(head, item, d)
7770
7771 elseif crep and crep.norule then
7772     -- 655360 = 10 pt = 10 * 65536 sp
7773     d = node.new(2, 3) -- (rule, empty) = \no*rule
7774     local quad = font.getfont(item_base.font).size or 655360
7775     d.width = tovalue(crep.norule[1]) * quad

```

```

7776         d.height = tovalue(crep.norule[2]) * quad
7777         d.depth = tovalue(crep.norule[3]) * quad
7778         head, new = node.insert_before(head, item, d)
7779
7780     elseif crep and crep.spacefactor then
7781         d = node.new(12, 13) -- (glue, spaceskip)
7782         local base_font = font.getfont(item_base.font)
7783         node.setglue(d,
7784             tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7785             tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7786             tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7787         if mode == 0 then
7788             placeholder = ' '
7789         end
7790         head, new = node.insert_before(head, item, d)
7791
7792     elseif mode == 0 and crep and crep.space then
7793         -- ERROR
7794
7795     elseif crep and crep.kern then
7796         d = node.new(13, 1) -- (kern, user)
7797         local quad = font.getfont(item_base.font).size or 655360
7798         d.attr = item_base.attr
7799         d.kern = tovalue(crep.kern) * quad
7800         head, new = node.insert_before(head, item, d)
7801
7802     elseif crep and crep.node then
7803         d = node.new(crep.node[1], crep.node[2])
7804         d.attr = item_base.attr
7805         head, new = node.insert_before(head, item, d)
7806
7807     end -- i.e., replacement cases
7808
7809     -- Shared by disc, space(factor), kern, node and penalty.
7810     if sc == 1 then
7811         word_head = head
7812     end
7813     if crep.insert then
7814         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7815         table.insert(w_nodes, sc, new)
7816         last = last + 1
7817     else
7818         w_nodes[sc] = d
7819         node.remove(head, item)
7820         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7821     end
7822
7823     last_match = utf8.offset(w, sc+1+step)
7824
7825     ::next::
7826
7827 end -- for each replacement
7828
7829 if Babel.show_transforms then texio.write_nl('> ' .. w) end
7830 if Babel.debug then
7831     print('.....', '/')
7832     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7833 end
7834
7835 if dummy_node then
7836     node.remove(head, dummy_node)
7837     dummy_node = nil
7838 end

```

```

7839
7840     end -- for match
7841
7842     end -- for patterns
7843
7844     ::next::
7845     word_head = nw
7846 end -- for substring
7847
7848 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7849 return head
7850 end
7851
7852 -- This table stores capture maps, numbered consecutively
7853 Babel.capture_maps = {}
7854
7855 function Babel.esc_hex_to_char(h)
7856   if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7857     tex.getcatcode(tonumber(h, 16)) ~= 12 then
7858     return string.format([[\\Uchar"%X ]], tonumber(h,16))
7859   else
7860     return unicode.utf8.char(tonumber(h, 16))
7861   end
7862 end
7863
7864 -- The following functions belong to the next macro
7865 function Babel.capture_func(key, cap)
7866   local ret = "[[" .. cap:gsub('{{([0-9])}}', "[ ]..m[%1]..[[[" .. "]"")
7867   local cnt
7868   local u = unicode.utf8
7869   ret = u.gsub(ret, '{{(%x%x%x%x+)}}', '\\x01%1\\x04')
7870   ret, cnt = ret:gsub('{{([0-9])|([^\]|+)|(.-)}}', Babel.capture_func_map)
7871   ret = u.gsub(ret, '\\x01(%x%x%x%x+)\\x04', Babel.esc_hex_to_char)
7872   ret = ret:gsub("%[%[%]%]%.%", '')
7873   ret = ret:gsub("%.%.%[%[%]%]", '')
7874   return key .. [[=function(m) return ]] .. ret .. [[ end]]
7875 end
7876
7877 function Babel.capt_map(from, mapno)
7878   return Babel.capture_maps[mapno][from] or from
7879 end
7880
7881 -- Handle the {n|abc|ABC} syntax in captures
7882 function Babel.capture_func_map(capno, from, to)
7883   local u = unicode.utf8
7884   from = u.gsub(from, '\\x01(%x%x%x%x+)\\x04',
7885     function (n)
7886       return u.char(tonumber(n, 16))
7887     end)
7888   to = u.gsub(to, '\\x01(%x%x%x%x+)\\x04',
7889     function (n)
7890       return u.char(tonumber(n, 16))
7891     end)
7892   local froms = {}
7893   for s in string.utfcharacters(from) do
7894     table.insert(froms, s)
7895   end
7896   local cnt = 1
7897   table.insert(Babel.capture_maps, {})
7898   local mlen = table.getn(Babel.capture_maps)
7899   for s in string.utfcharacters(to) do
7900     Babel.capture_maps[mlen][froms[cnt]] = s
7901     cnt = cnt + 1

```

```

7902 end
7903 return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7904         (mlen) .. ").." .. "["
7905 end
7906
7907 -- Create/Extend reversed sorted list of kashida weights:
7908 function Babel.capture_kashida(key, wt)
7909     wt = tonumber(wt)
7910     if Babel.kashida_wts then
7911         for p, q in ipairs(Babel.kashida_wts) do
7912             if wt == q then
7913                 break
7914             elseif wt > q then
7915                 table.insert(Babel.kashida_wts, p, wt)
7916                 break
7917             elseif table.getn(Babel.kashida_wts) == p then
7918                 table.insert(Babel.kashida_wts, wt)
7919             end
7920         end
7921     else
7922         Babel.kashida_wts = { wt }
7923     end
7924     return 'kashida = ' .. wt
7925 end
7926
7927 function Babel.capture_node(id, subtype)
7928     local sbt = 0
7929     for k, v in pairs(node.subtypes(id)) do
7930         if v == subtype then sbt = k end
7931     end
7932     return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7933 end
7934
7935 -- Experimental: applies prehyphenation transforms to a string (letters
7936 -- and spaces).
7937 function Babel.string_prehyphenation(str, locale)
7938     local n, head, last, res
7939     head = node.new(8, 0) -- dummy (hack just to start)
7940     last = head
7941     for s in string.utfvalues(str) do
7942         if s == 20 then
7943             n = node.new(12, 0)
7944         else
7945             n = node.new(29, 0)
7946             n.char = s
7947         end
7948         node.set_attribute(n, Babel.attr_locale, locale)
7949         last.next = n
7950         last = n
7951     end
7952     head = Babel.hyphenate_replace(head, 0)
7953     res = ''
7954     for n in node.traverse(head) do
7955         if n.id == 12 then
7956             res = res .. ' '
7957         elseif n.id == 29 then
7958             res = res .. unicode.utf8.char(n.char)
7959         end
7960     end
7961     tex.print(res)
7962 end
7963 (/transforms)

```

11.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```
% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%
```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In `babel` the `dir` is set by a higher protocol based on the language/script, which in turn sets the correct `dir` (<l>, <r> or <al>).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don't think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where `luatex` excels, because everything related to bidi writing is under our control.

```
7964 (*basic-r)
7965 Babel.bidi_enabled = true
7966
7967 require('babel-data-bidi.lua')
7968
7969 local characters = Babel.characters
7970 local ranges = Babel.ranges
7971
7972 local DIR = node.id("dir")
7973
7974 local function dir_mark(head, from, to, outer)
7975   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7976   local d = node.new(DIR)
7977   d.dir = '+' .. dir
7978   node.insert_before(head, from, d)
7979   d = node.new(DIR)
7980   d.dir = '-' .. dir
7981   node.insert_after(head, to, d)
7982 end
7983
7984 function Babel.bidi(head, ispar)
7985   local first_n, last_n      -- first and last char with nums
7986   local last_es              -- an auxiliary 'last' used with nums
7987   local first_d, last_d      -- first and last char in L/R block
7988   local dir, dir_real
```

Next also depends on script/lang (<al>/<r>). To be set by babel. tex.pardir is dangerous, could be (re)set but it should be changed only in vmode. There are two strong's – strong = l/al/r and strong_lr = l/r (there must be a better way):

```

7989 local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7990 local strong_lr = (strong == 'l') and 'l' or 'r'
7991 local outer = strong
7992
7993 local new_dir = false
7994 local first_dir = false
7995 local inmath = false
7996
7997 local last_lr
7998
7999 local type_n = ''
8000
8001 for item in node.traverse(head) do
8002
8003   -- three cases: glyph, dir, otherwise
8004   if item.id == node.id'glyph'
8005     or (item.id == 7 and item.subtype == 2) then
8006
8007     local itemchar
8008     if item.id == 7 and item.subtype == 2 then
8009       itemchar = item.replace.char
8010     else
8011       itemchar = item.char
8012     end
8013     local chardata = characters[itemchar]
8014     dir = chardata and chardata.d or nil
8015     if not dir then
8016       for nn, et in ipairs(ranges) do
8017         if itemchar < et[1] then
8018           break
8019         elseif itemchar <= et[2] then
8020           dir = et[3]
8021           break
8022         end
8023       end
8024     end
8025     dir = dir or 'l'
8026     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

8027   if new_dir then
8028     attr_dir = 0
8029     for at in node.traverse(item.attr) do
8030       if at.number == Babel.attr_dir then
8031         attr_dir = at.value & 0x3
8032       end
8033     end
8034     if attr_dir == 1 then
8035       strong = 'r'
8036     elseif attr_dir == 2 then
8037       strong = 'al'
8038     else
8039       strong = 'l'
8040     end
8041     strong_lr = (strong == 'l') and 'l' or 'r'
8042     outer = strong_lr

```

```

8043     new_dir = false
8044     end
8045
8046     if dir == 'nsm' then dir = strong end          -- W1

```

Numbers. The dual $\langle al \rangle / \langle r \rangle$ system for R is somewhat cumbersome.

```

8047     dir_real = dir          -- We need dir_real to set strong below
8048     if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no $\langle en \rangle$ $\langle et \rangle$ $\langle es \rangle$ if strong == $\langle al \rangle$, only $\langle an \rangle$. Therefore, there are not $\langle et \rangle$ nor $\langle en \rangle$, W5 can be ignored, and W6 applied:

```

8049     if strong == 'al' then
8050         if dir == 'en' then dir = 'an' end          -- W2
8051         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8052         strong_lr = 'r'                             -- W3
8053     end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8054     elseif item.id == node.id'dir' and not inmath then
8055         new_dir = true
8056         dir = nil
8057     elseif item.id == node.id'math' then
8058         inmath = (item.subtype == 0)
8059     else
8060         dir = nil          -- Not a char
8061     end

```

Numbers in R mode. A sequence of $\langle en \rangle$, $\langle et \rangle$, $\langle an \rangle$, $\langle es \rangle$ and $\langle cs \rangle$ is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only $\langle an \rangle$ is relevant if $\langle al \rangle$.

```

8062     if dir == 'en' or dir == 'an' or dir == 'et' then
8063         if dir ~= 'et' then
8064             type_n = dir
8065         end
8066         first_n = first_n or item
8067         last_n = last_es or item
8068         last_es = nil
8069     elseif dir == 'es' and last_n then -- W3+W6
8070         last_es = item
8071     elseif dir == 'cs' then          -- it's right - do nothing
8072     elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8073         if strong_lr == 'r' and type_n ~= '' then
8074             dir_mark(head, first_n, last_n, 'r')
8075         elseif strong_lr == 'l' and first_d and type_n == 'an' then
8076             dir_mark(head, first_n, last_n, 'r')
8077             dir_mark(head, first_d, last_d, outer)
8078             first_d, last_d = nil, nil
8079         elseif strong_lr == 'l' and type_n ~= '' then
8080             last_d = last_n
8081         end
8082         type_n = ''
8083         first_n, last_n = nil, nil
8084     end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8085     if dir == 'l' or dir == 'r' then
8086         if dir ~= outer then
8087             first_d = first_d or item
8088             last_d = item

```

```

8089     elseif first_d and dir ~= strong_lr then
8090         dir_mark(head, first_d, last_d, outer)
8091         first_d, last_d = nil, nil
8092     end
8093 end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it’s clearly <r> and <l>, resp’tly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn’t hurt.

```

8094     if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8095         item.char = characters[item.char] and
8096             characters[item.char].m or item.char
8097     elseif (dir or new_dir) and last_lr ~= item then
8098         local mir = outer .. strong_lr .. (dir or outer)
8099         if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8100             for ch in node.traverse(node.next(last_lr)) do
8101                 if ch == item then break end
8102                 if ch.id == node.id'glyph' and characters[ch.char] then
8103                     ch.char = characters[ch.char].m or ch.char
8104                 end
8105             end
8106         end
8107     end

```

Save some values for the next iteration. If the current node is ‘dir’, open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8108     if dir == 'l' or dir == 'r' then
8109         last_lr = item
8110         strong = dir_real          -- Don't search back - best save now
8111         strong_lr = (strong == 'l') and 'l' or 'r'
8112     elseif new_dir then
8113         last_lr = nil
8114     end
8115 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8116     if last_lr and outer == 'r' then
8117         for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8118             if characters[ch.char] then
8119                 ch.char = characters[ch.char].m or ch.char
8120             end
8121         end
8122     end
8123     if first_n then
8124         dir_mark(head, first_n, last_n, outer)
8125     end
8126     if first_d then
8127         dir_mark(head, first_d, last_d, outer)
8128     end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8129     return node.prev(head) or head
8130 end
8131 (/basic-r)

```

And here the Lua code for bidi=basic:

```

8132 (*basic)
8133 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8134
8135 Babel.fontmap = Babel.fontmap or {}
8136 Babel.fontmap[0] = {}          -- l

```

```

8137 Babel.fontmap[1] = {}      -- r
8138 Babel.fontmap[2] = {}      -- al/an
8139
8140 -- To cancel mirroring. Also OML, OMS, U?
8141 Babel.symbol_fonts = Babel.symbol_fonts or {}
8142 Babel.symbol_fonts[font.id('tenln')] = true
8143 Babel.symbol_fonts[font.id('tenlnw')] = true
8144 Babel.symbol_fonts[font.id('tencirc')] = true
8145 Babel.symbol_fonts[font.id('tencircw')] = true
8146
8147 Babel.bidi_enabled = true
8148 Babel.mirroring_enabled = true
8149
8150 require('babel-data-bidi.lua')
8151
8152 local characters = Babel.characters
8153 local ranges = Babel.ranges
8154
8155 local DIR = node.id('dir')
8156 local GLYPH = node.id('glyph')
8157
8158 local function insert_implicit(head, state, outer)
8159   local new_state = state
8160   if state.sim and state.eim and state.sim ~= state.eim then
8161     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8162     local d = node.new(DIR)
8163     d.dir = '+' .. dir
8164     node.insert_before(head, state.sim, d)
8165     local d = node.new(DIR)
8166     d.dir = '-' .. dir
8167     node.insert_after(head, state.eim, d)
8168   end
8169   new_state.sim, new_state.eim = nil, nil
8170   return head, new_state
8171 end
8172
8173 local function insert_numeric(head, state)
8174   local new
8175   local new_state = state
8176   if state.san and state.ean and state.san ~= state.ean then
8177     local d = node.new(DIR)
8178     d.dir = '+TLT'
8179     _, new = node.insert_before(head, state.san, d)
8180     if state.san == state.sim then state.sim = new end
8181     local d = node.new(DIR)
8182     d.dir = '-TLT'
8183     _, new = node.insert_after(head, state.ean, d)
8184     if state.ean == state.eim then state.eim = new end
8185   end
8186   new_state.san, new_state.ean = nil, nil
8187   return head, new_state
8188 end
8189
8190 local function glyph_not_symbol_font(node)
8191   if node.id == GLYPH then
8192     return not Babel.symbol_fonts[node.font]
8193   else
8194     return false
8195   end
8196 end
8197
8198 -- TODO - \hbox with an explicit dir can lead to wrong results
8199 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt

```

```

8200 -- was made to improve the situation, but the problem is the 3-dir
8201 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8202 -- well.
8203
8204 function Babel.bidi(head, ispar, hdir)
8205   local d    -- d is used mainly for computations in a loop
8206   local prev_d = ''
8207   local new_d = false
8208
8209   local nodes = {}
8210   local outer_first = nil
8211   local inmath = false
8212
8213   local glue_d = nil
8214   local glue_i = nil
8215
8216   local has_en = false
8217   local first_et = nil
8218
8219   local has_hyperlink = false
8220
8221   local ATDIR = Babel.attr_dir
8222   local attr_d, temp
8223   local locale_d
8224
8225   local save_outer
8226   local locale_d = node.get_attribute(head, ATDIR)
8227   if locale_d then
8228     locale_d = locale_d & 0x3
8229     save_outer = (locale_d == 0 and 'l') or
8230                  (locale_d == 1 and 'r') or
8231                  (locale_d == 2 and 'al')
8232   elseif ispar then    -- Or error? Shouldn't happen
8233     -- when the callback is called, we are just _after_ the box,
8234     -- and the textdir is that of the surrounding text
8235     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8236   else    -- Empty box
8237     save_outer = ('TRT' == hdir) and 'r' or 'l'
8238   end
8239   local outer = save_outer
8240   local last = outer
8241   -- 'al' is only taken into account in the first, current loop
8242   if save_outer == 'al' then save_outer = 'r' end
8243
8244   local fontmap = Babel.fontmap
8245
8246   for item in node.traverse(head) do
8247
8248     -- Mask: DxxxPPTT (Done, Pardir [0-2], Textdir [0-2])
8249     locale_d = node.get_attribute(item, ATDIR)
8250     node.set_attribute(item, ATDIR, 0x80)
8251
8252     -- In what follows, #node is the last (previous) node, because the
8253     -- current one is not added until we start processing the neutrals.
8254     -- three cases: glyph, dir, otherwise
8255     if glyph_not_symbol_font(item)
8256       or (item.id == 7 and item.subtype == 2) then
8257
8258       if inmath then goto nextnode end
8259
8260       if locale_d == 0x80 then goto nextnode end
8261       locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8262

```

```

8263     local d_font = nil
8264     local item_r
8265     if item.id == 7 and item.subtype == 2 then
8266         item_r = item.replace    -- automatic discs have just 1 glyph
8267     else
8268         item_r = item
8269     end
8270
8271     local chardata = characters[item_r.char]
8272     d = chardata and chardata.d or nil
8273     if not d or d == 'nsm' then
8274         for nn, et in ipairs(ranges) do
8275             if item_r.char < et[1] then
8276                 break
8277             elseif item_r.char <= et[2] then
8278                 if not d then d = et[3]
8279                 elseif d == 'nsm' then d_font = et[3]
8280                 end
8281                 break
8282             end
8283         end
8284     end
8285     d = d or 'l'
8286
8287     -- A short 'pause' in bidi for mapfont
8288     -- %%% TODO. move if fontmap here
8289     d_font = d_font or d
8290     d_font = (d_font == 'l' and 0) or
8291             (d_font == 'nsm' and 0) or
8292             (d_font == 'r' and 1) or
8293             (d_font == 'al' and 2) or
8294             (d_font == 'an' and 2) or nil
8295     if d_font and fontmap and fontmap[d_font][item_r.font] then
8296         item_r.font = fontmap[d_font][item_r.font]
8297     end
8298
8299     if new_d then
8300         table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8301         if inmath then
8302             attr_d = 0
8303         else
8304             attr_d = locale_d & 0x3
8305         end
8306         if attr_d == 1 then
8307             outer_first = 'r'
8308             last = 'r'
8309         elseif attr_d == 2 then
8310             outer_first = 'r'
8311             last = 'al'
8312         else
8313             outer_first = 'l'
8314             last = 'l'
8315         end
8316         outer = last
8317         has_en = false
8318         first_et = nil
8319         new_d = false
8320     end
8321
8322     if glue_d then
8323         if (d == 'l' and 'l' or 'r') ~= glue_d then
8324             table.insert(nodes, {glue_i, 'on', nil})
8325         end

```

```

8326         glue_d = nil
8327         glue_i = nil
8328     end
8329
8330     elseif item.id == DIR then
8331         if inmath then goto nextnode end
8332         d = nil
8333         new_d = true
8334
8335     elseif item.id == node.id'glue' and item.subtype == 13 then
8336         if inmath then goto nextnode end
8337         glue_d = d
8338         glue_i = item
8339         d = nil
8340
8341     elseif item.id == node.id'math' then
8342         if item.subtype == 0 then -- mathon
8343             inmath = true
8344             d = 'on'
8345             table.insert(nodes, {item, 'on', outer_first})
8346             outer_first = nil
8347             goto nextnode
8348         else -- mathoff
8349             inmath = false
8350         end
8351
8352     elseif item.id == 8 and item.subtype == 19 then
8353         has_hyperlink = true
8354
8355     else
8356         d = nil
8357     end
8358
8359     -- AL <= EN/ET/ES      -- W2 + W3 + W6
8360     if last == 'al' and d == 'en' then
8361         d = 'an'          -- W3
8362     elseif last == 'al' and (d == 'et' or d == 'es') then
8363         d = 'on'          -- W6
8364     end
8365
8366     -- EN + CS/ES + EN      -- W4
8367     if d == 'en' and #nodes >= 2 then
8368         if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8369             and nodes[#nodes-1][2] == 'en' then
8370             nodes[#nodes][2] = 'en'
8371         end
8372     end
8373
8374     -- AN + CS + AN        -- W4 too, because uax9 mixes both cases
8375     if d == 'an' and #nodes >= 2 then
8376         if (nodes[#nodes][2] == 'cs')
8377             and nodes[#nodes-1][2] == 'an' then
8378             nodes[#nodes][2] = 'an'
8379         end
8380     end
8381
8382     -- ET/EN                -- W5 + W7->l / W6->on
8383     if d == 'et' then
8384         first_et = first_et or (#nodes + 1)
8385     elseif d == 'en' then
8386         has_en = true
8387         first_et = first_et or (#nodes + 1)
8388     elseif first_et then -- d may be nil here !

```

```

8389     if has_en then
8390         if last == 'l' then
8391             temp = 'l'      -- W7
8392         else
8393             temp = 'en'     -- W5
8394         end
8395     else
8396         temp = 'on'        -- W6
8397     end
8398     for e = first_et, #nodes do
8399         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8400     end
8401     first_et = nil
8402     has_en = false
8403 end
8404
8405 -- Force mathdir in math if ON (currently works as expected only
8406 -- with 'l')
8407
8408 if inmath and d == 'on' then
8409     d = ('TRT' == tex.mathdir) and 'r' or 'l'
8410 end
8411
8412 if d then
8413     if d == 'al' then
8414         d = 'r'
8415         last = 'al'
8416     elseif d == 'l' or d == 'r' then
8417         last = d
8418     end
8419     prev_d = d
8420     table.insert(nodes, {item, d, outer_first})
8421 end
8422
8423 outer_first = nil
8424
8425 ::nextnode::
8426
8427 end -- for each node
8428
8429 -- TODO -- repeated here in case EN/ET is the last node. Find a
8430 -- better way of doing things:
8431 if first_et then      -- dir may be nil here !
8432     if has_en then
8433         if last == 'l' then
8434             temp = 'l'      -- W7
8435         else
8436             temp = 'en'     -- W5
8437         end
8438     else
8439         temp = 'on'        -- W6
8440     end
8441     for e = first_et, #nodes do
8442         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8443     end
8444 end
8445
8446 -- dummy node, to close things
8447 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8448
8449 ----- NEUTRAL -----
8450
8451 outer = save_outer

```

```

8452 last = outer
8453
8454 local first_on = nil
8455
8456 for q = 1, #nodes do
8457     local item
8458
8459     local outer_first = nodes[q][3]
8460     outer = outer_first or outer
8461     last = outer_first or last
8462
8463     local d = nodes[q][2]
8464     if d == 'an' or d == 'en' then d = 'r' end
8465     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8466
8467     if d == 'on' then
8468         first_on = first_on or q
8469     elseif first_on then
8470         if last == d then
8471             temp = d
8472         else
8473             temp = outer
8474         end
8475         for r = first_on, q - 1 do
8476             nodes[r][2] = temp
8477             item = nodes[r][1] -- MIRRORING
8478             if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8479                 and temp == 'r' and characters[item.char] then
8480                 local font_mode = ''
8481                 if item.font > 0 and font.fonts[item.font].properties then
8482                     font_mode = font.fonts[item.font].properties.mode
8483                 end
8484                 if font_mode ~= 'harf' and font_mode ~= 'plug' then
8485                     item.char = characters[item.char].m or item.char
8486                 end
8487             end
8488         end
8489         first_on = nil
8490     end
8491
8492     if d == 'r' or d == 'l' then last = d end
8493 end
8494
8495 ----- IMPLICIT, REORDER -----
8496
8497 outer = save_outer
8498 last = outer
8499
8500 local state = {}
8501 state.has_r = false
8502
8503 for q = 1, #nodes do
8504
8505     local item = nodes[q][1]
8506
8507     outer = nodes[q][3] or outer
8508
8509     local d = nodes[q][2]
8510
8511     if d == 'nsm' then d = last end -- W1
8512     if d == 'en' then d = 'an' end
8513     local isdir = (d == 'r' or d == 'l')
8514

```

```

8515     if outer == 'l' and d == 'an' then
8516         state.san = state.san or item
8517         state.ean = item
8518     elseif state.san then
8519         head, state = insert_numeric(head, state)
8520     end
8521
8522     if outer == 'l' then
8523         if d == 'an' or d == 'r' then      -- im -> implicit
8524             if d == 'r' then state.has_r = true end
8525             state.sim = state.sim or item
8526             state.eim = item
8527         elseif d == 'l' and state.sim and state.has_r then
8528             head, state = insert_implicit(head, state, outer)
8529         elseif d == 'l' then
8530             state.sim, state.eim, state.has_r = nil, nil, false
8531         end
8532     else
8533         if d == 'an' or d == 'l' then
8534             if nodes[q][3] then -- nil except after an explicit dir
8535                 state.sim = item -- so we move sim 'inside' the group
8536             else
8537                 state.sim = state.sim or item
8538             end
8539             state.eim = item
8540         elseif d == 'r' and state.sim then
8541             head, state = insert_implicit(head, state, outer)
8542         elseif d == 'r' then
8543             state.sim, state.eim = nil, nil
8544         end
8545     end
8546
8547     if isdir then
8548         last = d      -- Don't search back - best save now
8549     elseif d == 'on' and state.san then
8550         state.san = state.san or item
8551         state.ean = item
8552     end
8553
8554 end
8555
8556 head = node.prev(head) or head
8557 % \end{macrocode}
8558 %
8559 % Now direction nodes has been distributed with relation to characters
8560 % and spaces, we need to take into account \TeX-specific elements in
8561 % the node list, to move them at an appropriate place. Firstly, with
8562 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8563 % that the latter are still discardable.
8564 %
8565 % \begin{macrocode}
8566 --- FIXES ---
8567 if has_hyperlink then
8568     local flag, linking = 0, 0
8569     for item in node.traverse(head) do
8570         if item.id == DIR then
8571             if item.dir == '+TRT' or item.dir == '+TLT' then
8572                 flag = flag + 1
8573             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8574                 flag = flag - 1
8575             end
8576         elseif item.id == 8 and item.subtype == 19 then
8577             linking = flag

```

```

8578     elseif item.id == 8 and item.subtype == 20 then
8579         if linking > 0 then
8580             if item.prev.id == DIR and
8581                 (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8582                 d = node.new(DIR)
8583                 d.dir = item.prev.dir
8584                 node.remove(head, item.prev)
8585                 node.insert_after(head, item, d)
8586             end
8587         end
8588         linking = 0
8589     end
8590 end
8591 end
8592
8593 for item in node.traverse_id(10, head) do
8594     local p = item
8595     local flag = false
8596     while p.prev and p.prev.id == 14 do
8597         flag = true
8598         p = p.prev
8599     end
8600     if flag then
8601         node.insert_before(head, p, node.copy(item))
8602         node.remove(head, item)
8603     end
8604 end
8605
8606 return head
8607 end
8608
8608 function Babel.unset_atdir(head)
8609     local ATDIR = Babel.attr_dir
8610     for item in node.traverse(head) do
8611         node.set_attribute(item, ATDIR, 0x80)
8612     end
8613     return head
8614 end

```

The following code has been contributed by Udi Fogiel. It fixes misplaced glues when combined with penalties at line breaks.

```

8615 local GLUE      = node.id'glue'
8616 local PENALTY    = node.id'penalty'
8617 local HLIST      = node.id'hlist'
8618 local LINE       = table.swapped(node.subtypes("hlist")).line
8619
8620 local characters = Babel.characters
8621 local ranges     = Babel.ranges
8622
8623 -- L1's whitespace set: WS + isolate formats + (per the UBA reference) the
8624 -- explicit embedding/override formats + BN.
8625 -- Babel does not use embedding/override/isolates, but well, I copy pasted it...
8626 local ll_whitespace = {
8627     ws = true, bn = true,
8628     lre = true, rle = true, lro = true, rlo = true, pdf = true,
8629     lri = true, rli = true, fsi = true, pdi = true,
8630 }
8631
8632 local function bidi_class(c)
8633     local cd = characters[c]
8634     local d = cd and cd.d
8635     if not d then
8636         for _, et in ipairs(ranges) do
8637             if c < et[1] then

```

```

8638         break
8639     elseif c <= et[2] then
8640         d = et[3]
8641         break
8642     end
8643 end
8644 end
8645 return d or 'l'
8646 end
8647
8648 -- Nodes we step over as part of a trailing-whitespace run: glue,
8649 -- penalties, and a whitespace glyph.
8650 local function skippable(n)
8651     local id = n.id
8652     if id == GLUE or id == PENALTY then
8653         return true
8654     elseif id == GLYPH then
8655         return ll_whitespace[bidi_class(n.char)] == true
8656     end
8657     return false
8658 end
8659
8660 local function is_enddir(n)
8661     return n.id == DIR and n.subtype == 1
8662 end
8663
8664 -- Rule L1, clause 4, for one line: pull its trailing whitespace out past the
8665 -- closing dir run so it renders in the box (base) direction.
8666 local function reset_line(head)
8667     local last = node.slide(head) -- tail, fixes prev links
8668     -- 1. skip the tail that follows the closing dir run (rightskip/parfillskip/...)
8669     local q = last
8670     while q and skippable(q) do q = q.prev end
8671     if not (q and is_enddir(q)) then return head end
8672     local d_last = q
8673     -- 2. step back over the whole closing dir run
8674     while q and q.id == DIR do q = q.prev end
8675     -- 3. what precedes the dirs must be the trailing whitespace run
8676     if not (q and skippable(q)) then return head end
8677     local w_last = q
8678     while q and skippable(q) do q = q.prev end
8679     local before_w = q
8680     if not before_w then return head end -- whole line is whitespace; leave it
8681     local w_first = before_w.next
8682     local d_first = w_last.next
8683     local after_d = d_last.next
8684     -- detach W = [w_first .. w_last]
8685     before_w.next = d_first
8686     if d_first then d_first.prev = before_w end
8687     -- reinsert W after the dir run (outside every dir => box/base direction)
8688     d_last.next = w_first
8689     w_first.prev = d_last
8690     w_last.next = after_d
8691     if after_d then after_d.prev = w_last end
8692     return head
8693 end
8694
8695 function Babel.reset_lines(head)
8696     for line, subtype in node.traverse_id(HLIST, head) do
8697         if subtype == LINE then
8698             line.list = reset_line(line.list)
8699         end
8700     end

```

```

8701 return head
8702 end
8703
8704 luatexbase.add_to_callback('post_linebreak_filter',
8705   function(head, gc)
8706     if Babel.bidi_enabled then head = Babel.reset_lines(head) end
8707     return head
8708   end,
8709   'Babel.reset_lines')
8710 </basic>

```

12. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%

```

For the meaning of these codes, see the Unicode standard.

13. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```

8711 <*nil>
8712 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8713 \LdfInit{nil}{datenil}

```

When this file is read as an option, i.e., by the `\usepackage` command, nil could be an ‘unknown’ language in which case we have to make it known.

```

8714 \ifx\l@nil\undefined
8715   \newlanguage\l@nil
8716   \namedef{bbl@hyphendata@the\l@nil}{}{}% Remove warning
8717   \let\bbl@elt\relax
8718   \edef\bbl@languages{% Add it to the list of languages
8719     \bbl@languages\bbl@elt{nil}{the\l@nil}{}{}}
8720 \fi

```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```

8721 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}

```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

`\captionnil`
`\datenil`

```

8722 \let\captionnil\@empty
8723 \let\datenil\@empty

```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```

8724 \def\bbl@inidata@nil{%
8725   \bbl@elt{identification}{tag.ini}{und}%
8726   \bbl@elt{identification}{load.level}{0}%
8727   \bbl@elt{identification}{charset}{utf8}%
8728   \bbl@elt{identification}{version}{1.0}%

```

```

8729 \bbl@elt{identification}{date}{2022-05-16}%
8730 \bbl@elt{identification}{name.local}{nil}%
8731 \bbl@elt{identification}{name.english}{nil}%
8732 \bbl@elt{identification}{name.babel}{nil}%
8733 \bbl@elt{identification}{tag.bcp47}{und}%
8734 \bbl@elt{identification}{language.tag.bcp47}{und}%
8735 \bbl@elt{identification}{tag.opentype}{dflt}%
8736 \bbl@elt{identification}{script.name}{Latin}%
8737 \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8738 \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8739 \bbl@elt{identification}{level}{1}%
8740 \bbl@elt{identification}{encodings}{}%
8741 \bbl@elt{identification}{derivate}{no}}
8742 \@namedef{bbl@tbcplnil}{und}
8743 \@namedef{bbl@lbcplnil}{und}
8744 \@namedef{bbl@casingnil}{und}
8745 \@namedef{bbl@lotfnil}{dflt}
8746 \@namedef{bbl@elname@nil}{nil}
8747 \@namedef{bbl@lname@nil}{nil}
8748 \@namedef{bbl@esname@nil}{Latin}
8749 \@namedef{bbl@sname@nil}{Latin}
8750 \@namedef{bbl@sbcplnil}{Latn}
8751 \@namedef{bbl@sotfnil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8752 \ldf@finish{nil}
8753 (/nil)

```

14. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8754 ((*Compute Julian day)) ≡
8755 \def\bbl@fpmmod#1#2{(#1-#2*floor((#1)/(#2)))}
8756 \def\bbl@cs@gregleap#1{%
8757   (\bbl@fpmmod{#1}{4} == 0) &&
8758   (!((\bbl@fpmmod{#1}{100} == 0) && (\bbl@fpmmod{#1}{400} != 0)))}
8759 \def\bbl@cs@jd#1#2#3{% year, month, day
8760   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8761     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8762     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8763     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3 } }
8764 (/*Compute Julian day)

```

14.1. Islamic

The code for the Civil calendar is based on it, too.

```

8765 (*ca-islamic)
8766 <@Compute Julian day@>
8767 % == islamic (default)
8768 % Not yet implemented
8769 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

```

The Civil calendar.

```

8770 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8771   ((#3 + ceil(29.5 * (#2 - 1)) +
8772     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8773     1948439.5) - 1) }
8774 \@namedef{bbl@ca@islamic-civil++}{\bbl@ca@islamicvl{x}{+2}}

```

```

8775 \@namedef{bbl@ca@islamic-civil+}{\bbl@ca@islamicvl@x{+1}}
8776 \@namedef{bbl@ca@islamic-civil}{\bbl@ca@islamicvl@x{}}
8777 \@namedef{bbl@ca@islamic-civil-}{\bbl@ca@islamicvl@x{-1}}
8778 \@namedef{bbl@ca@islamic-civil--}{\bbl@ca@islamicvl@x{-2}}
8779 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8780   \edef\bbl@tempa{%
8781     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1}}%
8782   \edef#5{%
8783     \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8784   \edef#6{\fpeval{
8785     min(12, ceil((\bbl@tempa - (29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8786   \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```

8787 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8788 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8789 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8790 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8791 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8792 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8793 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8794 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8795 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8796 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8797 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8798 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8799 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8800 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8801 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8802 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8803 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8804 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8805 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8806 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8807 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8808 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8809 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8810 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8811 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8812 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8813 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8814 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8815 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8816 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8817 65401,65431,65460,65490,65520}
8818 \@namedef{bbl@ca@islamic-umalqura+}{\bbl@ca@islamcuqr@x{+1}}
8819 \@namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{}}
8820 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8821 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8822   \ifnum#2>2014 \ifnum#2<2038
8823     \bbl@afterfi\expandafter\@gobble
8824   \fi\fi
8825   {\bbl@error{year-out-range}{2014-2038}{}}}%
8826   \edef\bbl@tempd{\fpeval{ % (Julian) day
8827     \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8828   \count@\@ne
8829   \bbl@foreach\bbl@cs@umalqura@data{%
8830     \advance\count@\@ne
8831     \ifnum#1>\bbl@tempd\else
8832       \edef\bbl@tempe{\the\count@}%

```

```

8833 \edef\bbl@tempb{##1}%
8834 \fi}%
8835 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8836 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1 ) / 12) }}% annus
8837 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8838 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8839 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8840 \bbl@add\bbl@precalendar{%
8841 \bbl@replace\bbl@ld@calendar{-civil}}}%
8842 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8843 \bbl@replace\bbl@ld@calendar{+}}}%
8844 \bbl@replace\bbl@ld@calendar{-}}}%
8845 (/ca-islamic)

```

14.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaptations by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```

8846 (*ca-hebrew)
8847 \newcount\bbl@cntcommon
8848 \def\bbl@remainder#1#2#3{%
8849 #3=#1\relax
8850 \divide #3 by #2\relax
8851 \multiply #3 by -#2\relax
8852 \advance #3 by #1\relax}%
8853 \newif\ifbbl@divisible
8854 \def\bbl@checkifdivisible#1#2{%
8855 {\countdef\tmp=0
8856 \bbl@remainder{#1}{#2}{\tmp}%
8857 \ifnum \tmp=0
8858 \global\bbl@divisibletrue
8859 \else
8860 \global\bbl@divisiblefalse
8861 \fi}}
8862 \newif\ifbbl@gregleap
8863 \def\bbl@ifgregleap#1{%
8864 \bbl@checkifdivisible{#1}{4}%
8865 \ifbbl@divisible
8866 \bbl@checkifdivisible{#1}{100}%
8867 \ifbbl@divisible
8868 \bbl@checkifdivisible{#1}{400}%
8869 \ifbbl@divisible
8870 \bbl@gregleaptrue
8871 \else
8872 \bbl@gregleapfalse
8873 \fi
8874 \else
8875 \bbl@gregleaptrue
8876 \fi
8877 \else
8878 \bbl@gregleapfalse
8879 \fi
8880 \ifbbl@gregleap}
8881 \def\bbl@gregdayspriormonths#1#2#3{%
8882 #3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8883 181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8884 \bbl@ifgregleap{#2}%
8885 \ifnum #1 > 2
8886 \advance #3 by 1
8887 \fi
8888 \fi
8889 \global\bbl@cntcommon=#3}%

```

```

8890     #3=\bbl@cntcommon}
8891 \def\bbl@gregdaysprioryears#1#2{%
8892     {\countdef\tmpc=4
8893     \countdef\tmpb=2
8894     \tmpb=#1\relax
8895     \advance \tmpb by -1
8896     \tmpc=\tmpb
8897     \multiply \tmpc by 365
8898     #2=\tmpc
8899     \tmpc=\tmpb
8900     \divide \tmpc by 4
8901     \advance #2 by \tmpc
8902     \tmpc=\tmpb
8903     \divide \tmpc by 100
8904     \advance #2 by -\tmpc
8905     \tmpc=\tmpb
8906     \divide \tmpc by 400
8907     \advance #2 by \tmpc
8908     \global\bbl@cntcommon=#2\relax}%
8909     #2=\bbl@cntcommon}
8910 \def\bbl@absfromgreg#1#2#3#4{%
8911     {\countdef\tmpd=0
8912     #4=#1\relax
8913     \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8914     \advance #4 by \tmpd
8915     \bbl@gregdaysprioryears{#3}{\tmpd}%
8916     \advance #4 by \tmpd
8917     \global\bbl@cntcommon=#4\relax}%
8918     #4=\bbl@cntcommon}
8919 \newif\ifbbl@hebrleap
8920 \def\bbl@checkleaphebryear#1{%
8921     {\countdef\tmpa=0
8922     \countdef\tmpb=1
8923     \tmpa=#1\relax
8924     \multiply \tmpa by 7
8925     \advance \tmpa by 1
8926     \bbl@remainder{\tmpa}{19}{\tmpb}%
8927     \ifnum \tmpb < 7
8928         \global\bbl@hebrleaptrue
8929     \else
8930         \global\bbl@hebrleapfalse
8931     \fi}}
8932 \def\bbl@hebrlapsedmonths#1#2{%
8933     {\countdef\tmpa=0
8934     \countdef\tmpb=1
8935     \countdef\tmpc=2
8936     \tmpa=#1\relax
8937     \advance \tmpa by -1
8938     #2=\tmpa
8939     \divide #2 by 19
8940     \multiply #2 by 235
8941     \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8942     \tmpc=\tmpb
8943     \multiply \tmpb by 12
8944     \advance #2 by \tmpb
8945     \multiply \tmpc by 7
8946     \advance \tmpc by 1
8947     \divide \tmpc by 19
8948     \advance #2 by \tmpc
8949     \global\bbl@cntcommon=#2}%
8950     #2=\bbl@cntcommon}
8951 \def\bbl@hebrlapseddays#1#2{%
8952     {\countdef\tmpa=0

```

```

8953 \countdef\tmpb=1
8954 \countdef\tmpc=2
8955 \bbl@hebreleapsedmonths{#1}{#2}%
8956 \tmpa=#2\relax
8957 \multiply \tmpa by 13753
8958 \advance \tmpa by 5604
8959 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8960 \divide \tmpa by 25920
8961 \multiply #2 by 29
8962 \advance #2 by 1
8963 \advance #2 by \tmpa
8964 \bbl@remainder{#2}{7}{\tmpa}%
8965 \ifnum \tmpc < 19440
8966     \ifnum \tmpc < 9924
8967     \else
8968         \ifnum \tmpa=2
8969             \bbl@checkleaphebreyear{#1}% of a common year
8970             \ifbbl@hebrleap
8971                 \else
8972                     \advance #2 by 1
8973                 \fi
8974             \fi
8975         \fi
8976     \ifnum \tmpc < 16789
8977     \else
8978         \ifnum \tmpa=1
8979             \advance #1 by -1
8980             \bbl@checkleaphebreyear{#1}% at the end of leap year
8981             \ifbbl@hebrleap
8982                 \advance #2 by 1
8983             \fi
8984         \fi
8985     \fi
8986 \else
8987     \advance #2 by 1
8988 \fi
8989 \bbl@remainder{#2}{7}{\tmpa}%
8990 \ifnum \tmpa=0
8991     \advance #2 by 1
8992 \else
8993     \ifnum \tmpa=3
8994         \advance #2 by 1
8995     \else
8996         \ifnum \tmpa=5
8997             \advance #2 by 1
8998         \fi
8999     \fi
9000 \fi
9001 \global\bbl@cntcommon=#2\relax}%
9002 #2=\bbl@cntcommon}
9003 \def\bbl@daysinhebreyear#1#2{%
9004     {\countdef\tmpe=12
9005     \bbl@hebreleapseddays{#1}{\tmpe}%
9006     \advance #1 by 1
9007     \bbl@hebreleapseddays{#1}{#2}%
9008     \advance #2 by -\tmpe
9009     \global\bbl@cntcommon=#2}%
9010 #2=\bbl@cntcommon}
9011 \def\bbl@hebrdayspriormonths#1#2#3{%
9012     {\countdef\tmpf= 14
9013     #3=\ifcase #1
9014         0 \or
9015         0 \or

```

```

9016         30 \or
9017         59 \or
9018         89 \or
9019         118 \or
9020         148 \or
9021         148 \or
9022         177 \or
9023         207 \or
9024         236 \or
9025         266 \or
9026         295 \or
9027         325 \or
9028         400
9029 \fi
9030 \bbl@checkleaphebrewyear{#2}%
9031 \ifbbl@hebrleap
9032     \ifnum #1 > 6
9033         \advance #3 by 30
9034     \fi
9035 \fi
9036 \bbl@daysinhebrewyear{#2}{\tmpf}%
9037 \ifnum #1 > 3
9038     \ifnum \tmpf=353
9039         \advance #3 by -1
9040     \fi
9041     \ifnum \tmpf=383
9042         \advance #3 by -1
9043     \fi
9044 \fi
9045 \ifnum #1 > 2
9046     \ifnum \tmpf=355
9047         \advance #3 by 1
9048     \fi
9049     \ifnum \tmpf=385
9050         \advance #3 by 1
9051     \fi
9052 \fi
9053 \global\bbl@cntcommon=#3\relax}%
9054 #3=\bbl@cntcommon}
9055 \def\bbl@absfromhebr#1#2#3#4{%
9056     {#4=#1\relax
9057     \bbl@hebrdayspriormonths{#2}{#3}{#1}%
9058     \advance #4 by #1\relax
9059     \bbl@hebreleaseddays{#3}{#1}%
9060     \advance #4 by #1\relax
9061     \advance #4 by -1373429
9062     \global\bbl@cntcommon=#4\relax}%
9063 #4=\bbl@cntcommon}
9064 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
9065     {\countdef\tmpx= 17
9066     \countdef\tmpy= 18
9067     \countdef\tmpz= 19
9068     #6=#3\relax
9069     \global\advance #6 by 3761
9070     \bbl@absfromgreg{#1}{#2}{#3}{#4}%
9071     \tmpz=1 \tmpy=1
9072     \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9073     \ifnum \tmpx > #4\relax
9074         \global\advance #6 by -1
9075         \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9076     \fi
9077     \advance #4 by -\tmpx
9078     \advance #4 by 1

```

```

9079 #5=#4\relax
9080 \divide #5 by 30
9081 \loop
9082     \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
9083     \ifnum \tmpx < #4\relax
9084         \advance #5 by 1
9085         \tmpy=\tmpx
9086     \repeat
9087     \global\advance #5 by -1
9088     \global\advance #4 by -\tmpy}}
9089 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebyear
9090 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
9091 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
9092     \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
9093     \bbl@hebrfromgreg
9094     {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
9095     {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebyear}%
9096     \edef#4{\the\bbl@hebyear}%
9097     \edef#5{\the\bbl@hebrmonth}%
9098     \edef#6{\the\bbl@hebrday}}
9099 (/ca-hebrew)

```

14.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

9100 (*ca-persian)
9101 <@Compute Julian day@>
9102 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
9103     2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
9104 \def\bbl@ca@persian#1-#2-#3\@#4#5#6{%
9105     \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
9106     \ifnum\bbl@tempa>2012 \ifnum\bbl@tempe<2051
9107         \bbl@afterfi\expandafter\@gobble
9108     \fi\fi
9109     {\bbl@error{year-out-range}{2013-2050}}{}}}%
9110 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9111 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9112 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
9113 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
9114 \ifnum\bbl@tempc<\bbl@tempb
9115     \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
9116     \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9117     \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9118     \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
9119 \fi
9120 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
9121 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
9122 \edef#5{\fpeval{% set Jalali month
9123     (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
9124 \edef#6{\fpeval{% set Jalali day
9125     (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : (((#5 - 1) * 30) + 6))}}}%
9126 (/ca-persian)

```

14.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

9127 (*ca-coptic)

```

```

9128 <@Compute Julian day@>
9129 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
9130 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9131 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
9132 \edef#4{\fpeval{%
9133 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9134 \edef\bbl@tempc{\fpeval{%
9135 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
9136 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9137 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9138 (/ca-coptic)
9139 (*ca-ethiopic)
9140 <@Compute Julian day@>
9141 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%
9142 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9143 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
9144 \edef#4{\fpeval{%
9145 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9146 \edef\bbl@tempc{\fpeval{%
9147 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
9148 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9149 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9150 (/ca-ethiopic)

```

14.5. Julian

Based on [ReinDersh].

```

9151 (*ca-julian)
9152 <@Compute Julian day@>
9153 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
9154 \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9155 \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9156 \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9157 \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9158 \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9159 \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9160 \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9161 \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}%
9162 (/ca-julian)

```

The Indian National Calendar, Saka era.

```

9163 (*ca-indian)
9164 <@Compute Julian day@>
9165 \def\bbl@sakachaitra#1{%
9166 \bbl@cs@jd{#1}{3}{%
9167 \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100}!=0)
9168 ||\bbl@fpmmod{#1}{400}==0) ? 21 : 22 }}%
9169 \def\bbl@ca@indian#1-#2-#3\@@#4#5#6{%
9170 \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%
9171 \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9172 \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9173 \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9174 \edef\bbl@tempe{\fpeval{ ((\bbl@fpmmod{\bbl@tempc}{4})==0 &&
9175 \bbl@fpmmod{\bbl@tempc}{100}!=0) || \bbl@fpmmod{\bbl@tempc}{400}==0)?31:30}}%
9176 \edef#4{\fpeval{\bbl@tempc-78}}%
9177 \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9178 (\bbl@tempd-\bbl@tempe < 155 ?
9179 2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9180 7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0)}}}%
9181 \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9182 (\bbl@tempd-\bbl@tempe < 155 ?
9183 \bbl@fpmmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9184 \bbl@fpmmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}%

```

```

9185 (/ca-indian)
9186 %

```

14.6. Buddhist

That's very simple.

```

9187 (*ca-buddhist)
9188 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9189   \edef#4{\number\numexpr#1+543\relax}%
9190   \edef#5{#2}%
9191   \edef#6{#3}}
9192 (/ca-buddhist)
9193 %
9194 % \subsection{Chinese}
9195 %
9196 % Brute force, with the Julian day of first day of each month. The
9197 % table has been computed with the help of \textsf{python-lunardate} by
9198 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9199 % is 2015-2044.
9200 %
9201 % \begin{macrocode}
9202 (*ca-chinese)
9203 \ExplSyntaxOn
9204 <@Compute Julian day@>
9205 \def\bbl@ca@chinese#1-#2-#3\@@#4#5#6{%
9206   \edef\bbl@tempd{\fpeval{%
9207     \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9208   \count@ \z@
9209   \@tempcnta=2015
9210   \bbl@foreach\bbl@cs@chinese@data{%
9211     \ifnum##1>\bbl@tempd\else
9212       \advance\count@\@ne
9213       \ifnum\count@>12
9214         \count@\@ne
9215         \advance\@tempcnta\@ne\fi
9216         \bbl@xin@{,##1,},{,\bbl@cs@chinese@leap,}%
9217         \ifin@
9218           \advance\count@\m@ne
9219           \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9220         \else
9221           \edef\bbl@tempe{\the\count@}%
9222         \fi
9223         \edef\bbl@tempb{##1}%
9224         \fi}%
9225   \edef#4{\the\@tempcnta}%
9226   \edef#5{\bbl@tempe}%
9227   \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9228 \def\bbl@cs@chinese@leap{%
9229   885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9230 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9231   354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9232   768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9233   1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9234   1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9235   1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9236   2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9237   2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9238   2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9239   3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9240   3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9241   3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9242   4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9243   4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%

```

```

9244 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9245 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9246 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9247 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9248 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9249 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9250 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9251 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9252 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9253 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9254 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9255 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9256 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9257 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9258 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9259 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9260 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9261 10896,10926,10956,10986,11015,11045,11074,11103}
9262 \ExplSyntaxOff
9263 (/ca-chinese)

```

15. Support for Plain T_EX (plain.def)

15.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `localhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `blplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9264 (*bplain|blplain)
9265 \catcode`\{=1 % left brace is begin-group character
9266 \catcode`\}=2 % right brace is end-group character
9267 \catcode`\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```

9268 \openin 0 hyphen.cfg
9269 \ifeof0
9270 \else
9271   \let\input

```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that’s done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```

9272 \def\input #1 {%
9273   \let\input\input
9274   \a hyphen.cfg
9275   \let\input\input
9276 }
9277 \fi
9278 (/bplain|blplain)

```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```
9279 \bplain\la plain.tex
9280 \bplain\la lplain.tex
```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```
9281 \bplain\def\fmtname{babel-plain}
9282 \bplain\def\fmtname{babel-lplain}
```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

15.2. Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX}_{2\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9283 \def\EmulateLaTeX{}
9284 \def\@empty{}
9285 \def\loadlocalcfg#1{%
9286   \openin0#1.cfg
9287   \ifeof0
9288     \closein0
9289   \else
9290     \closein0
9291     {\immediate\write16{*****}%
9292      \immediate\write16{* Local config file #1.cfg used}%
9293      \immediate\write16{*}%
9294     }
9295     \input #1.cfg\relax
9296   \fi
9297 \endofldef}
```

15.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```
9298 \long\def\@firstofone#1{#1}
9299 \long\def\@firstoftwo#1#2{#1}
9300 \long\def\@secondoftwo#1#2{#2}
9301 \def\@nnil{\@nil}
9302 \def\@gobbletwo#1#2{}
9303 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9304 \def\@star@or@long#1{%
9305   \@ifstar
9306   {\let\l@ngrel@x\relax#1}%
9307   {\let\l@ngrel@x\long#1}}
9308 \let\l@ngrel@x\relax
9309 \def\@car#1#2\@nil{#1}
9310 \def\@cdr#1#2\@nil{#2}
9311 \let\@typeset@protect\relax
9312 \let\protected@edef\edef
9313 \long\def\@gobble#1{}
9314 \edef\@backslashchar{\expandafter\@gobble\string\}
9315 \def\strip@prefix#1>{}
9316 \def\g@addto@macro#1#2{{%
9317   \toks@\expandafter{#1#2}%
9318   \xdef#1{\the\toks@}}}
9319 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9320 \def\@nameuse#1{\csname #1\endcsname}
```

```

9321 \def\@ifundefined#1{%
9322   \expandafter\ifx\csname#1\endcsname\relax
9323     \expandafter\@firstoftwo
9324   \else
9325     \expandafter\@secondoftwo
9326   \fi}
9327 \def\@expandtwoargs#1#2#3{%
9328   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9329 \def\zap@space#1 #2{%
9330   #1%
9331   \ifx#2\@empty\else\expandafter\zap@space\fi
9332   #2}
9333 \let\bbl@trace\@gobble
9334 \def\bbl@error#1{% Implicit #2#3#4
9335   \begingroup
9336     \catcode`\=0 \catcode`\==12 \catcode`\=12
9337     \catcode`\^^M=5 \catcode`\%=14
9338     \input errbabel.def
9339   \endgroup
9340   \bbl@error{#1}}
9341 \def\bbl@warning#1{%
9342   \begingroup
9343     \newlinechar=`^^J
9344     \def\{`^^J(babel) }%
9345     \message{\{#1}%
9346   \endgroup}
9347 \let\bbl@infowarn\bbl@warning
9348 \def\bbl@info#1{%
9349   \begingroup
9350     \newlinechar=`^^J
9351     \def\{`^^J}%
9352     \wlog{#1}%
9353   \endgroup}

```

$\text{\LaTeX 2}_\epsilon$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9354 \ifx\@preamblecmds\undefined
9355   \def\@preamblecmds{}
9356 \fi
9357 \def\@onlypreamble#1{%
9358   \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9359     \@preamblecmds\do#1}}
9360 \@onlypreamble\@onlypreamble

```

Mimic $\text{\LaTeX}$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9361 \def\begindocument{%
9362   \@begindocumenthook
9363   \global\let\@begindocumenthook\@undefined
9364   \def\do##1{\global\let##1\@undefined}%
9365   \@preamblecmds
9366   \global\let\do\noexpand}

9367 \ifx\@begindocumenthook\@undefined
9368   \def\@begindocumenthook{}
9369 \fi

9370 \@onlypreamble\@begindocumenthook
9371 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\text{\LaTeX}$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endofldf`.

```

9372 \def\AtEndOfPackage#1{\g@addto@macro\@endofldf{#1}}
9373 \@onlypreamble\AtEndOfPackage
9374 \def\@endofldf{}
9375 \@onlypreamble\@endofldf

```

```

9376 \let\bbl@afterlang\@empty
9377 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9378 \catcode`\&=\z@
9379 \ifx&\if@files\@undefined
9380   \expandafter\let\csname if@files\expandafter\endcsname
9381     \csname iffalse\endcsname
9382 \fi
9383 \catcode`\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9384 \def\newcommand{\@star@or@long\new@command}
9385 \def\new@command#1{%
9386   \@testopt{\@newcommand#1}0}
9387 \def\@newcommand#1[#2]{%
9388   \@ifnextchar [{\@xargdef#1[#2]}%
9389     {\@argdef#1[#2]}}
9390 \long\def\@argdef#1[#2]#3{%
9391   \@yargdef#1\@ne{#2}{#3}}
9392 \long\def\@xargdef#1[#2][#3]#4{%
9393   \expandafter\def\expandafter#1\expandafter{%
9394     \expandafter\@protected@testopt\expandafter #1%
9395     \csname\string#1\expandafter\endcsname{#3}}}%
9396   \expandafter\@yargdef \csname\string#1\endcsname
9397   \tw@{#2}{#4}}
9398 \long\def\@yargdef#1#2#3{%
9399   \@tempcnta#3\relax
9400   \advance \@tempcnta \@ne
9401   \let\@hash@\relax
9402   \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9403   \@tempcntb #2%
9404   \@whilenum \@tempcntb < \@tempcnta
9405   \do{%
9406     \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%
9407     \advance\@tempcntb \@ne}%
9408   \let\@hash@##%
9409   \l@ngrelx\expandafter\def\expandafter#1\reserved@a}
9410 \def\providecommand{\@star@or@long\provide@command}
9411 \def\provide@command#1{%
9412   \begingroup
9413     \escapechar\m@ne\xdef\@gtempa{\string#1}%
9414   \endgroup
9415   \expandafter\ifundefined\@gtempa
9416     {\def\reserved@a{\new@command#1}}%
9417     {\let\reserved@a\relax
9418       \def\reserved@a{\new@command\reserved@a}}%
9419   \reserved@a}%
9420 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9421 \def\declare@robustcommand#1{%
9422   \edef\reserved@a{\string#1}%
9423   \def\reserved@b{#1}%
9424   \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9425   \edef#1{%
9426     \ifx\reserved@a\reserved@b
9427       \noexpand\x@protect
9428       \noexpand#1%
9429     \fi
9430     \noexpand\protect
9431     \expandafter\noexpand\csname
9432       \expandafter\@gobble\string#1 \endcsname

```

```

9433 }%
9434 \expandafter\new\command\csname
9435 \expandafter@gobble\string#1 \endcsname
9436 }
9437 \def\x@protect#1{%
9438 \ifx\protect\@typeset@protect\else
9439 \x@protect#1%
9440 \fi
9441 }
9442 \catcode\&=\z@ % Trick to hide conditionals
9443 \def\x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9444 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9445 \catcode\&=4
9446 \ifx\in@\@undefined
9447 \def\in@#1#2{%
9448 \def\in@##1##2##3\in@@{%
9449 \ifx\in@##2\in@false\else\in@true\fi}%
9450 \in@@#2#1\in@\in@@}
9451 \else
9452 \let\bbl@tempa\@empty
9453 \fi
9454 \bbl@tempa

```

$\TeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (`activegrave` and `activeacute`). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9455 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\TeX$ macro `\ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9456 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\TeX 2_{\epsilon}$ versions; just enough to make things work in plain $\TeX$ environments.

```

9457 \ifx\@tempcnta\@undefined
9458 \csname newcount\endcsname\@tempcnta\relax
9459 \fi
9460 \ifx\@tempcntb\@undefined
9461 \csname newcount\endcsname\@tempcntb\relax
9462 \fi

```

To prevent wasting two counters in $\TeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9463 \ifx\bye\@undefined
9464 \advance\count10 by -2\relax
9465 \fi
9466 \ifx\@ifnextchar\@undefined
9467 \def\@ifnextchar#1#2#3{%
9468 \let\reserved@d=#1%
9469 \def\reserved@a{#2}\def\reserved@b{#3}%
9470 \futurelet\@let@token\@ifnch}
9471 \def\@ifnch{%
9472 \ifx\@let@token\@sptoken
9473 \let\reserved@c\@xifnch
9474 \else
9475 \ifx\@let@token\reserved@d
9476 \let\reserved@c\reserved@a

```

```

9477     \else
9478     \let\reserved@c\reserved@b
9479     \fi
9480   \fi
9481   \reserved@c}
9482   \def:{\let\@sptoken= } \: % this makes \@sptoken a space token
9483   \def:{\@xifnch} \expandafter\def: {\futurelet\@let@token\@ifnch}
9484 \fi
9485 \def\@testopt#1#2{%
9486   \@ifnextchar[{\#1}{\#1{\#2}}]
9487 \def\@protected@testopt#1{%
9488   \ifx\protect\@typeset@protect
9489     \expandafter\@testopt
9490   \else
9491     \@x@protect#1%
9492   \fi}
9493 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9494   #2\relax}\fi}
9495 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9496   \else\expandafter\@gobble\fi{#1}}

```

15.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9497 \def\DeclareTextCommand{%
9498   \@dec@text@cmd\providecommand
9499 }
9500 \def\ProvideTextCommand{%
9501   \@dec@text@cmd\providecommand
9502 }
9503 \def\DeclareTextSymbol#1#2#3{%
9504   \@dec@text@cmd\chardef#1{\#2}\#3\relax
9505 }
9506 \def\@dec@text@cmd#1#2#3{%
9507   \expandafter\def\expandafter#2%
9508     \expandafter{%
9509       \csname#3-cmd\expandafter\endcsname
9510       \expandafter#2%
9511       \csname#3\string#2\endcsname
9512     }%
9513 %   \let\@ifdefinable\@rc@ifdefinable
9514   \expandafter#1\csname#3\string#2\endcsname
9515 }
9516 \def\@current@cmd#1{%
9517   \ifx\protect\@typeset@protect\else
9518     \noexpand#1\expandafter\@gobble
9519   \fi
9520 }
9521 \def\@changed@cmd#1#2{%
9522   \ifx\protect\@typeset@protect
9523     \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9524       \expandafter\ifx\csname ?\string#1\endcsname\relax
9525         \expandafter\def\csname ?\string#1\endcsname{%
9526           \@changed@x@err{#1}%
9527         }%
9528       \fi
9529       \global\expandafter\let
9530         \csname\cf@encoding \string#1\expandafter\endcsname
9531         \csname ?\string#1\endcsname
9532       \fi
9533       \csname\cf@encoding\string#1%
9534         \expandafter\endcsname
9535     \else

```

```

9536     \noexpand#1%
9537   \fi
9538 }
9539 \def\@changed@x@err#1{%
9540   \errhelp{Your command will be ignored, type <return> to proceed}%
9541   \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9542 \def\DeclareTextCommandDefault#1{%
9543   \DeclareTextCommand#1?%
9544 }
9545 \def\ProvideTextCommandDefault#1{%
9546   \ProvideTextCommand#1?%
9547 }
9548 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9549 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9550 \def\DeclareTextAccent#1#2#3{%
9551   \DeclareTextCommand#1{#2}[1]{\accent#3 ##1}
9552 }
9553 \def\DeclareTextCompositeCommand#1#2#3#4{%
9554   \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9555   \edef\reserved@b{\string##1}%
9556   \edef\reserved@c{%
9557     \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9558   \ifx\reserved@b\reserved@c
9559     \expandafter\expandafter\expandafter\ifx
9560       \expandafter\@car\reserved@a\relax\relax\@nil
9561       \@text@composite
9562     \else
9563       \edef\reserved@b##1{%
9564         \def\expandafter\noexpand
9565           \csname#2\string#1\endcsname####1{%
9566             \noexpand\@text@composite
9567             \expandafter\noexpand\csname#2\string#1\endcsname
9568             ####1\noexpand\@empty\noexpand\@text@composite
9569             {##1}%
9570           }%
9571         }%
9572       \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9573     \fi
9574     \expandafter\def\csname\expandafter\string\csname
9575       #2\endcsname\string#1-\string#3\endcsname{#4}
9576   \else
9577     \errhelp{Your command will be ignored, type <return> to proceed}%
9578     \errmessage{\string\DeclareTextCompositeCommand\space used on
9579       inappropriate command \protect#1}
9580   \fi
9581 }
9582 \def\@text@composite#1#2#3\@text@composite{%
9583   \expandafter\@text@composite@x
9584     \csname\string#1-\string#2\endcsname
9585 }
9586 \def\@text@composite@x#1#2{%
9587   \ifx#1\relax
9588     #2%
9589   \else
9590     #1%
9591   \fi
9592 }
9593 %
9594 \def\@strip@args#1:#2-#3\@strip@args{#2}
9595 \def\DeclareTextComposite#1#2#3#4{%
9596   \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9597   \bgroup
9598     \lccode`\@=#4%

```

```

9599     \lowercase{%
9600     \egroup
9601     \reserved@a @%
9602     }%
9603 }
9604 %
9605 \def\UseTextSymbol#1#2{#2}
9606 \def\UseTextAccent#1#2#3{}
9607 \def\@use@text@encoding#1{}
9608 \def\DeclareTextSymbolDefault#1#2{%
9609     \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9610 }
9611 \def\DeclareTextAccentDefault#1#2{%
9612     \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9613 }
9614 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX 2}_{\epsilon}$ method for accents for those that are known to be made active in *some* language definition file.

```

9615 \DeclareTextAccent{"}{OT1}{127}
9616 \DeclareTextAccent{'}{OT1}{19}
9617 \DeclareTextAccent{^}{OT1}{94}
9618 \DeclareTextAccent{\`}{OT1}{18}
9619 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for `PLAIN  $\text{\TeX}$` .

```

9620 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9621 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9622 \DeclareTextSymbol{\textquoteleft}{OT1}{`\'}
9623 \DeclareTextSymbol{\textquoteright}{OT1}{`\'}
9624 \DeclareTextSymbol{\i}{OT1}{16}
9625 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because `plain  $\text{\TeX}$` doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9626 \ifx\scriptsize\undefined
9627     \let\scriptsize\sevenrm
9628 \fi

```

And a few more “dummy” definitions.

```

9629 \def\language{english}%
9630 \let\bbl@opt@shorthands\@nnil
9631 \def\bbl@ifshorthand#1#2#3{#2}%
9632 \let\bbl@language@opts\@empty
9633 \let\bbl@provide@locale\relax
9634 \ifx\babeloptionstrings\undefined
9635     \let\bbl@opt@strings\@nnil
9636 \else
9637     \let\bbl@opt@strings\babeloptionstrings
9638 \fi
9639 \def\BabelStringsDefault{generic}
9640 \def\bbl@tempa{normal}
9641 \ifx\babeloptionmath\bbl@tempa
9642     \def\bbl@mathnormal{\noexpand\textormath}
9643 \fi
9644 \def\AfterBabelLanguage#1#2{}
9645 \ifx\BabelModifiers\undefined\let\BabelModifiers\relax\fi
9646 \let\bbl@afterlang\relax
9647 \def\bbl@opt@safe{BR}
9648 \ifx\@uclclist\undefined\let\@uclclist\@empty\fi
9649 \ifx\bbl@trace\undefined\def\bbl@trace#1{}\fi
9650 \expandafter\newif\csname ifbbl@single\endcsname
9651 \chardef\bbl@bidimode\z@
9652 \let\bbl@ifmetadata\@gobble
9653 \ifx\EmulateLaTeX

```

A proxy file:

```
9654 {*plain}  
9655 \input babel.def  
9656 (/plain)
```

16. Acknowledgements

In the initial stages of the development of babel, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitschuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\TeX$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\TeX$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\TeX$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\TeX$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\TeX$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\TeX$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\TeX$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus, *Tekstwijzer, een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).